

CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

BST Analysis – Running Time

Operation	BST Average Case	BST Worst Case	Sorted Array	Sorted List
find	$O(\lg n)$			
insert	$O(\lg n)$			
delete	$O(\lg n)$			
traverse	$O(n)$			

BST Analysis – Running Time

Operation	BST Average Case	BST Worst Case	Sorted Array	Sorted List
find	$O(\lg n)$	$O(n)$	$O(\lg n)$	$O(n)$
insert	$O(\lg n)$	$O(n)$	$O(n)$	Given location: $O(1)$ Arbitrary: $O(n)$
delete	$O(\lg n)$	$O(n)$	$O(n)$	↓ the same.
traverse	$O(n)$			

BST Reflection

We know the **height** of a tree.

We know if a tree is **full, complete**, and/or **perfect**.

We know that every binary tree has $n+1$ NULL pointers.

We know many **traversals** of trees.

We know that a **BST's height is bound by n** such that:

$$\underline{\lg(n)} \leq h \leq \underline{n}$$

We know all key **BST operations** run in $O(h)$ time.

We know a BST can be used to **implement a Dictionary**.

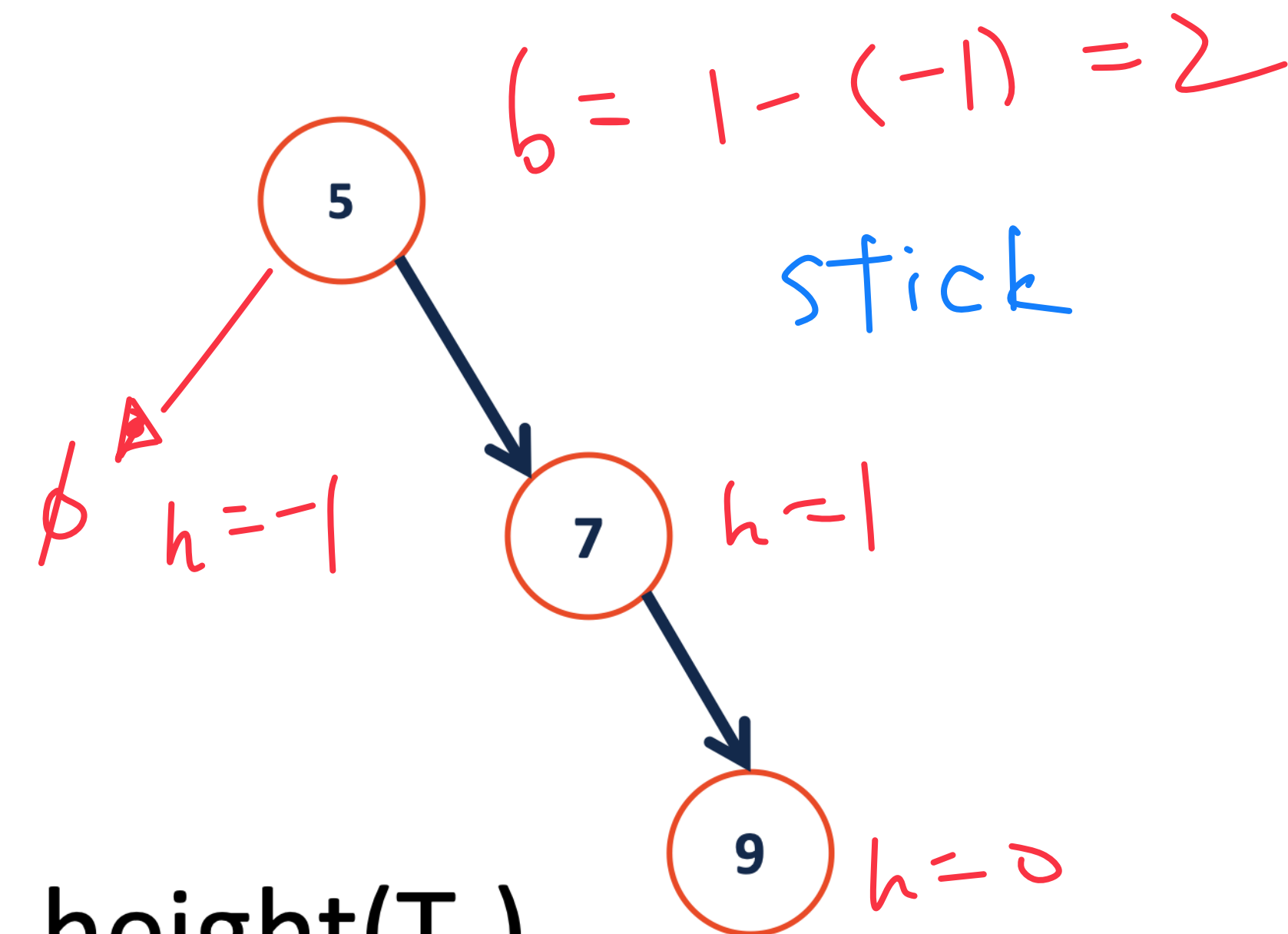
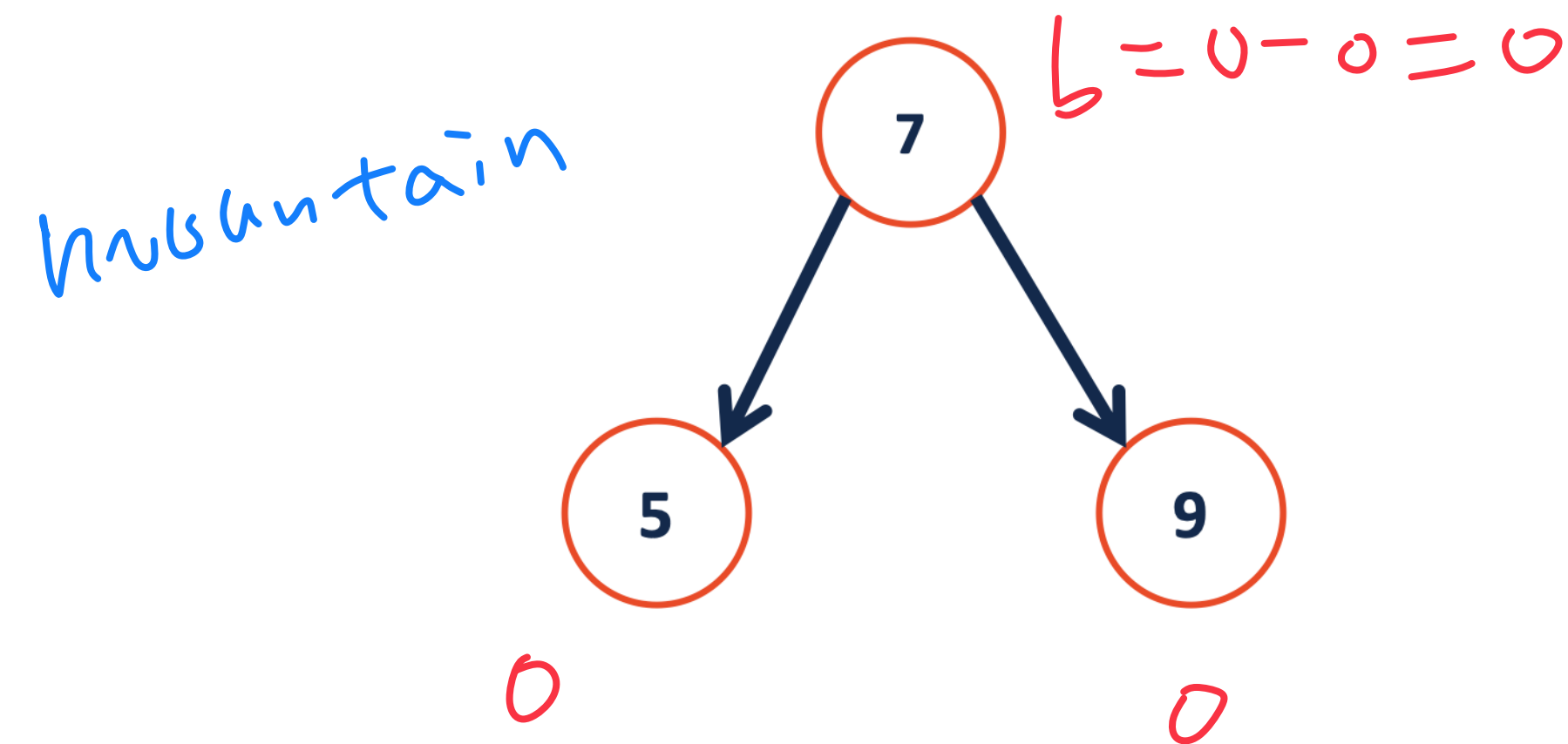
We know that a **random BST** has an average height of $\lg(n)$.

We know that an **inorder traversal** of a BST is a Ordered List.

We know **how to implement a BST** in C++.

Height-Balanced Tree

What tree makes you happier?

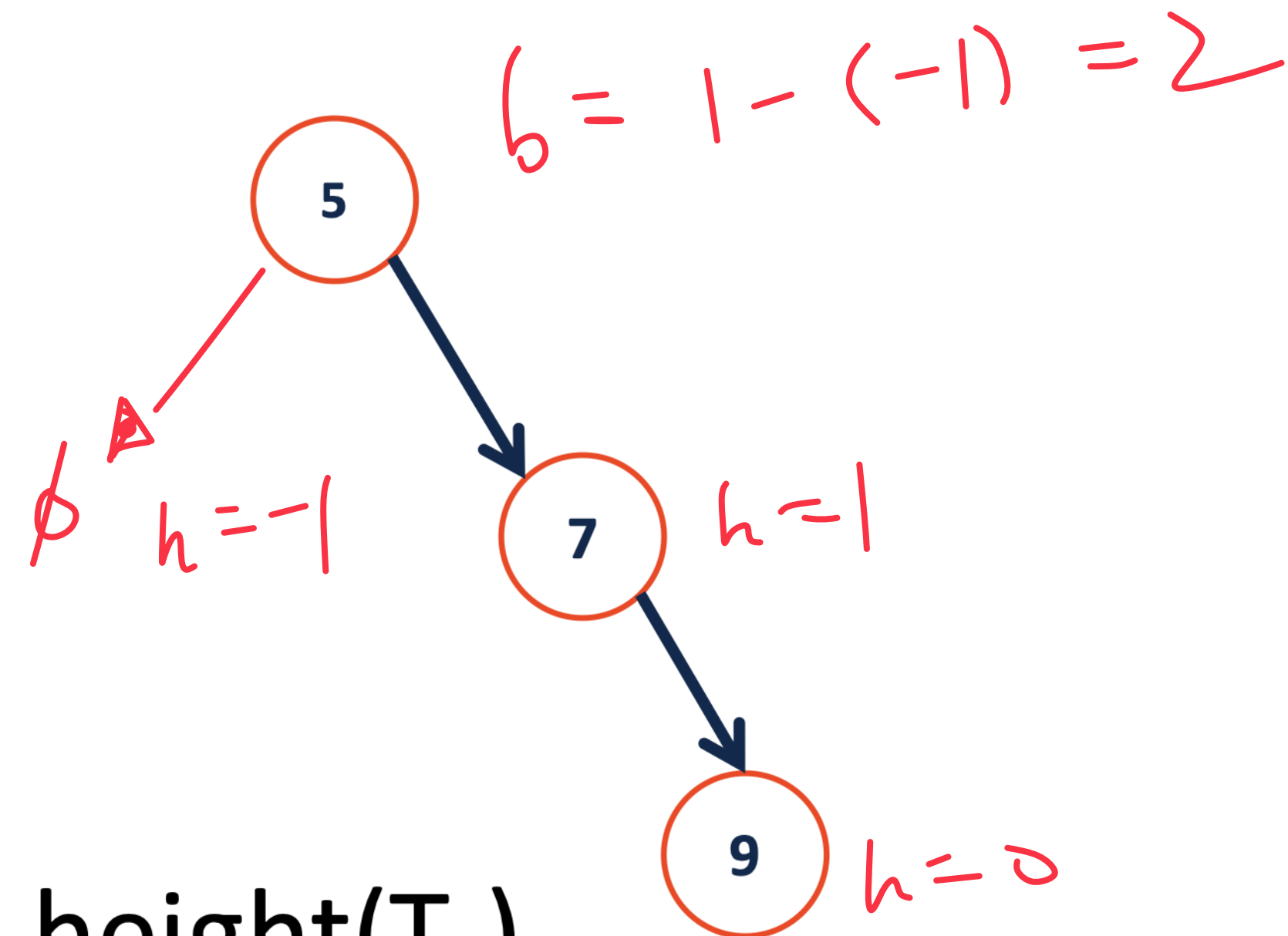
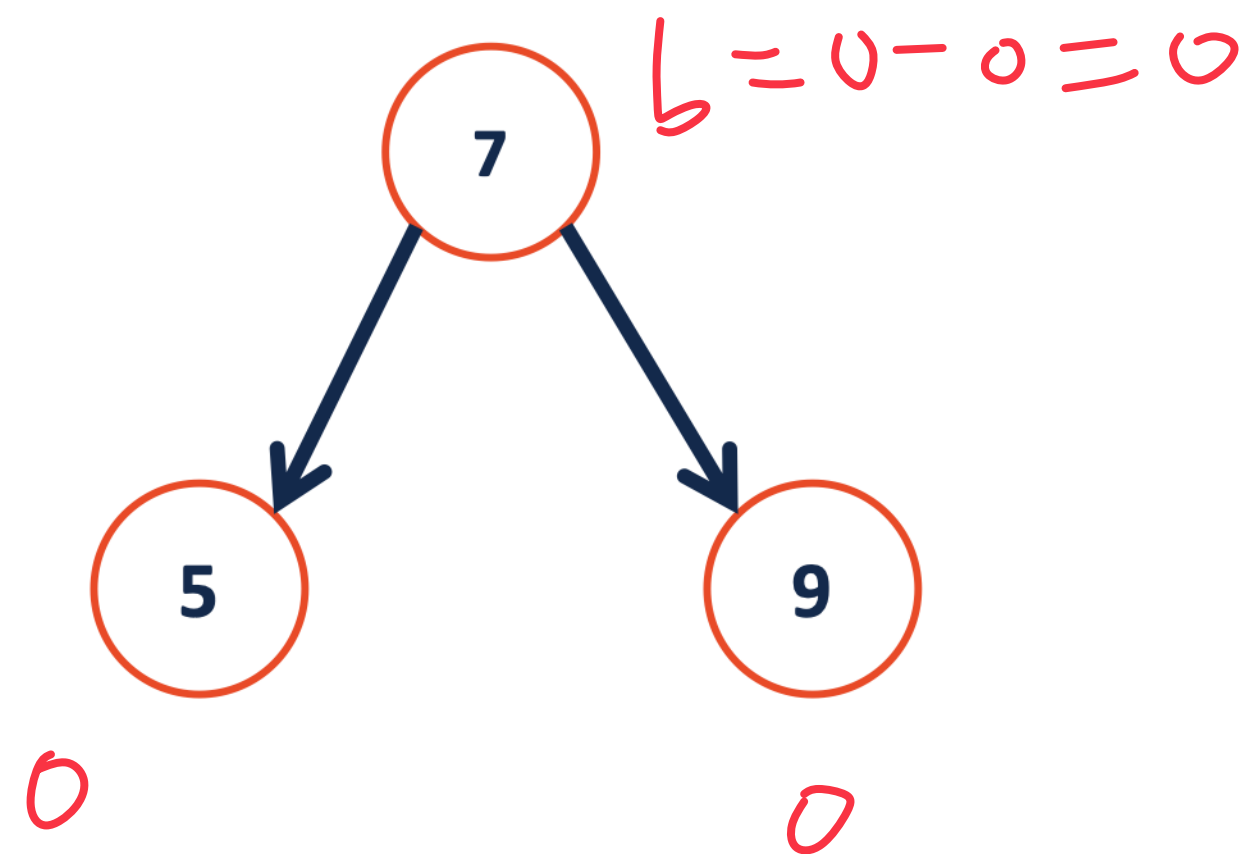


Height balance: $b = \text{height}(T_R) - \text{height}(T_L)$

A tree is height balanced if:

Height-Balanced Tree

What tree makes you happier?



Height balance: $b = \text{height}(T_R) - \text{height}(T_L)$

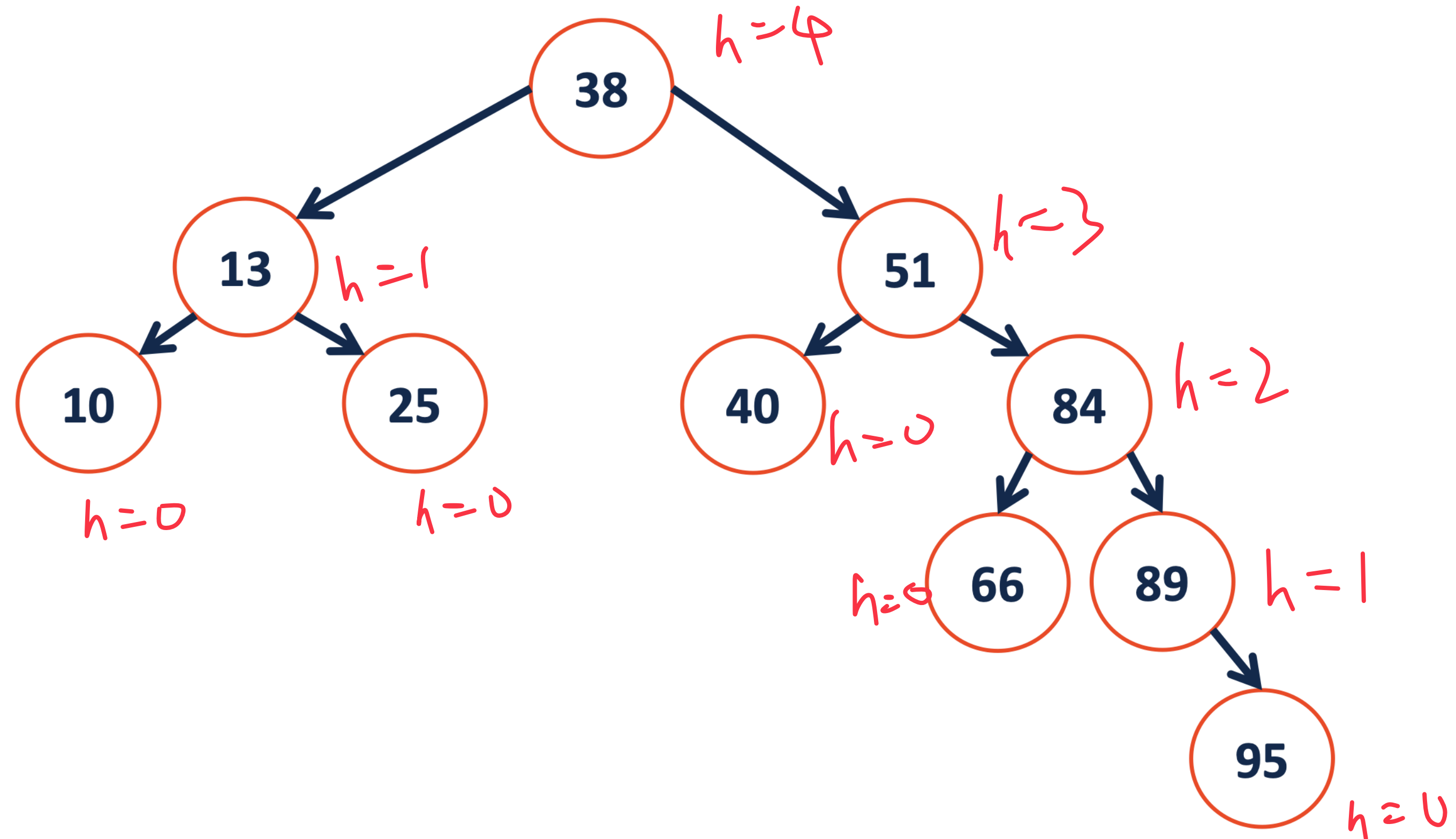
A tree is height balanced if:

$$T = \{\} \quad \text{or} \quad T = \{r, T_R, T_L\}$$

and $|b| \leq 1$

height of tree: 4

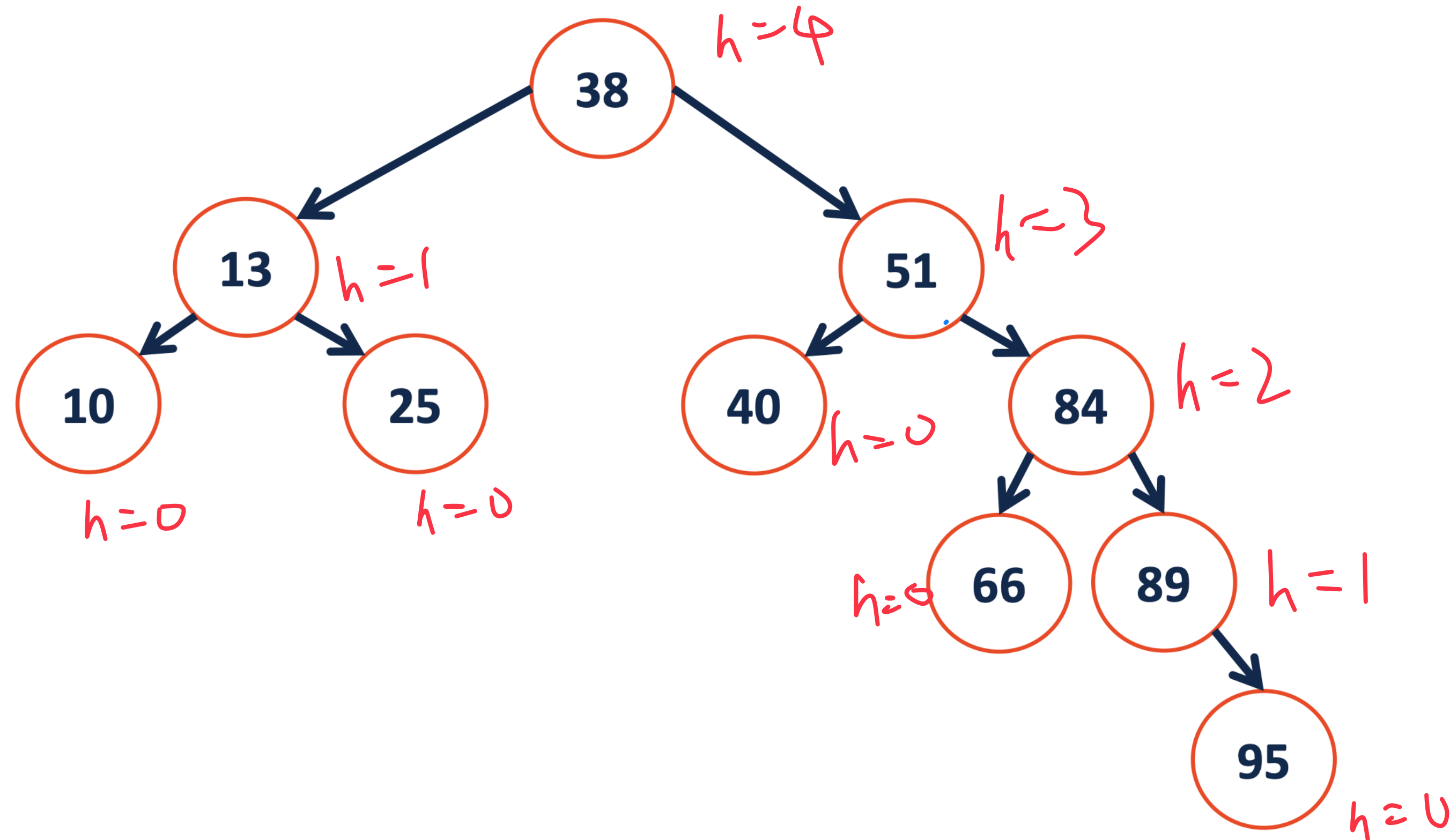
balance factor b: $T_R - T_L = 3 - 1 = \underline{2}$



height of tree: 4

balance factor b : $T_R - T_L = 3 - 1 = \underline{2}$

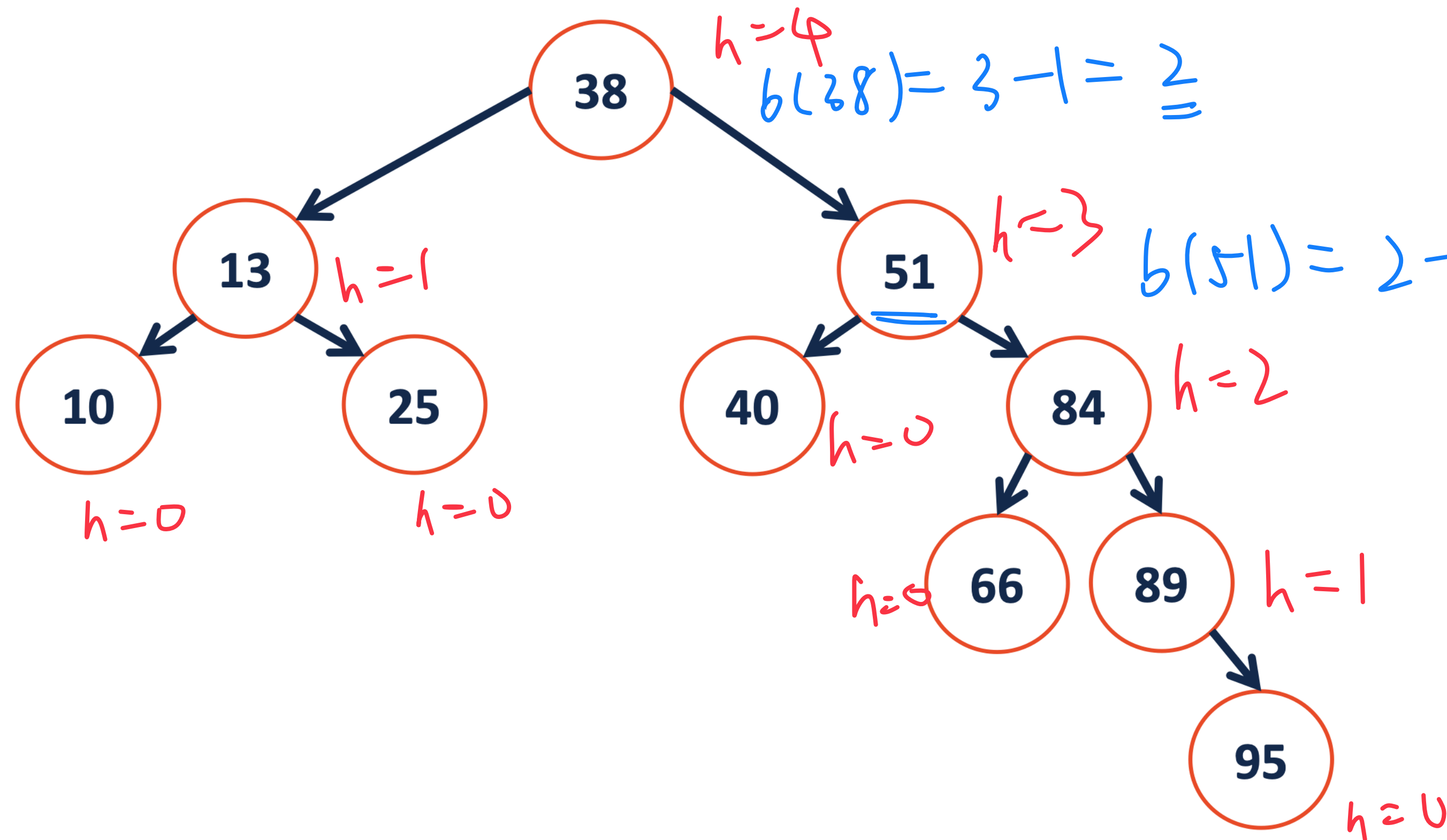
- lowest/deepest node that is
"out of balance" $b=2$??



height of tree: 4

balance factor b : $T_R - T_L = 3 - 1 = \underline{\underline{2}}$

- lowest/deepest node that is
"out of balance" $b=2$??



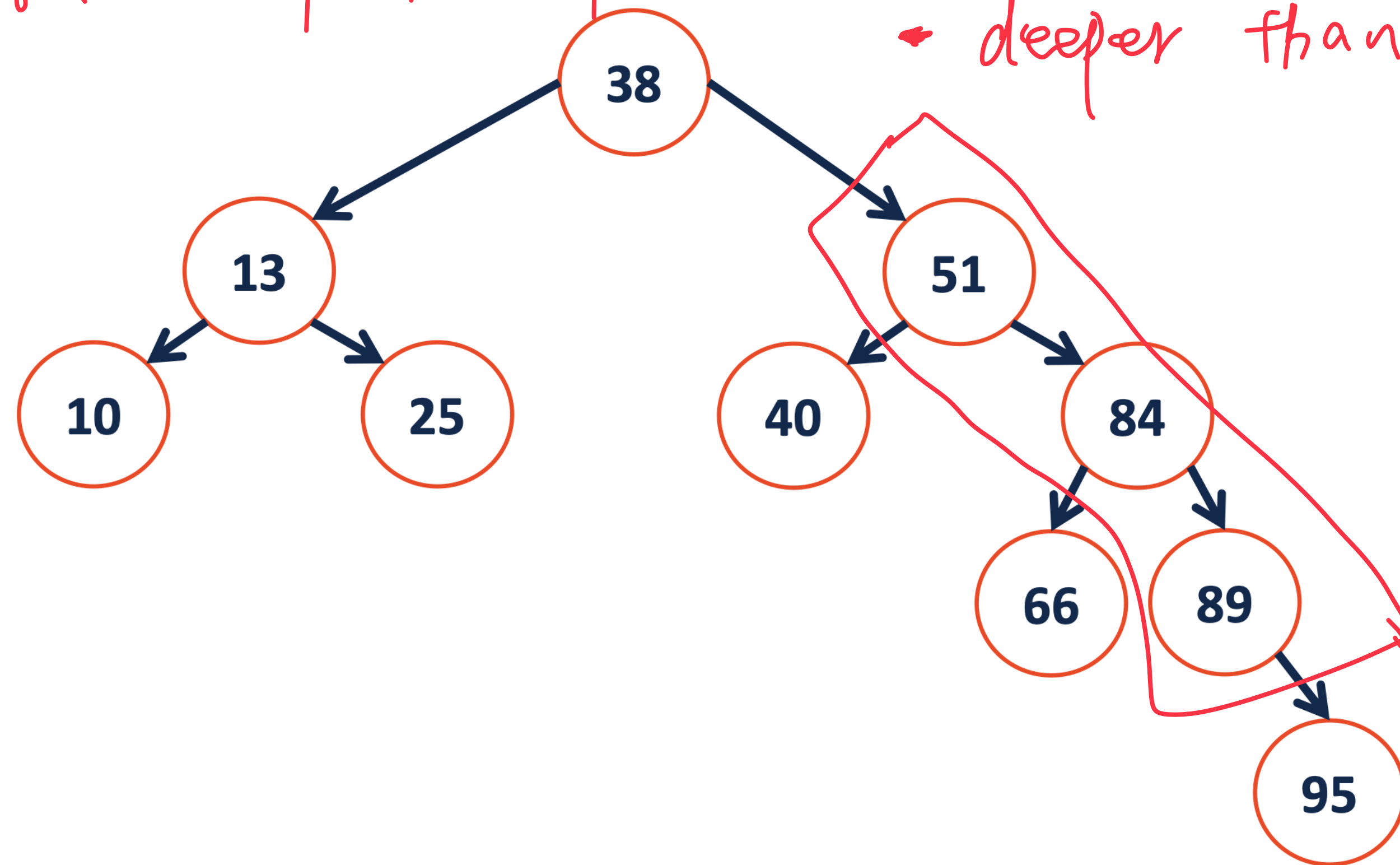
How to bring it to balance??

BST Rotation

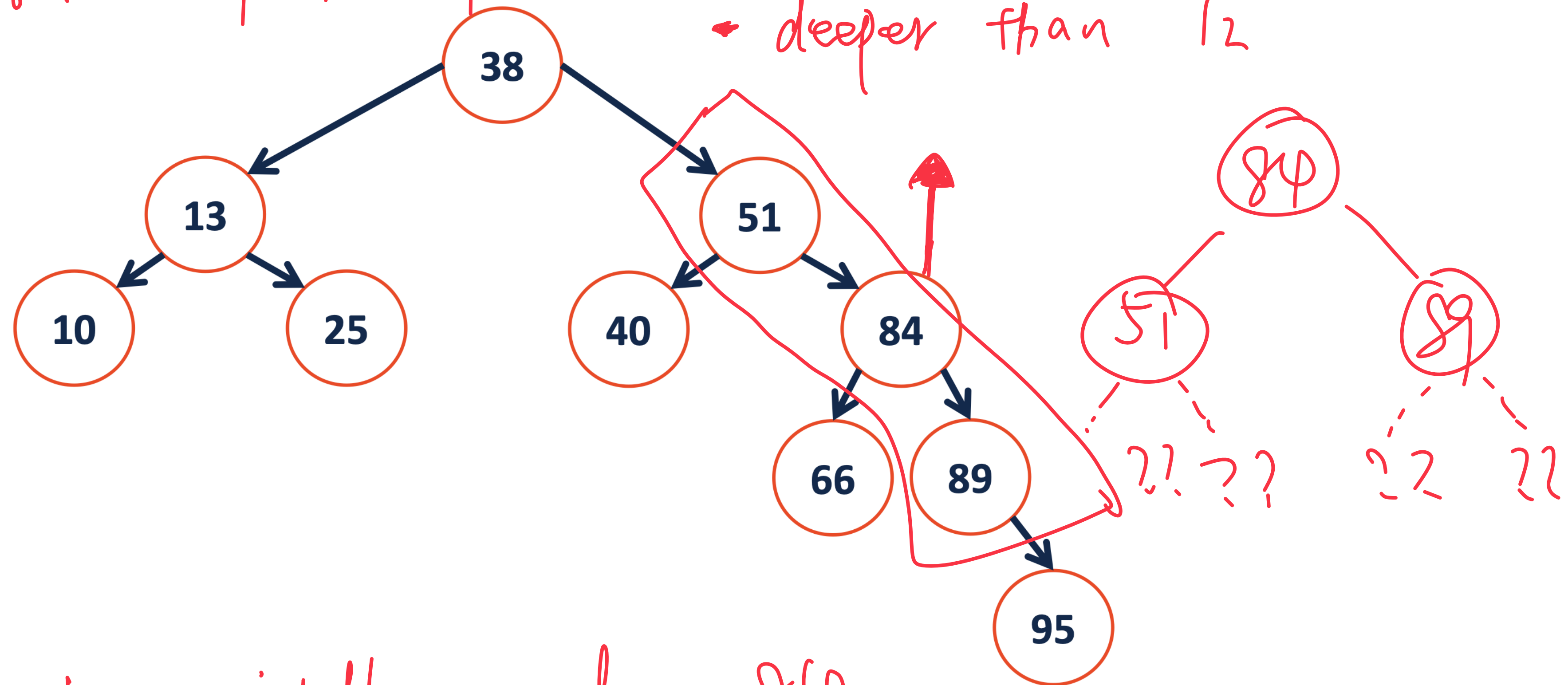
We will perform a rotation that maintains two properties:

1. maintain the order of BST
2. Bring the tree to balance
'stick' → 'mountain'

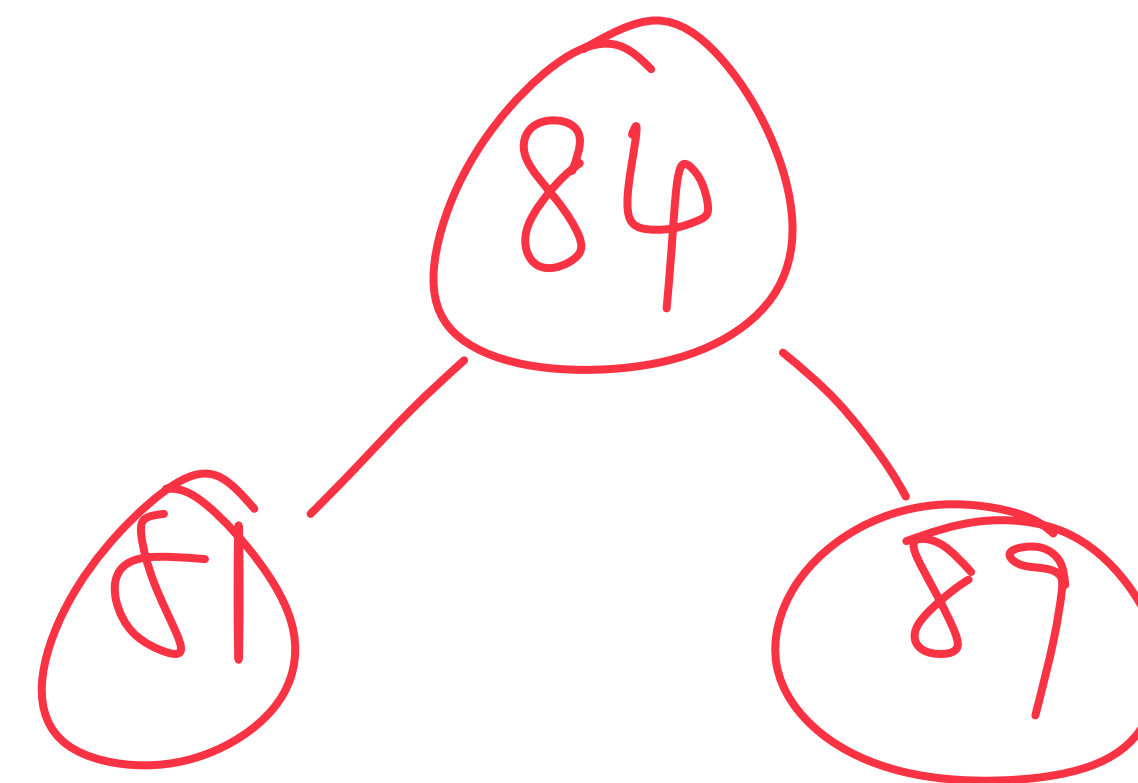
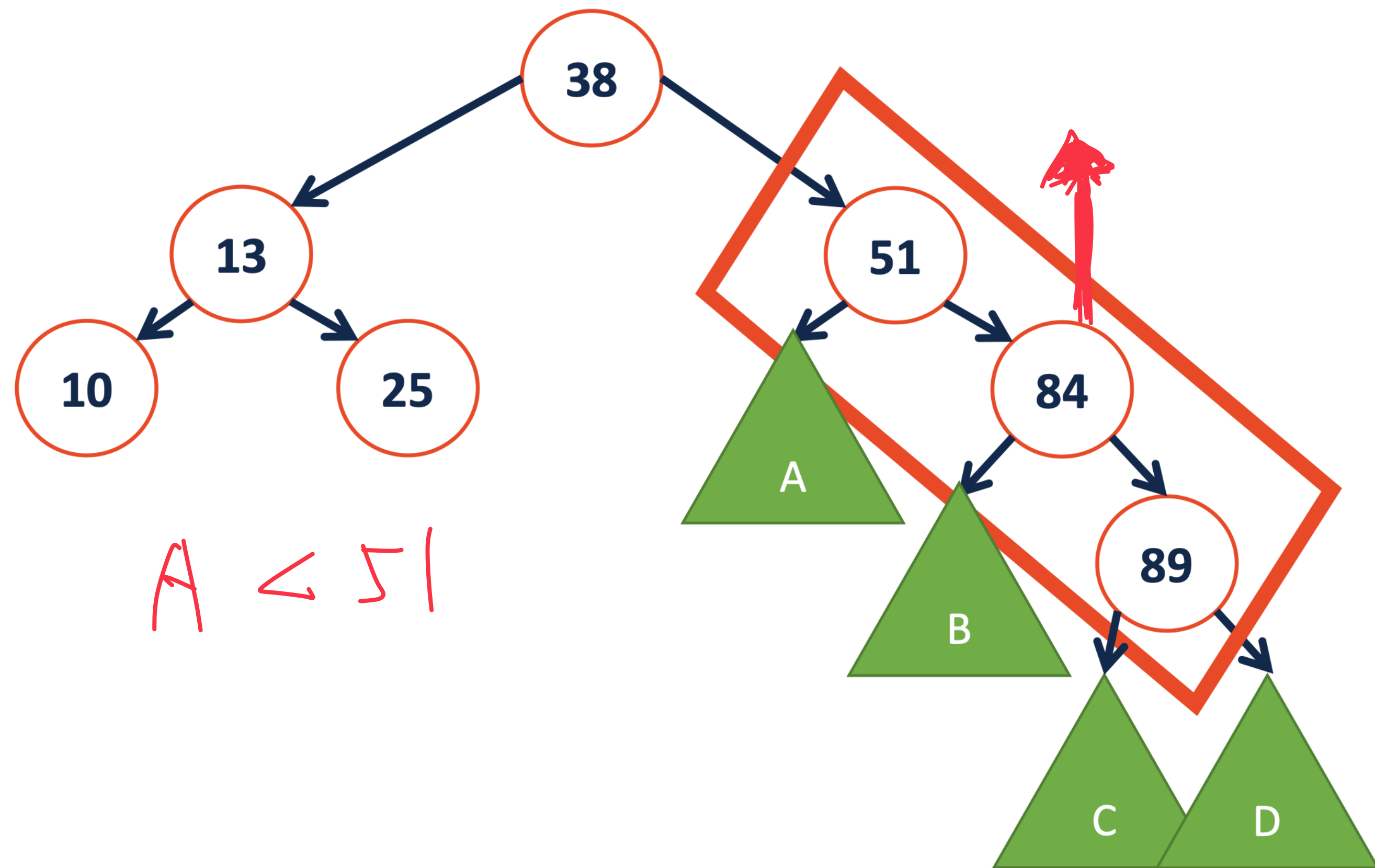
- lowest node imbalance: 51
- stick: $51 \rightarrow 84 \rightarrow 89$, right subtree out of balance
 - deeper than T_2

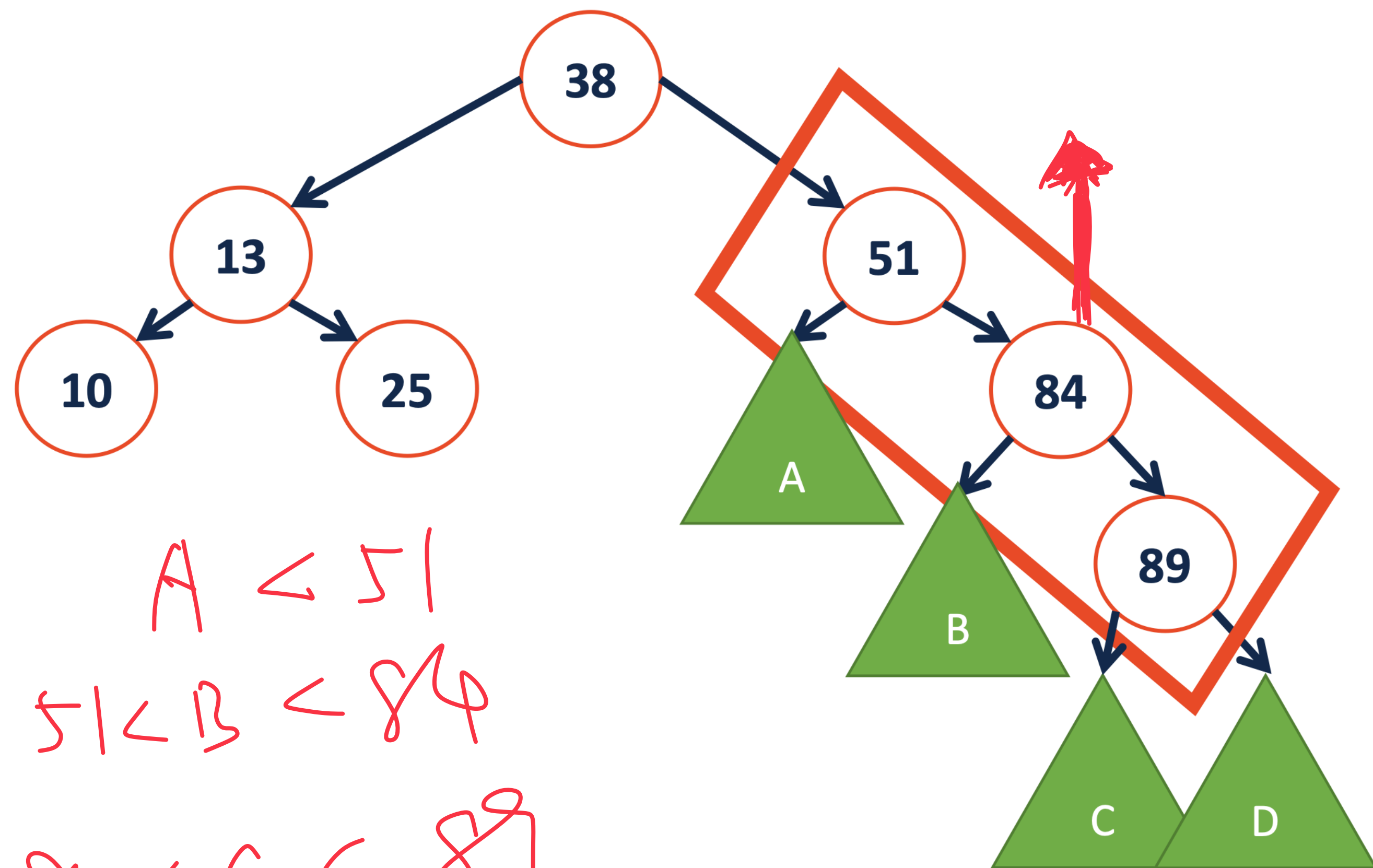


- lowest node imbalance: 51
- stick: $51 \rightarrow 84 \rightarrow 89$, right subtree out of balance
 - deeper than T_2

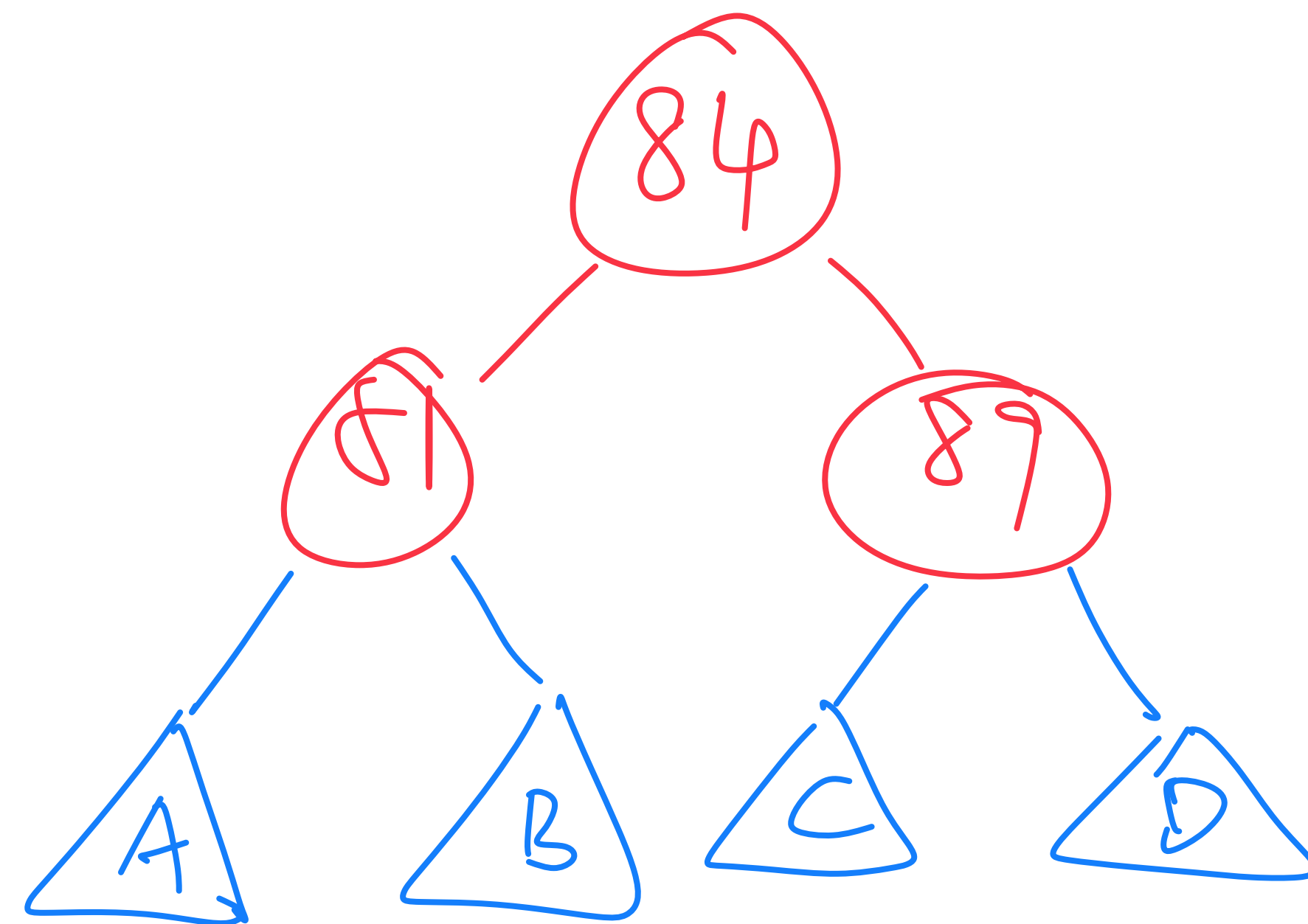


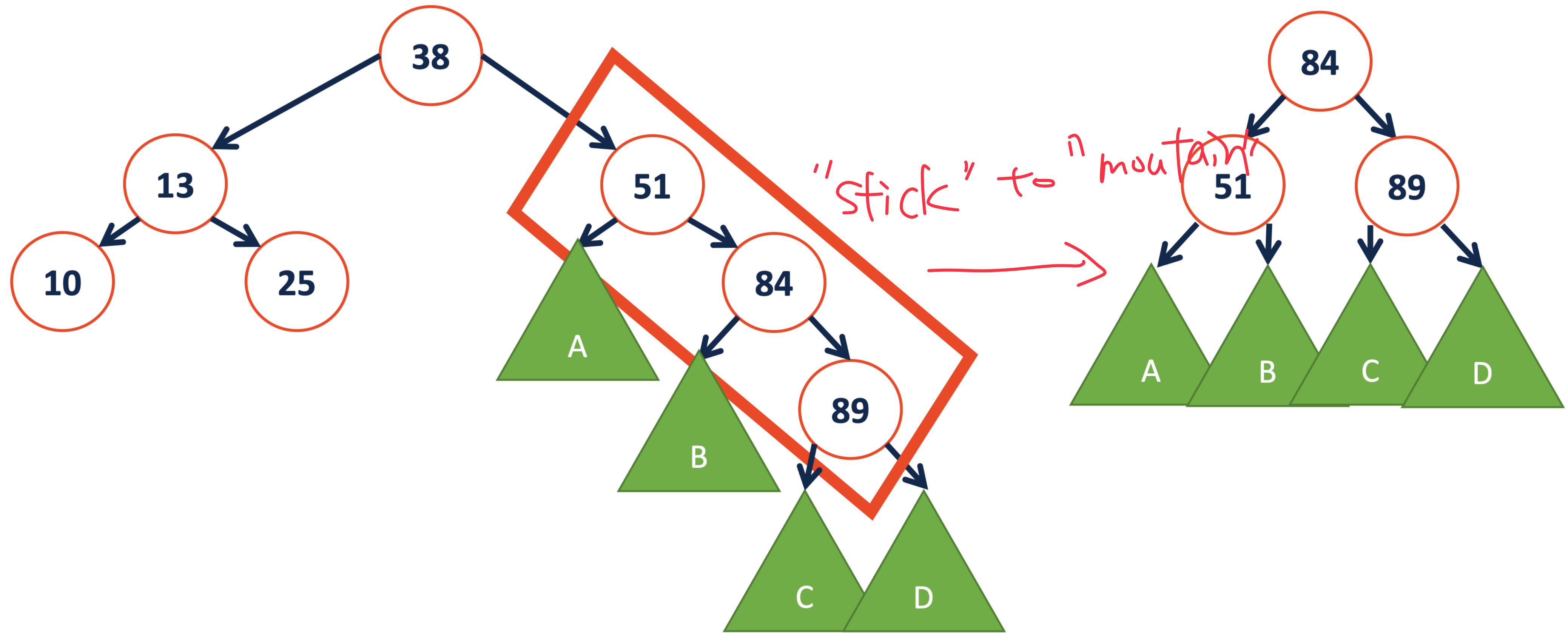
- raise up the middle node 84



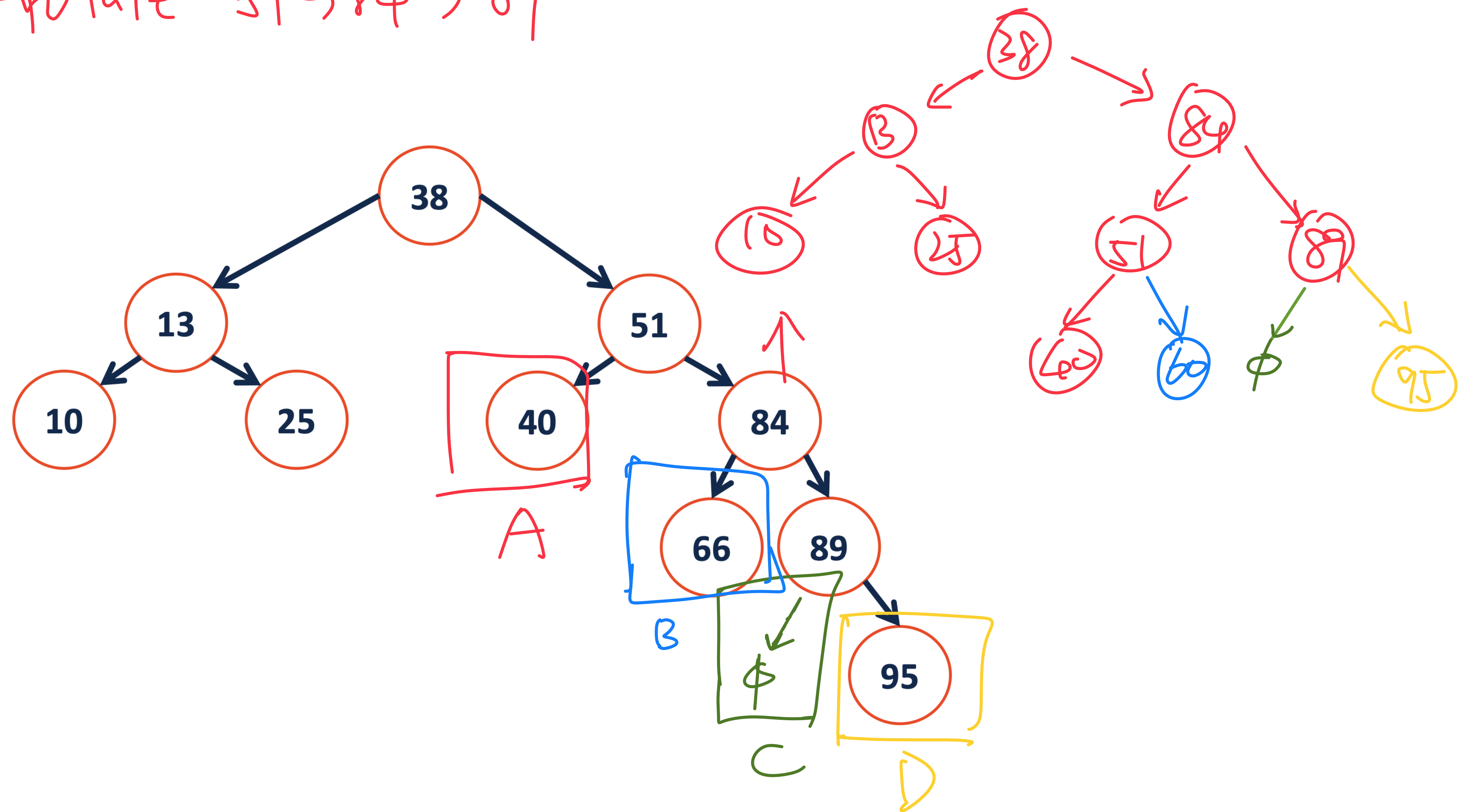


$A < 51$
 $51 < B < 84$
 $84 < C < 89$
 $89 < D$

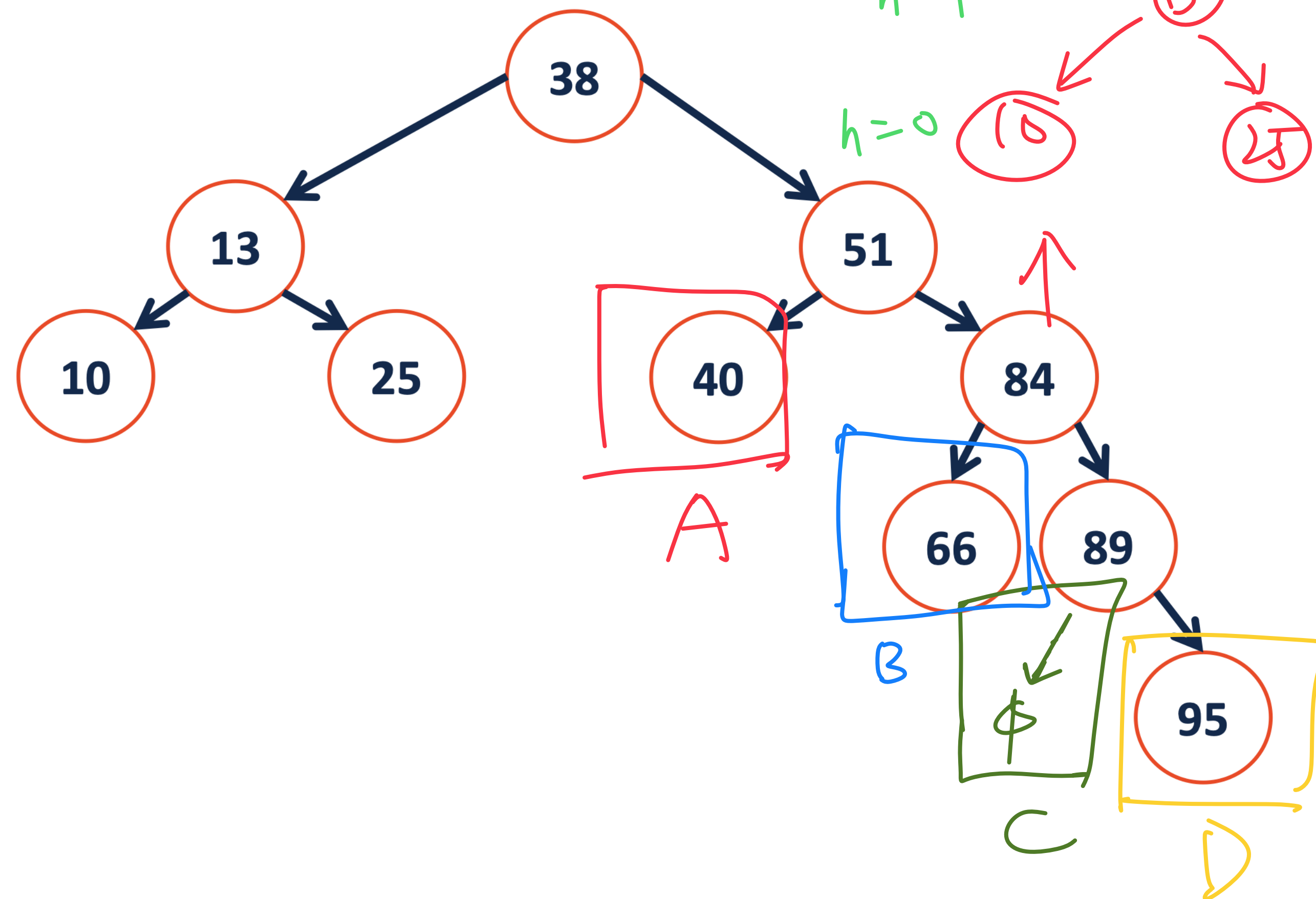




- rotate "51 → 84 → 89"



- rotate "51 → 84 → 89"



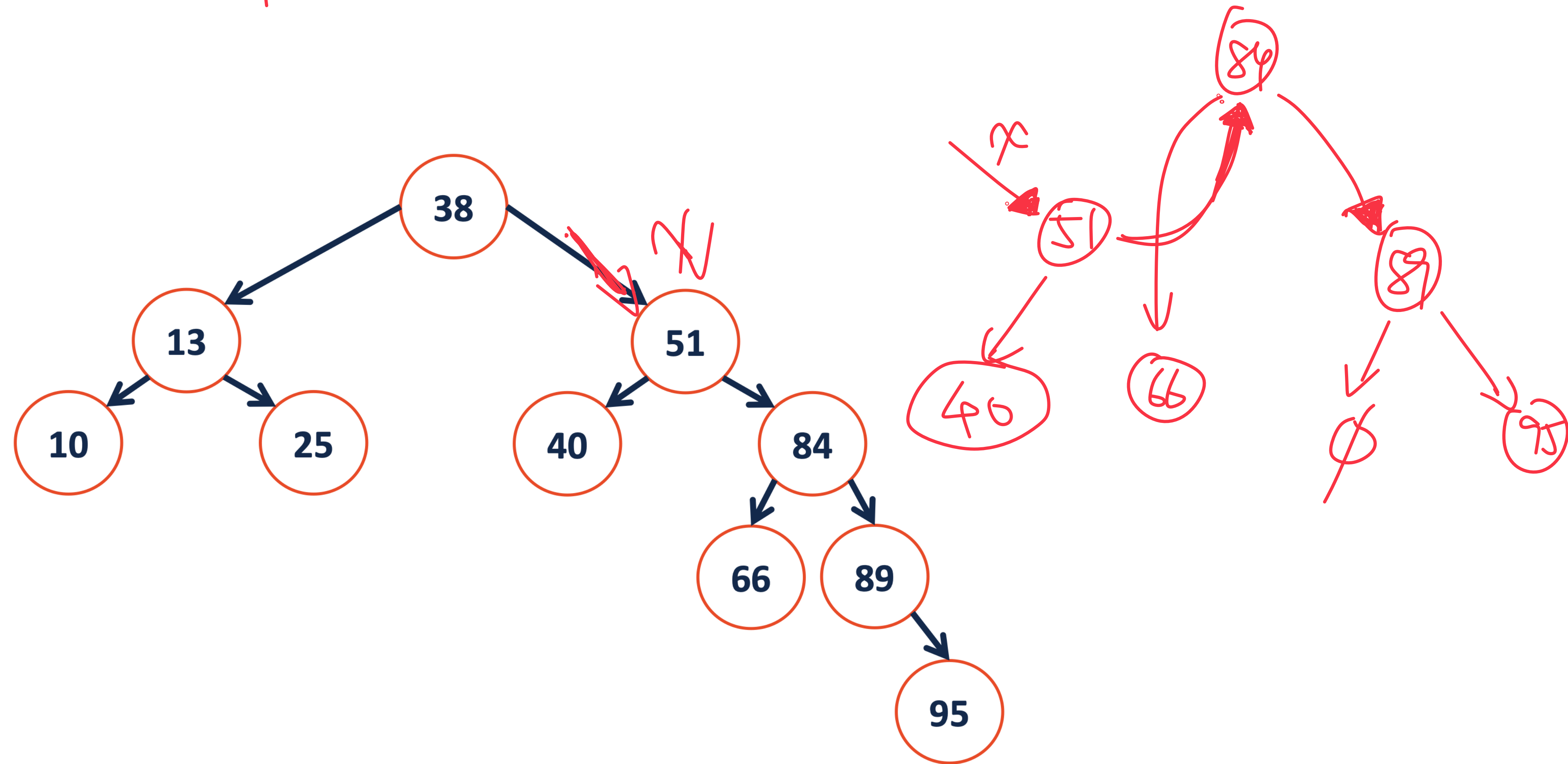
$$b(38) = 2 - 1 = 1$$

$$h=2$$

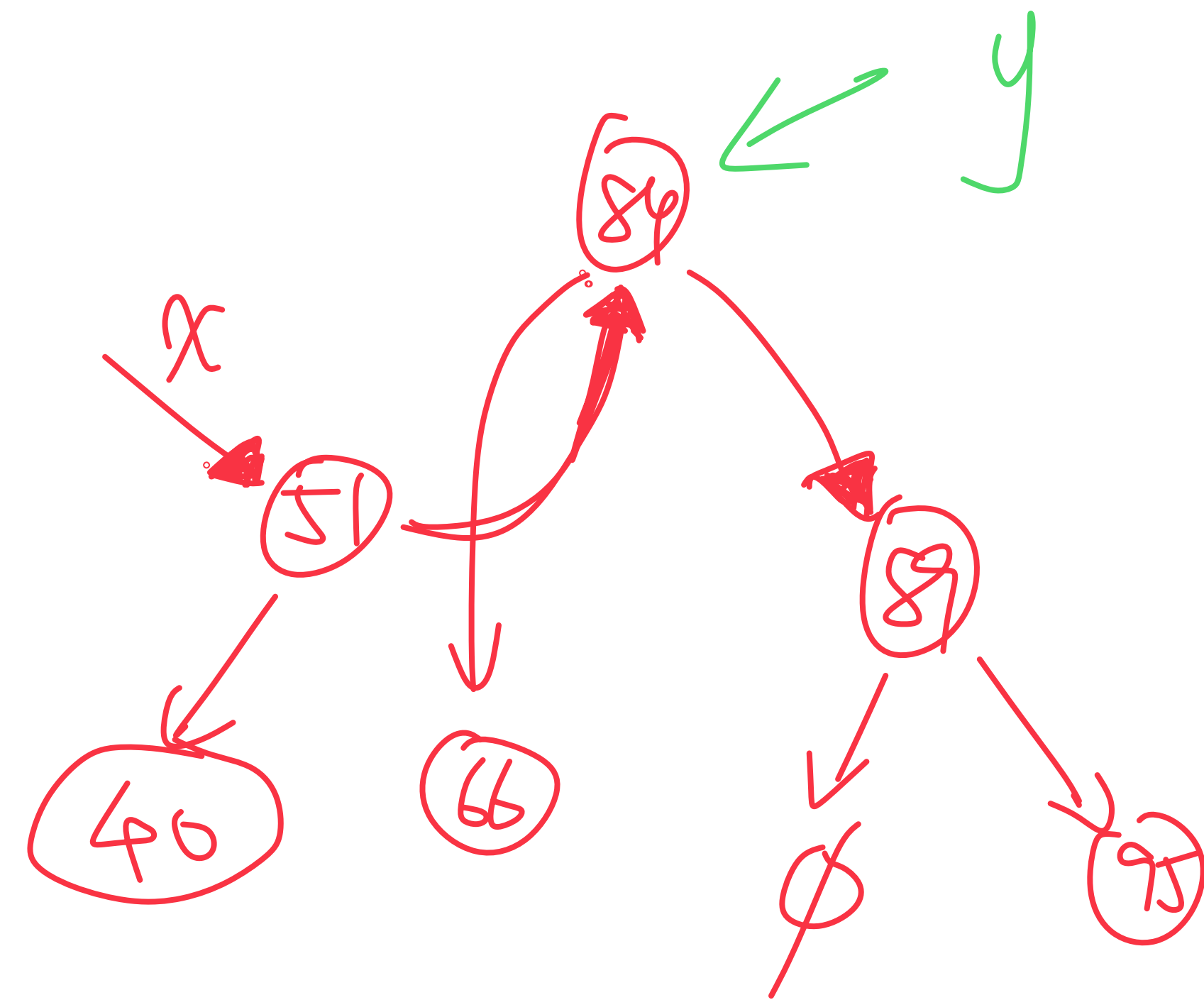
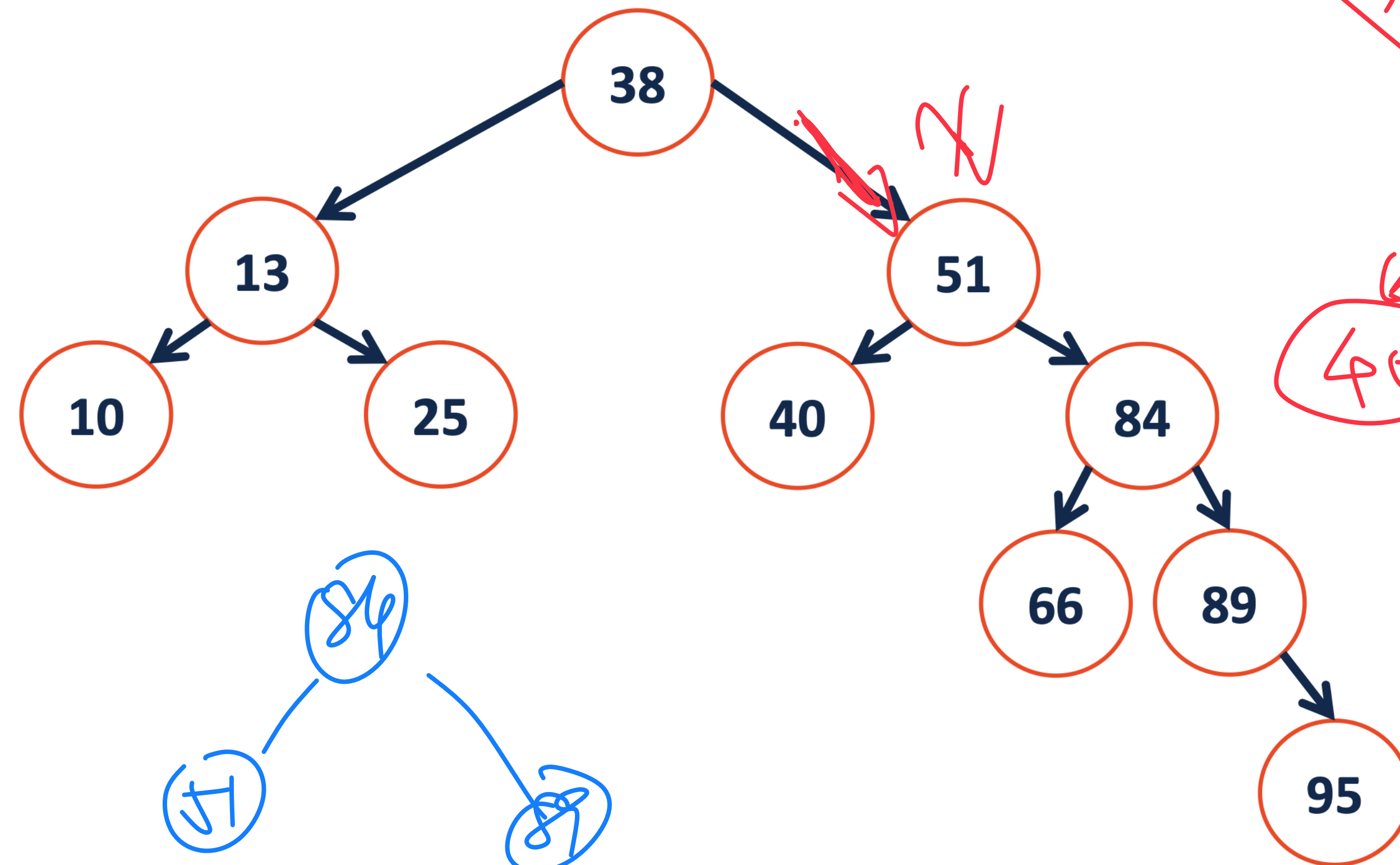
$$h=1$$

$$h=0$$

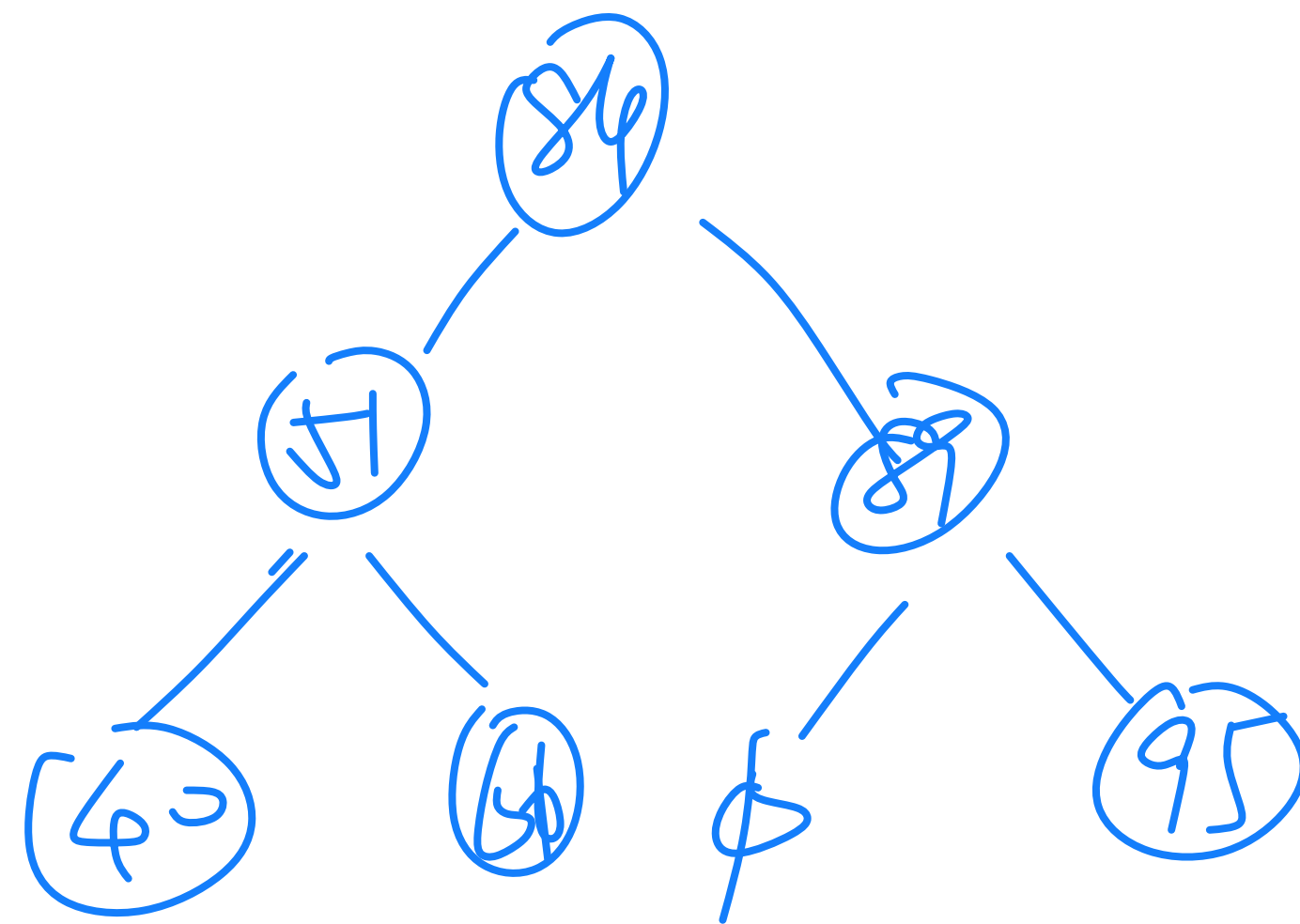
overall procedure



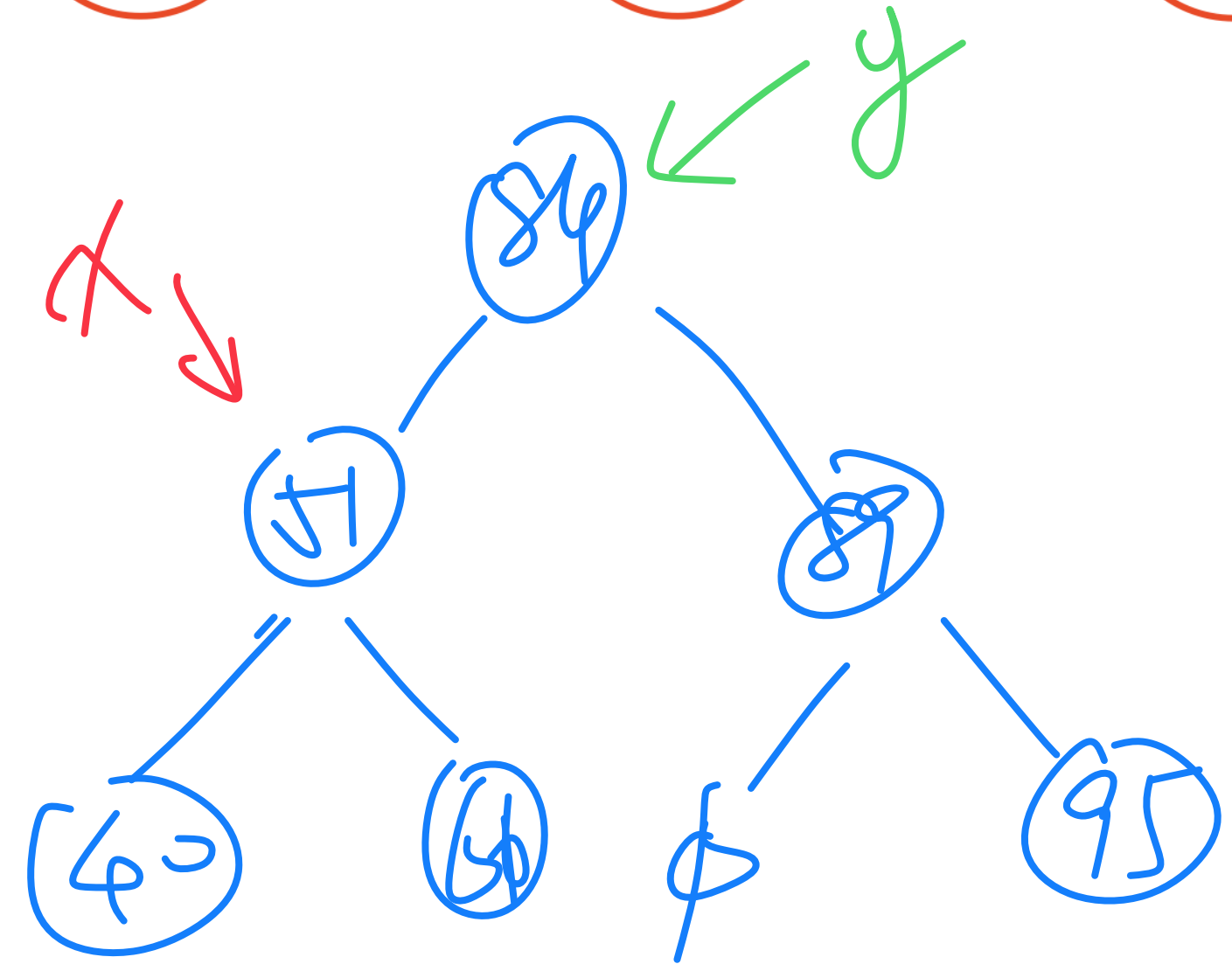
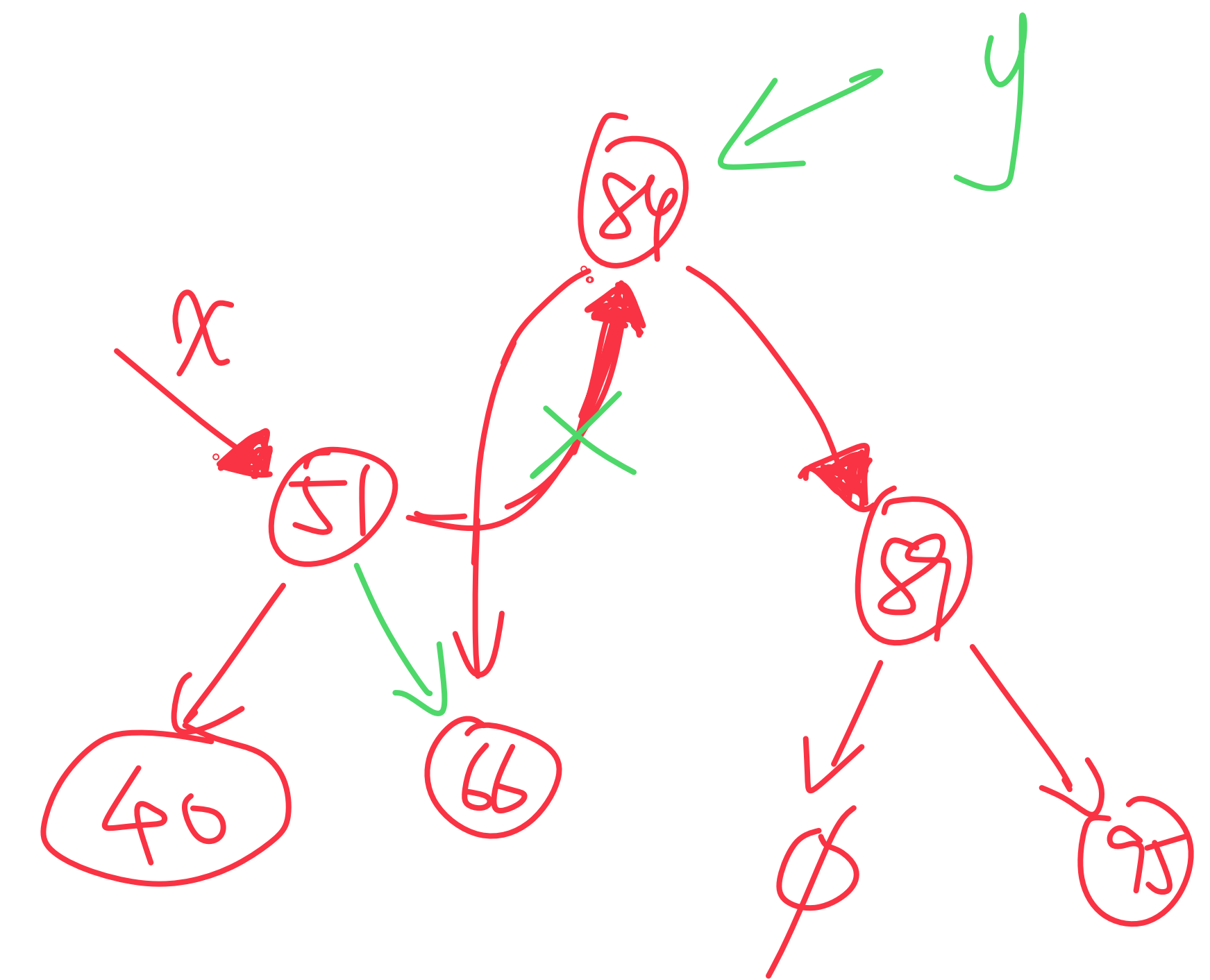
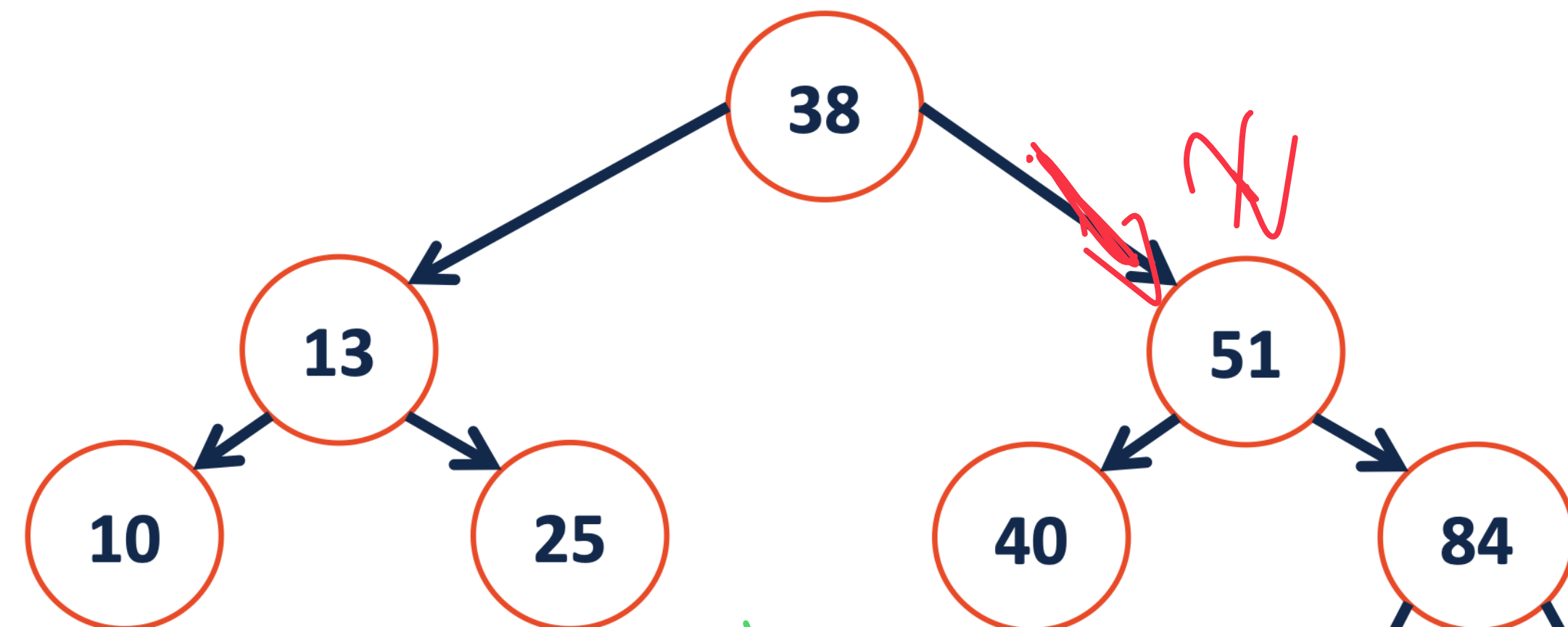
• overall procedure



$y = x \rightarrow \text{right}$

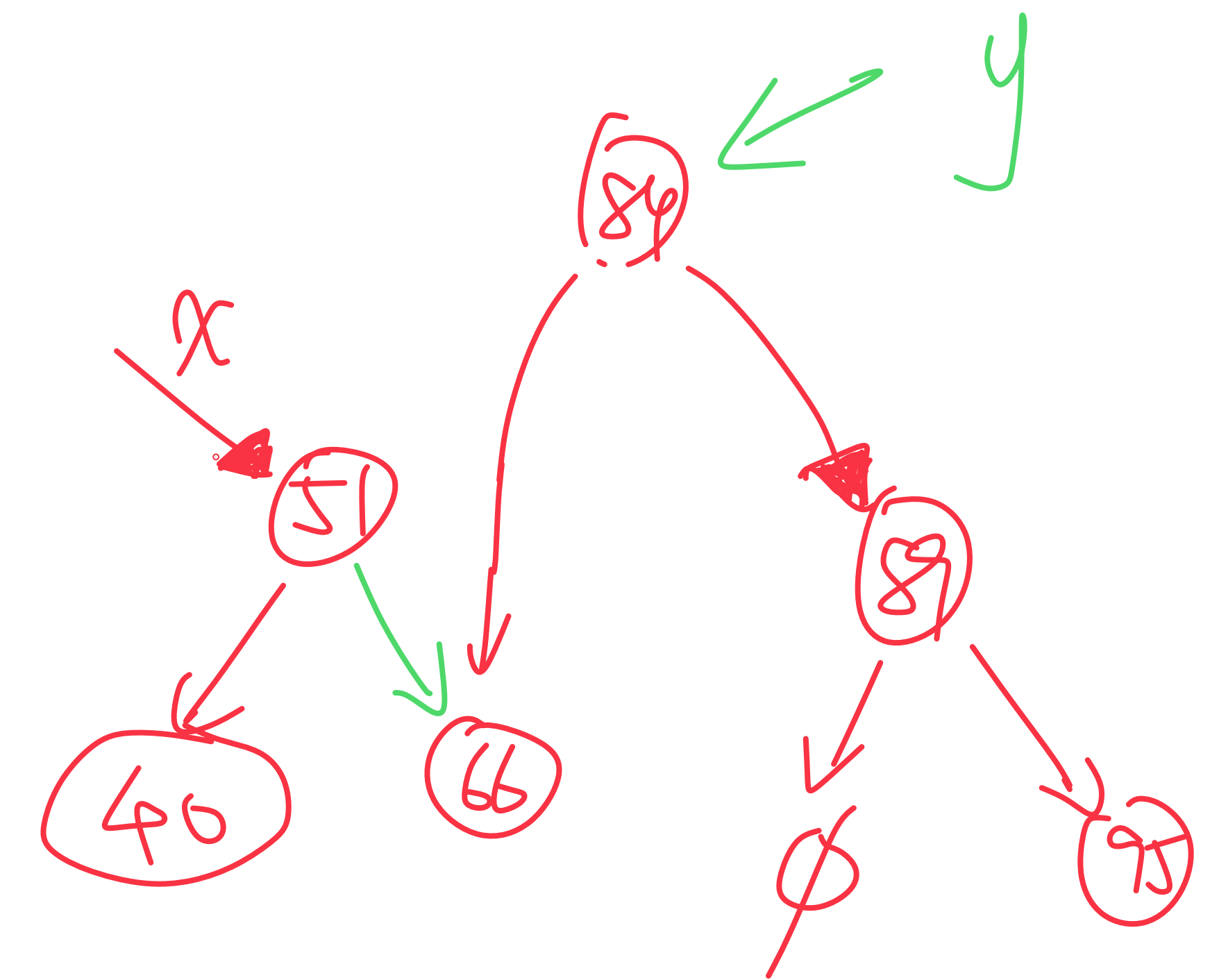
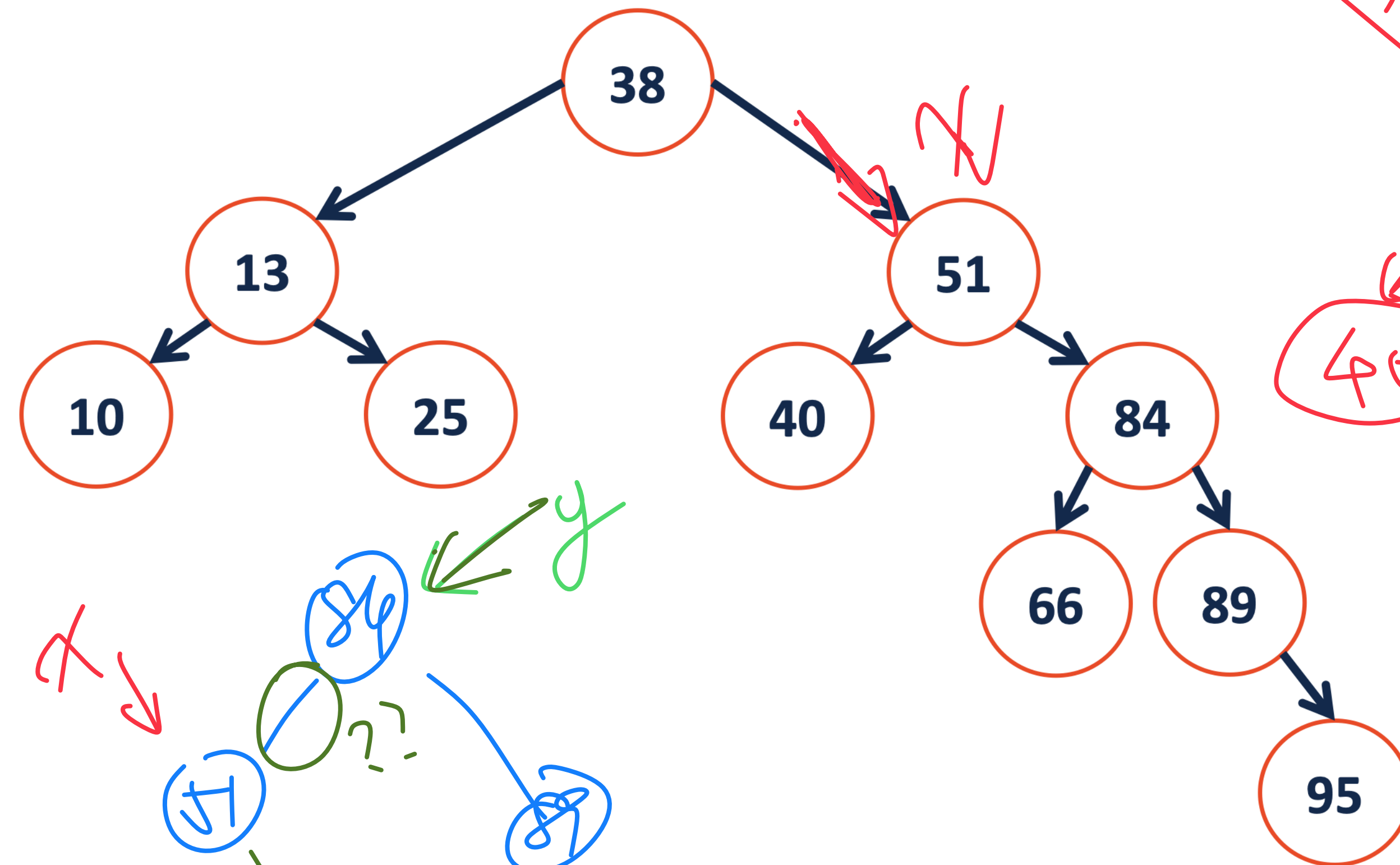


overall procedure



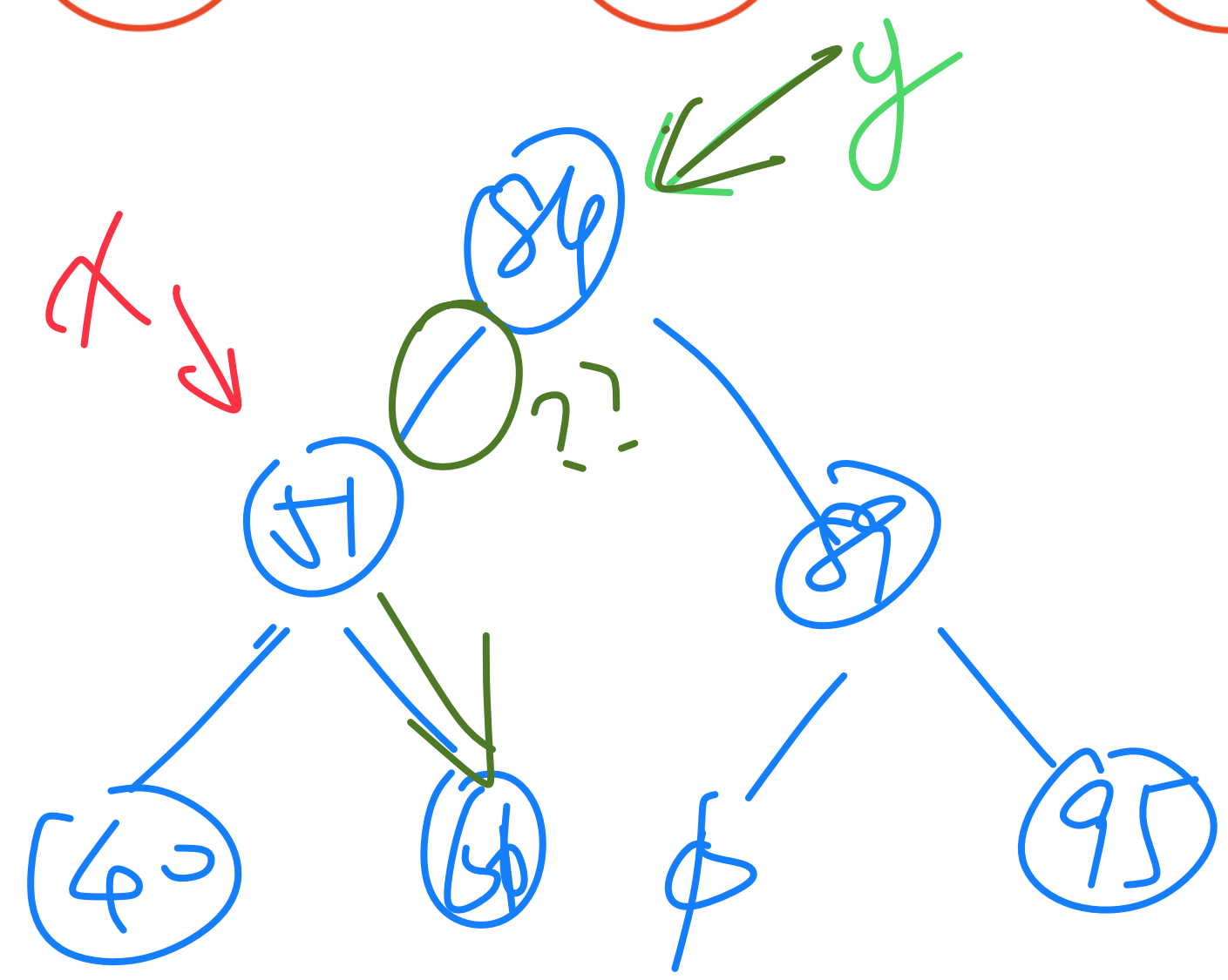
$y = x \rightarrow \text{right} ;$
 $x \rightarrow \text{right} = y \rightarrow \text{left} ;$

• overall procedure

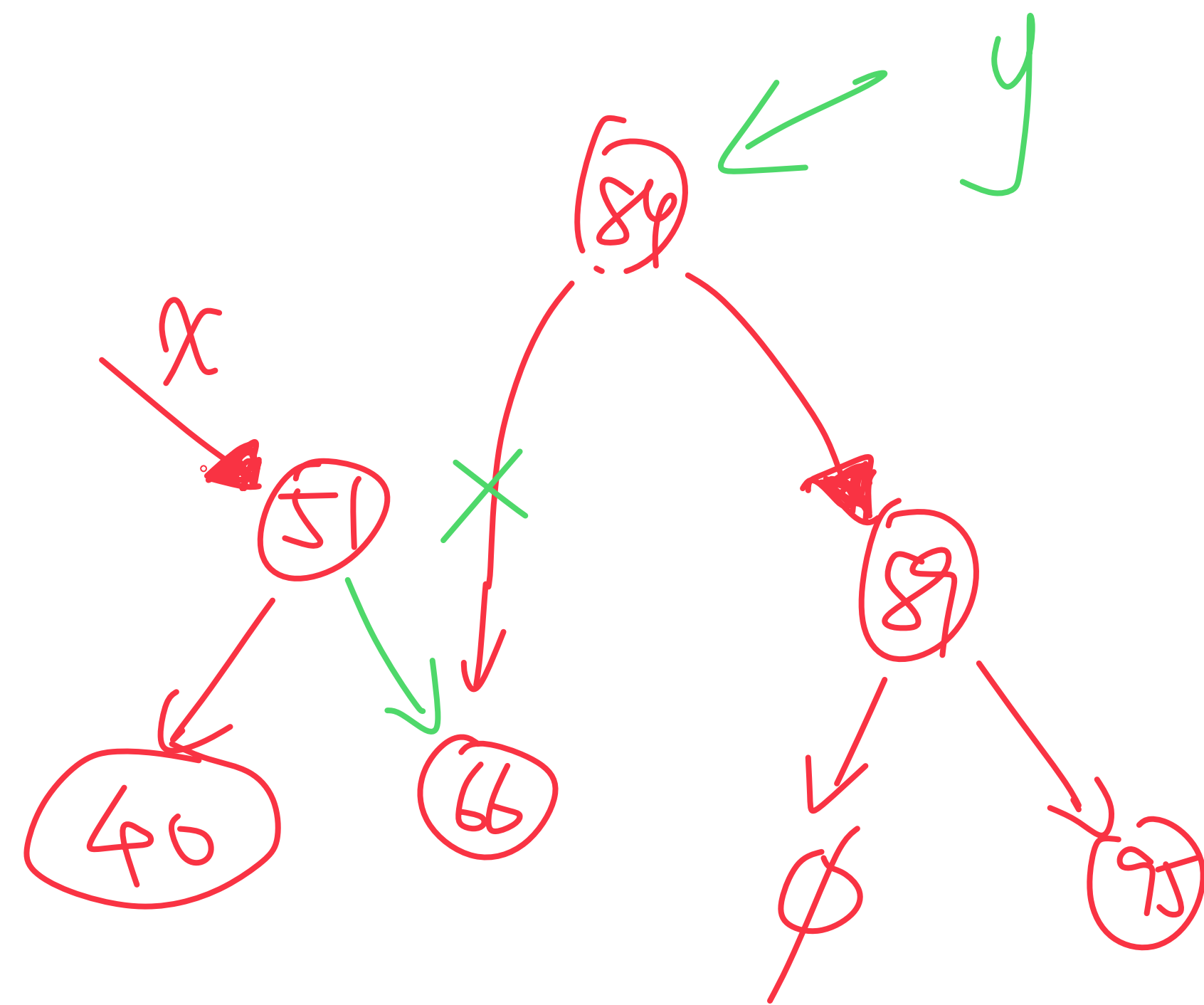
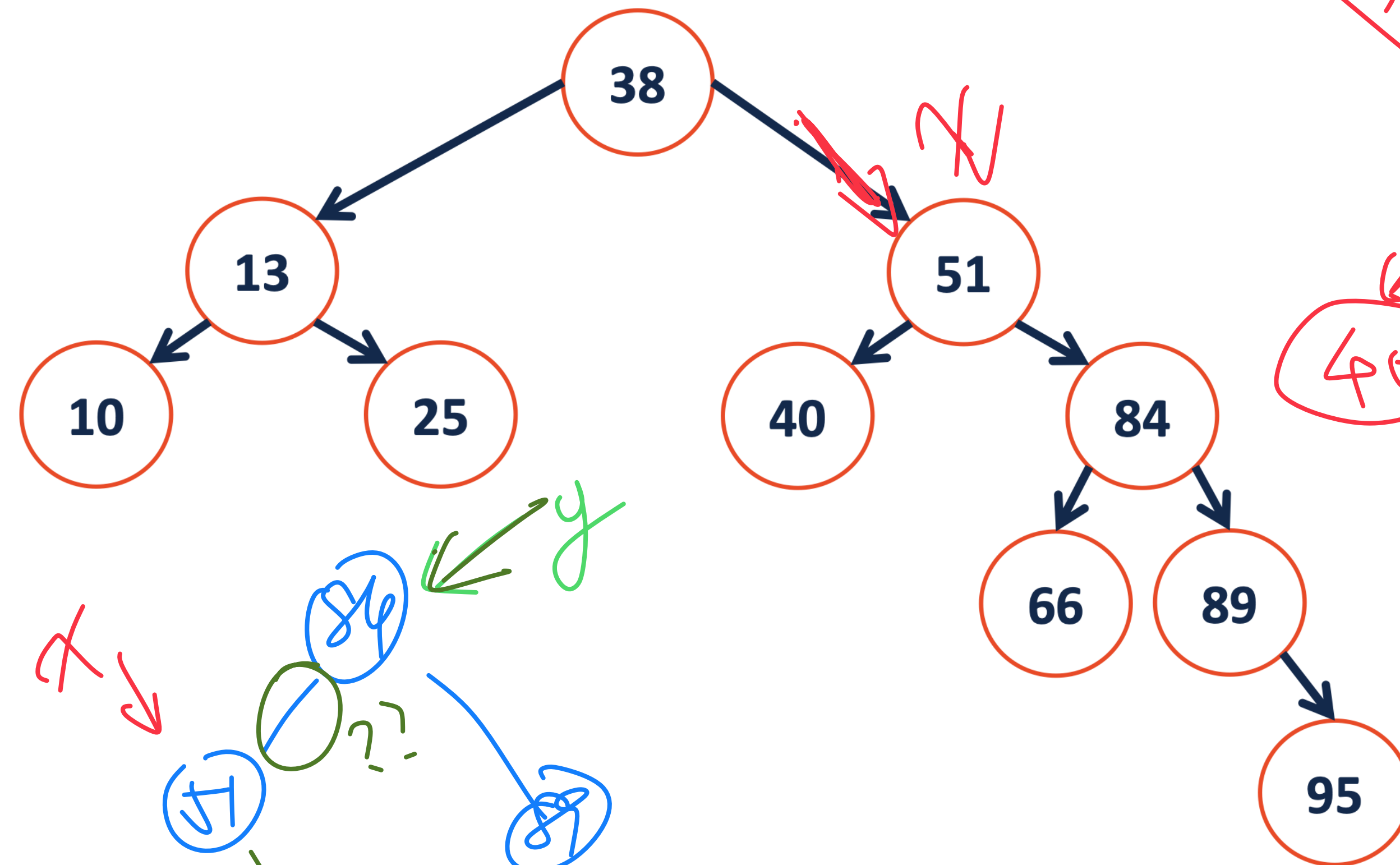


$y = x \rightarrow \text{right};$

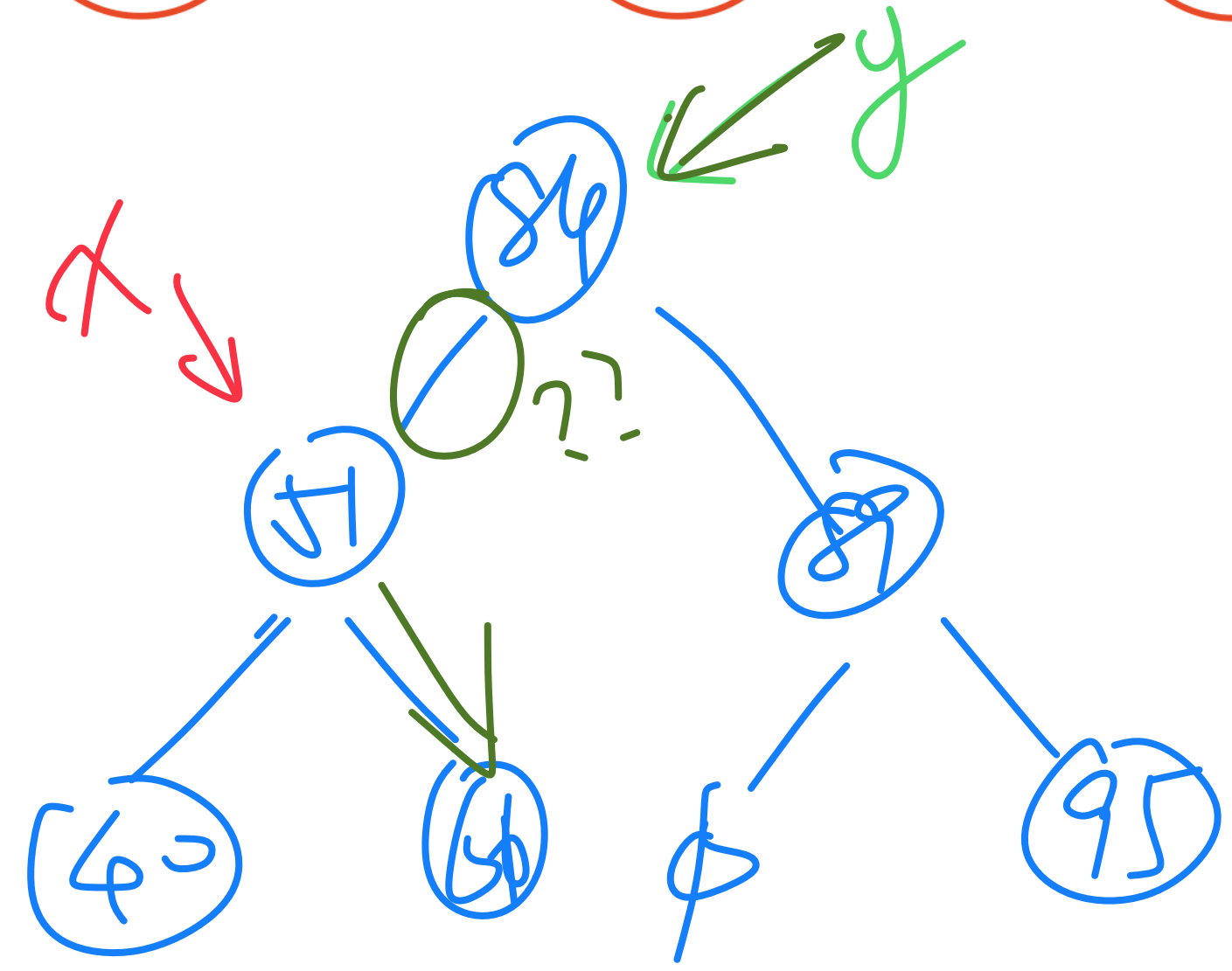
$x \rightarrow \text{right} = y \rightarrow \text{left};$



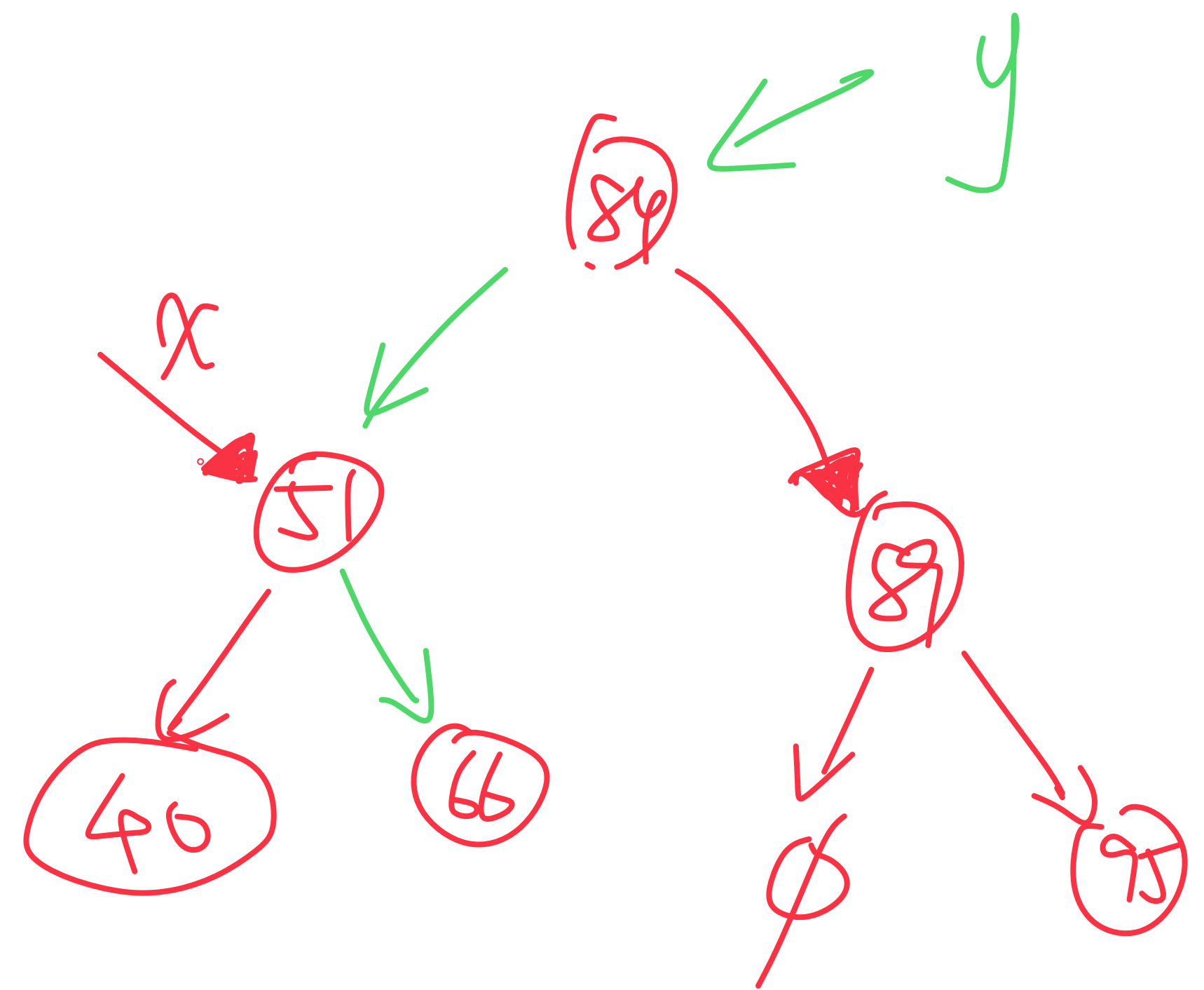
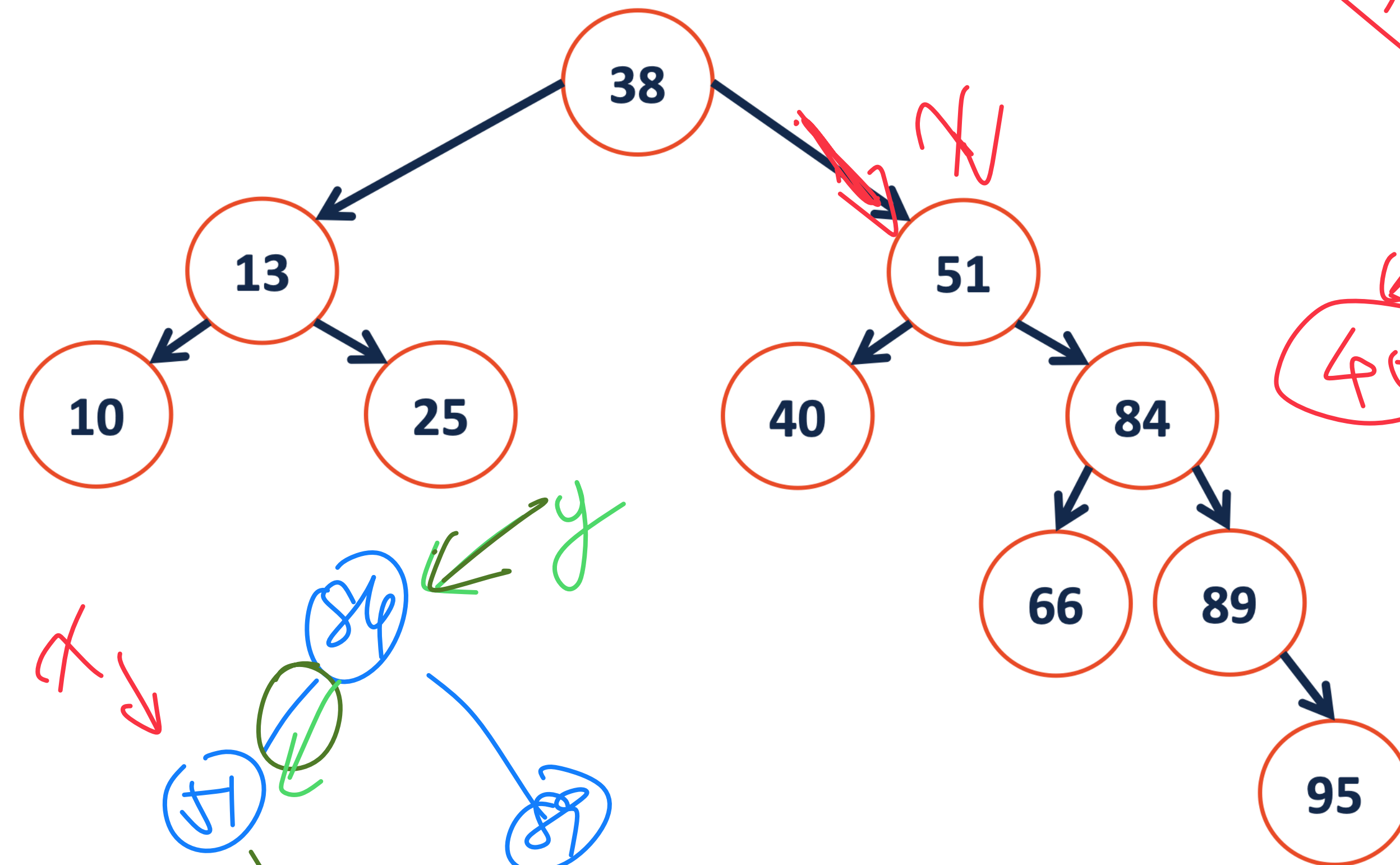
overall procedure



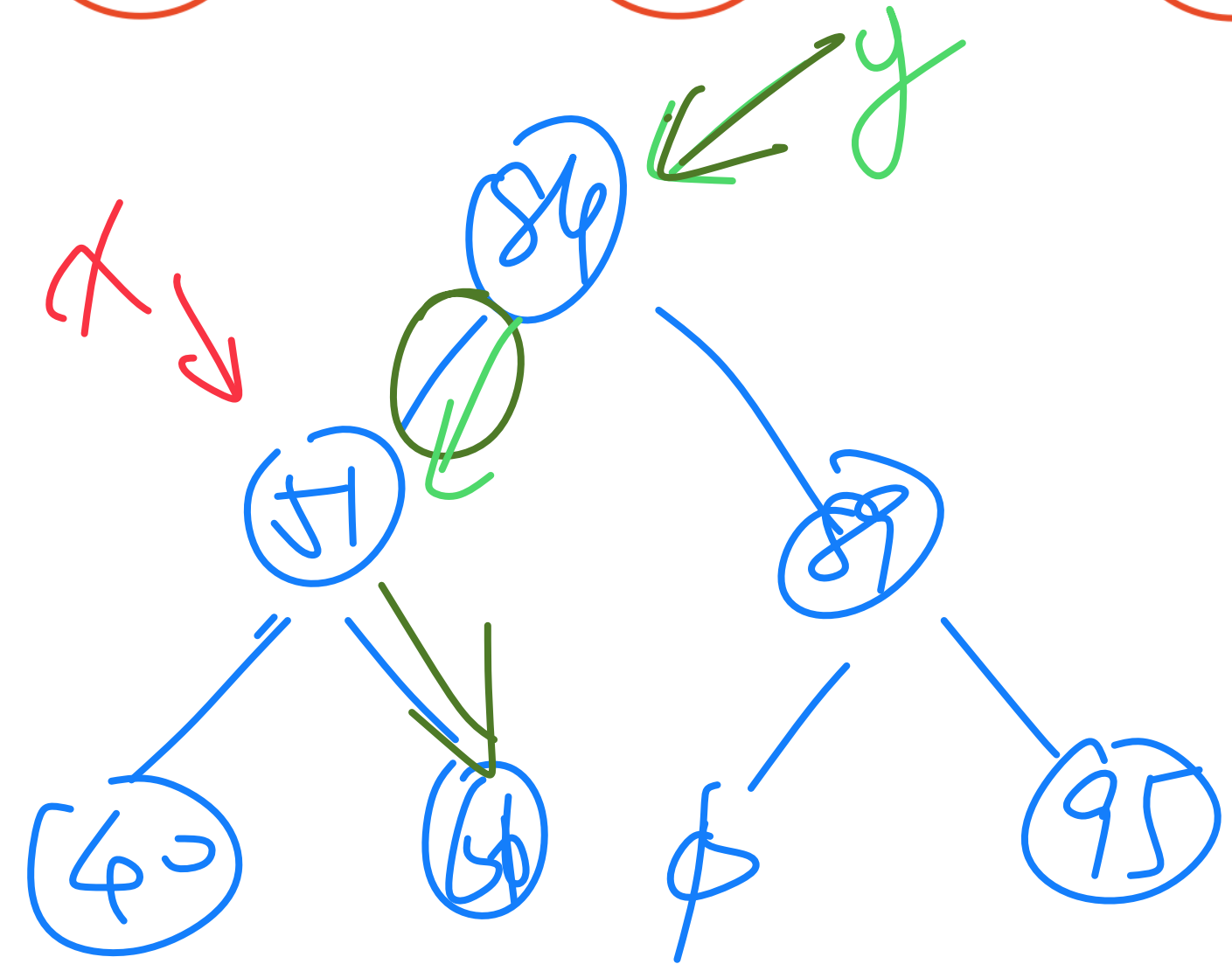
$y = x \rightarrow \text{right} ;$
 $x \rightarrow \text{right} = y \rightarrow \text{left} ;$
 $y \rightarrow \text{left} = x ;$



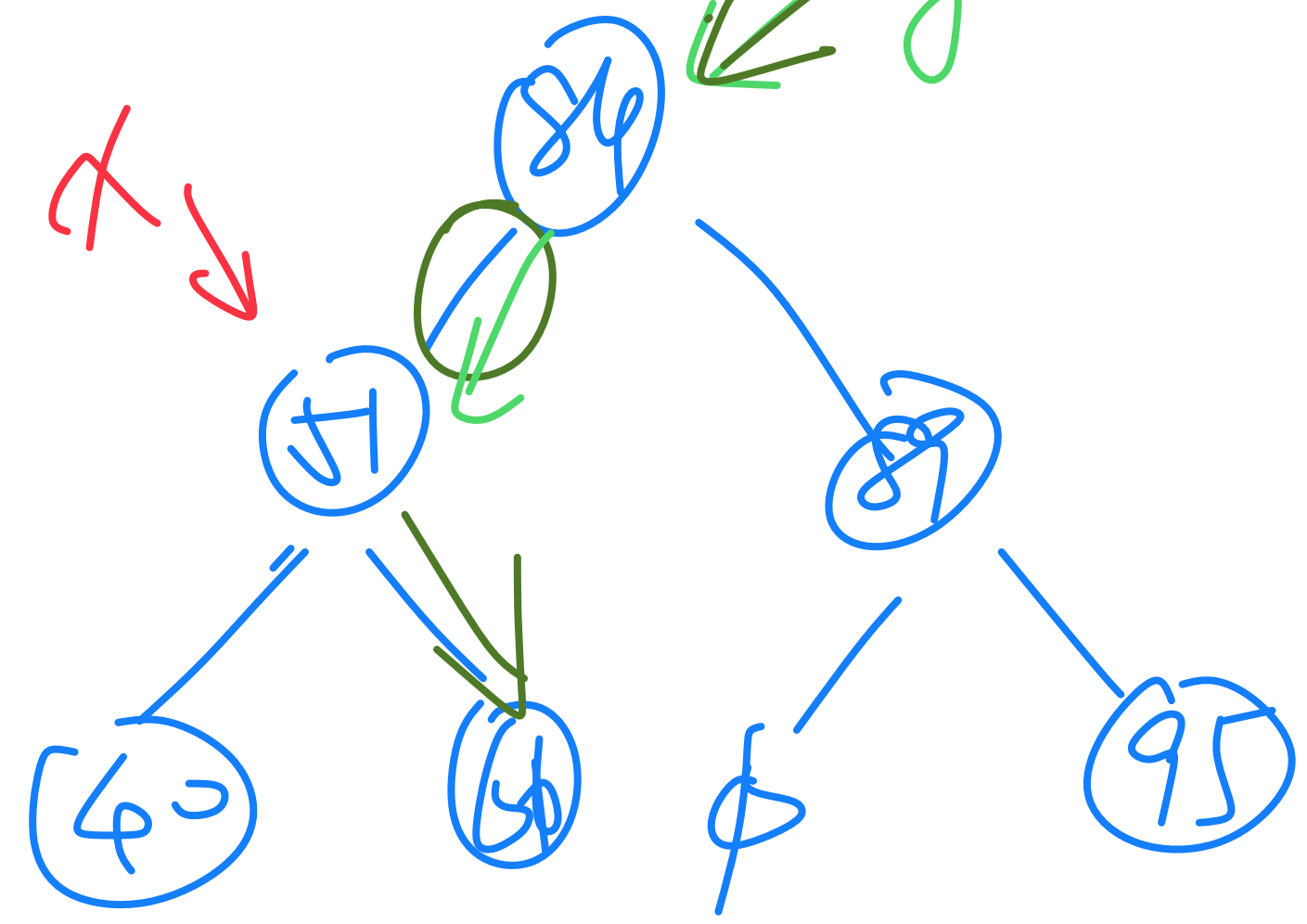
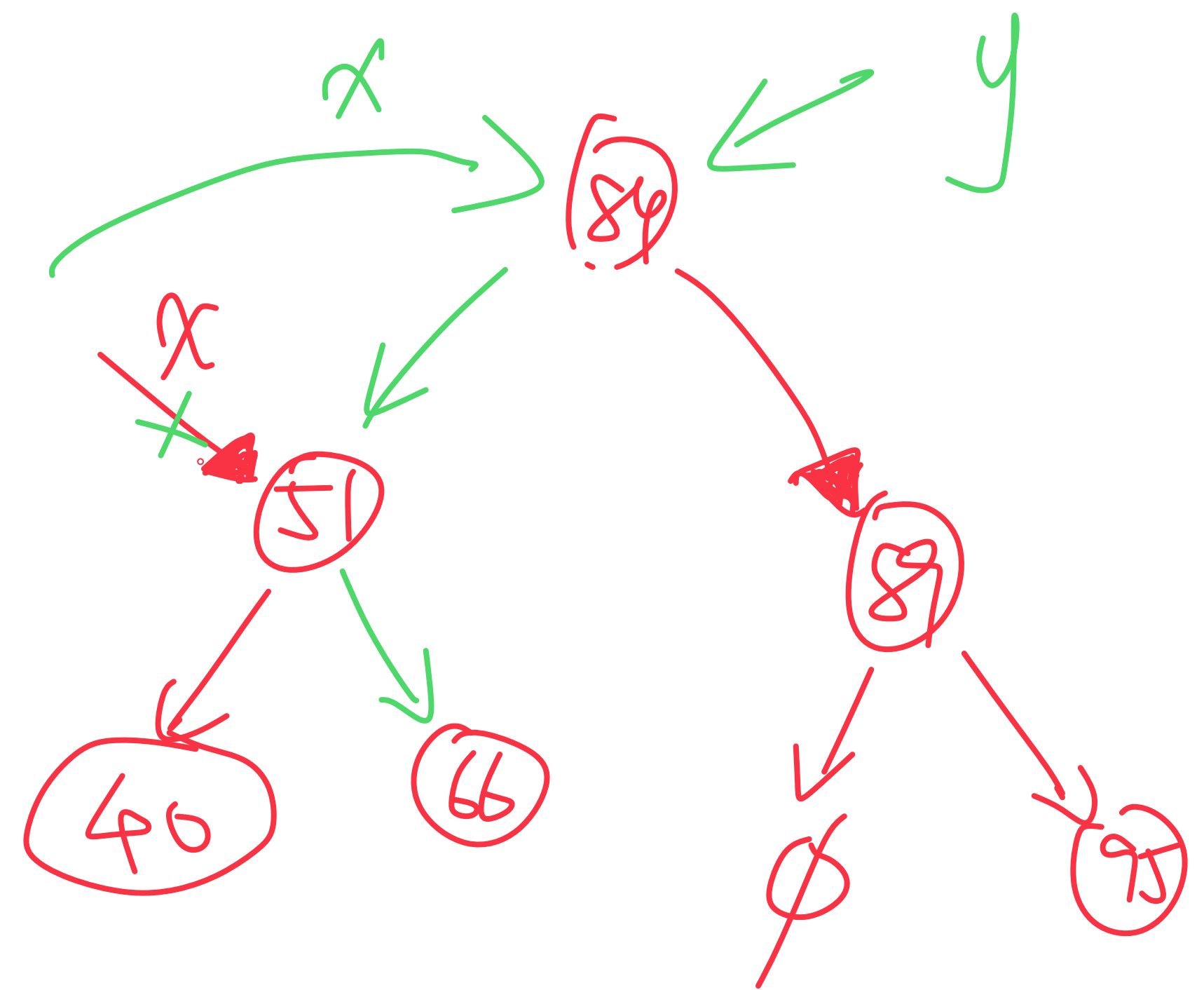
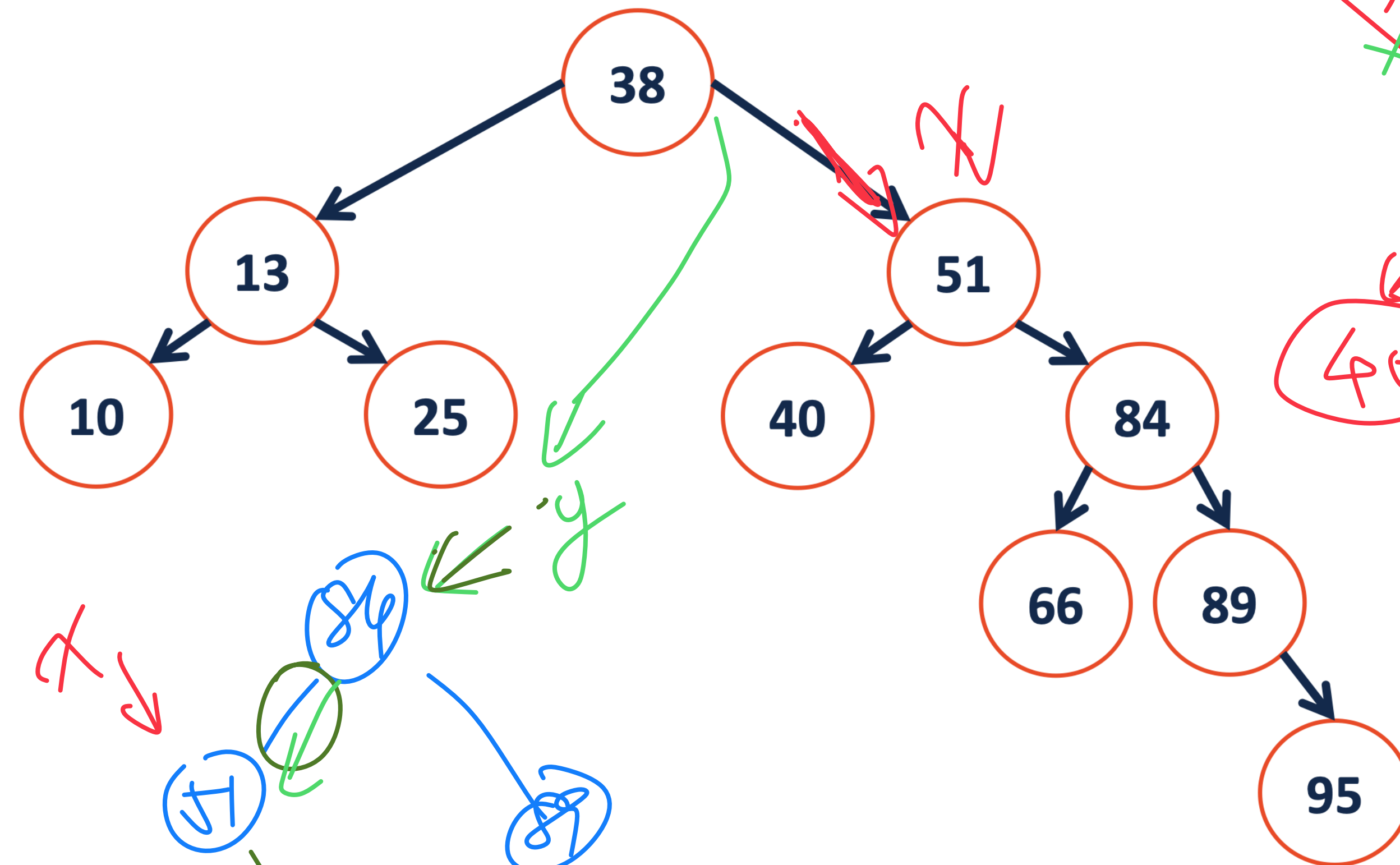
overall procedure



$y = x \rightarrow \text{right} ;$
 $x \rightarrow \text{right} = y \rightarrow \text{left} ;$
 $y \rightarrow \text{left} = x ;$



Overall procedure

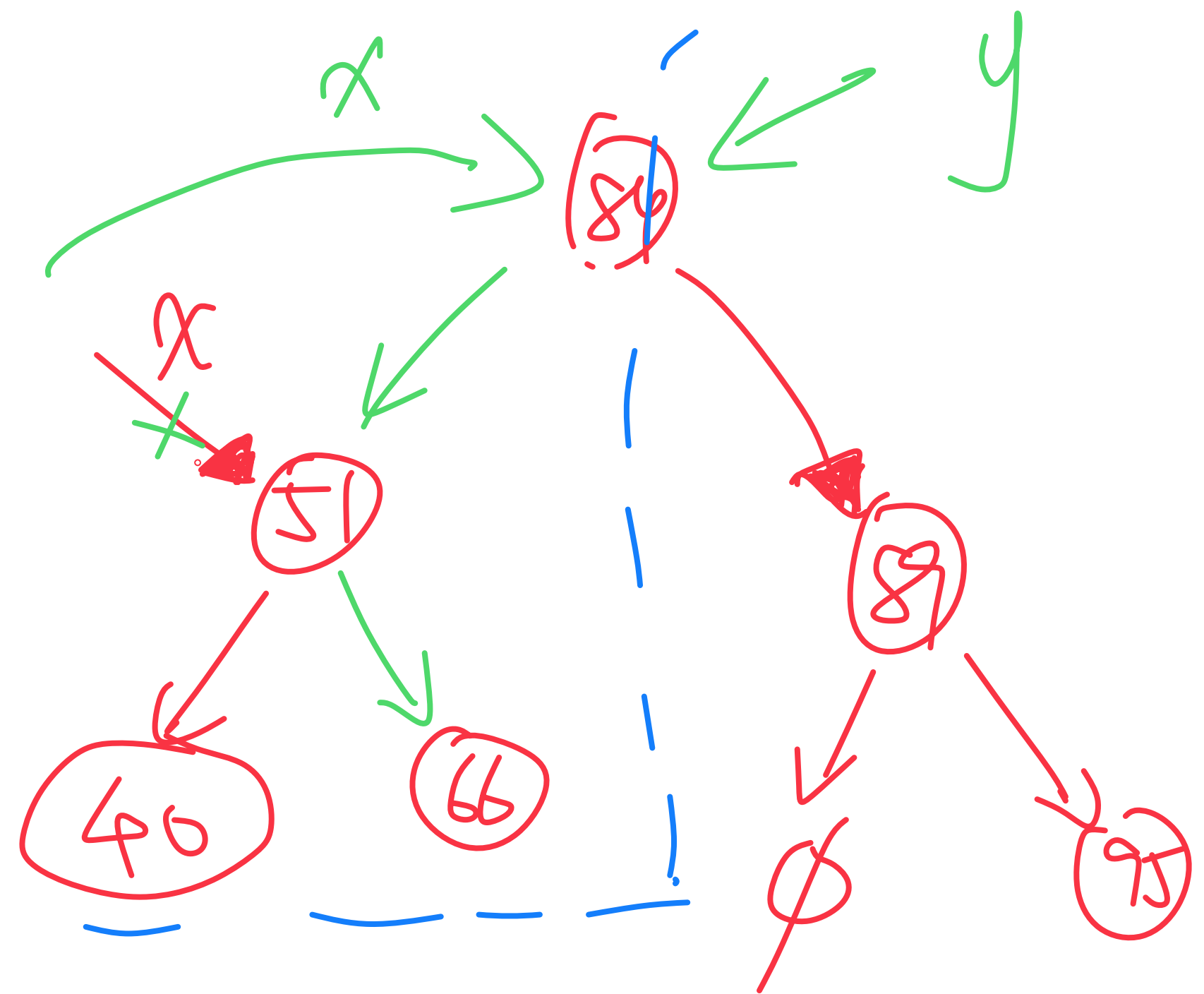
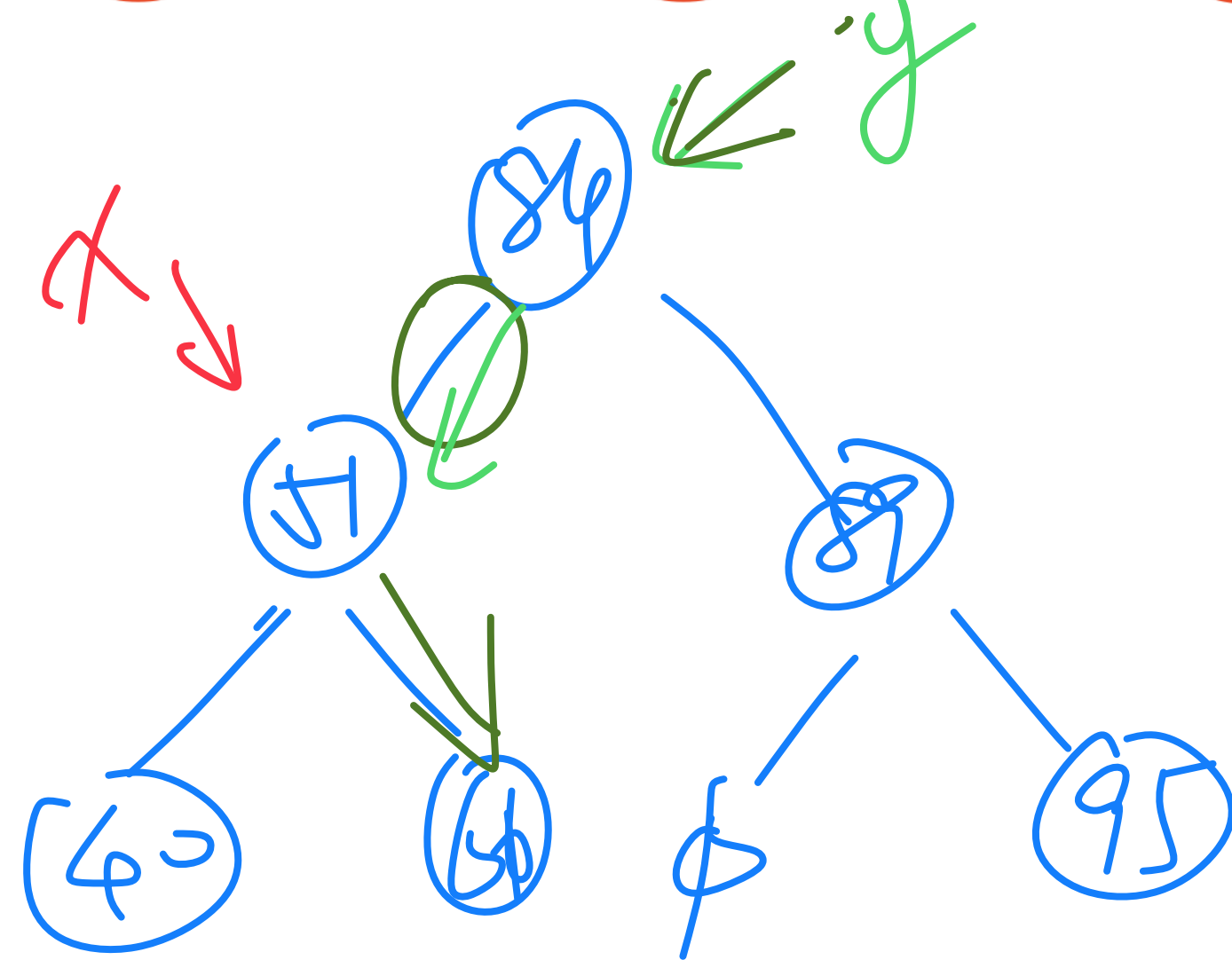
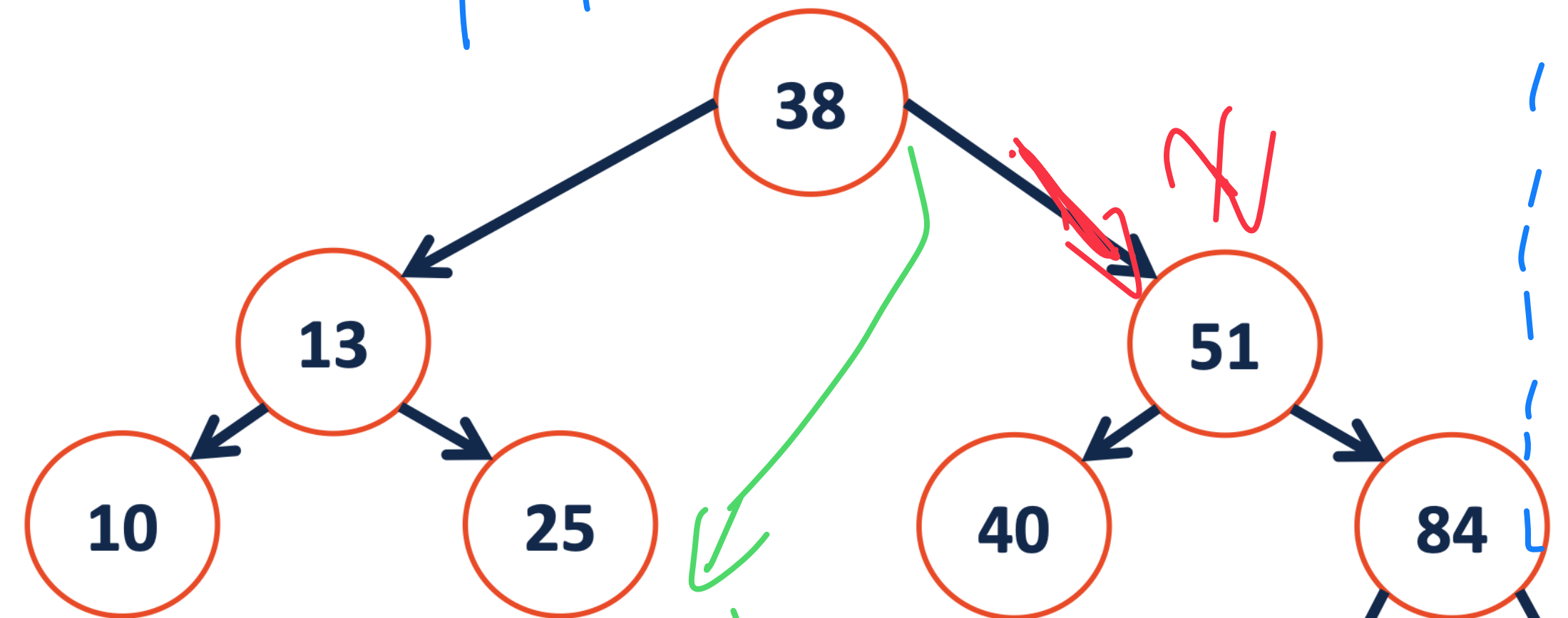


$y = x \rightarrow \text{right} ;$
 $x \rightarrow \text{right} = y \rightarrow \text{left} ;$
 $y \rightarrow \text{left} = x ;$
 $x = y ;$

4 Swaps

= overall procedure

- Left Rotation
↳ swap pointers in the left child



$y = x \rightarrow \text{right} ;$

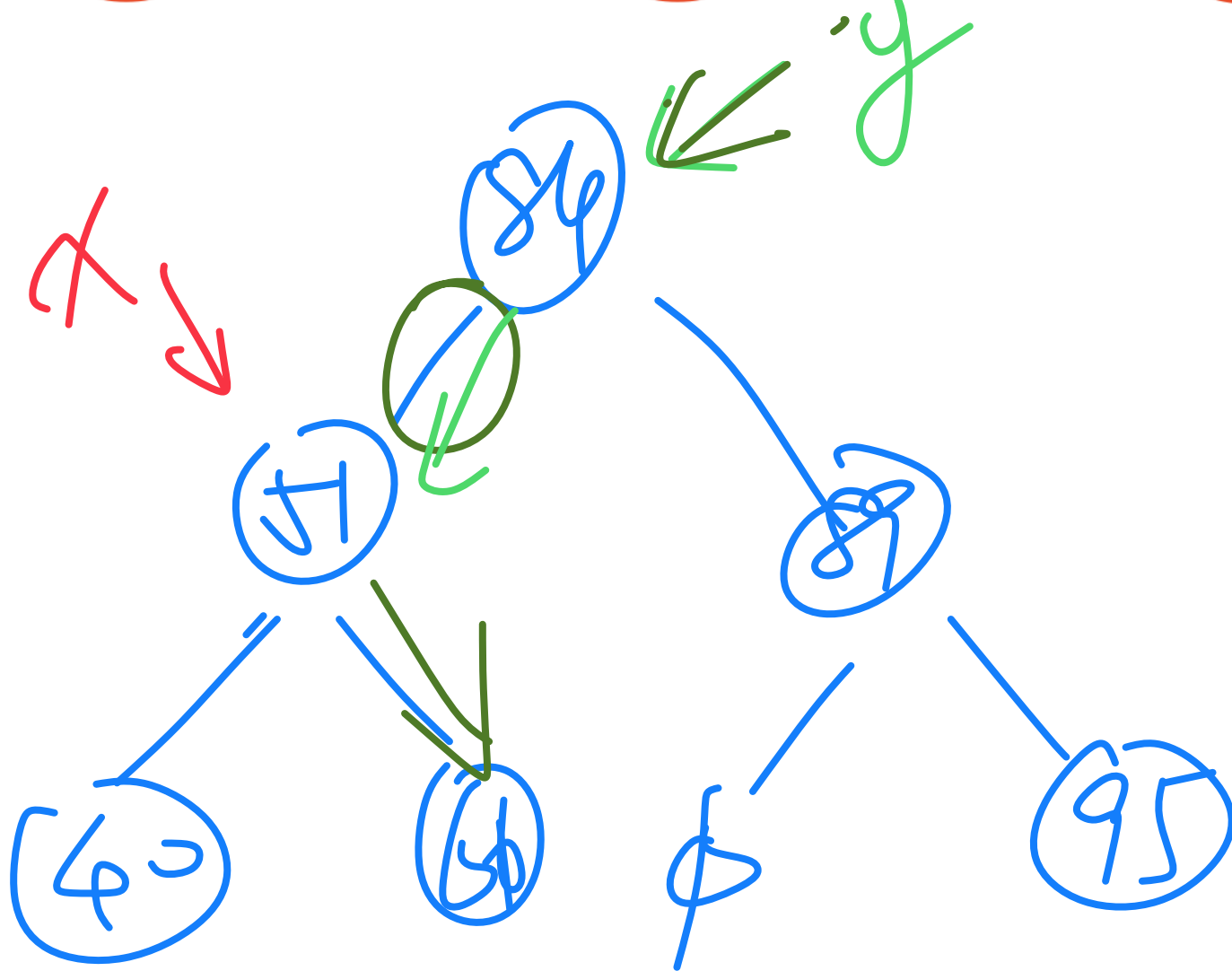
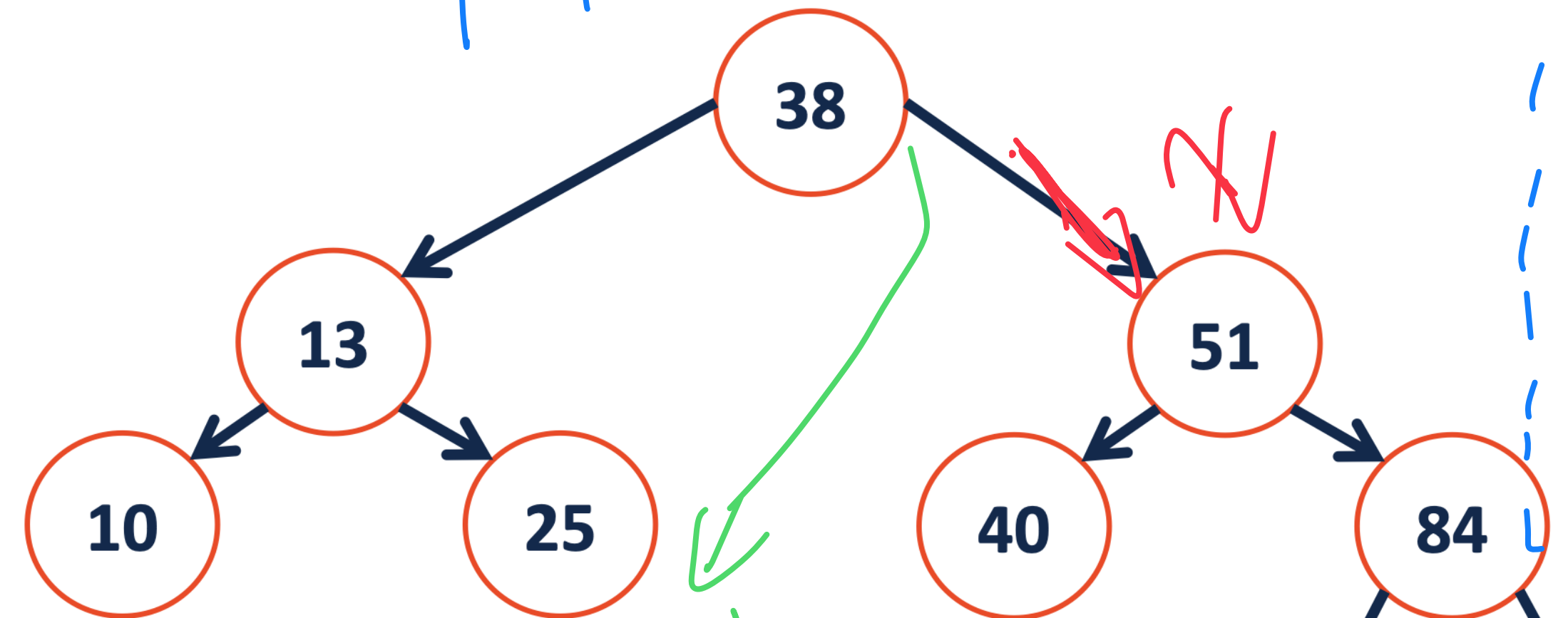
$x \rightarrow \text{right} = y \rightarrow \text{left} ;$

$y \rightarrow \text{left} = x ;$

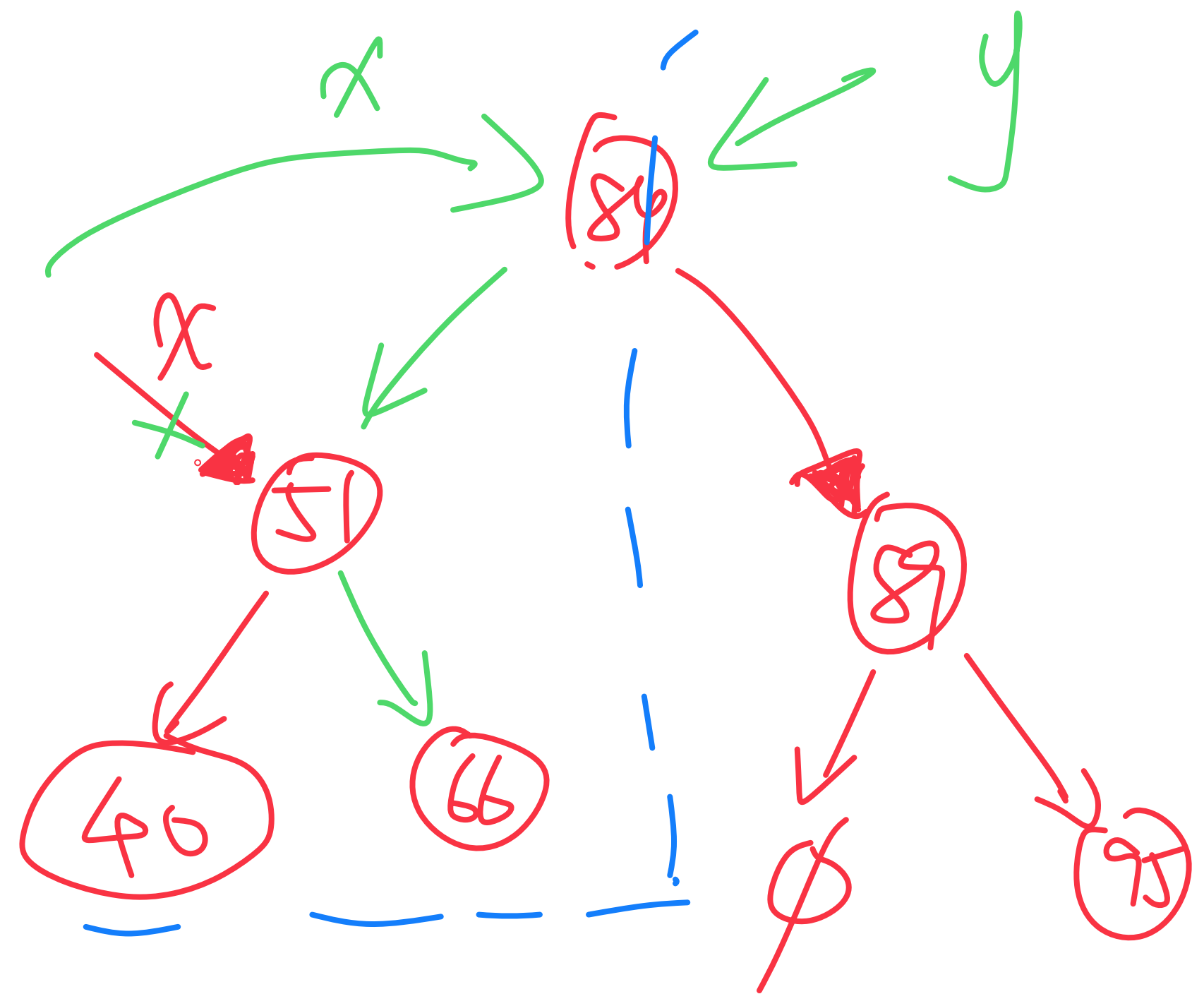
$x = y ;$

= overall procedure

- Left Rotation
 ↳ swap pointers in the left child



- runtime: $O(1)$, constant operation.

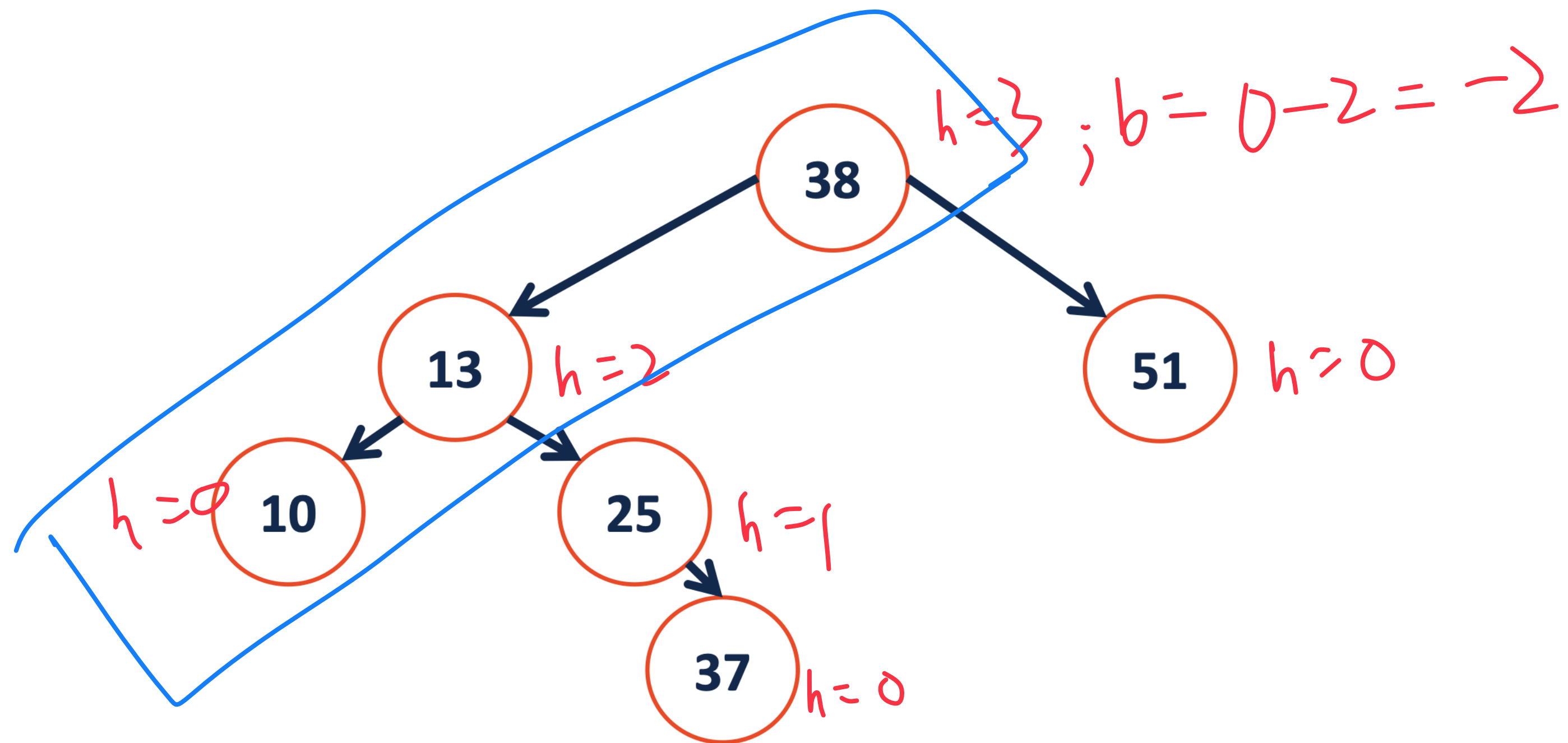


$y = x \rightarrow \text{right} ;$

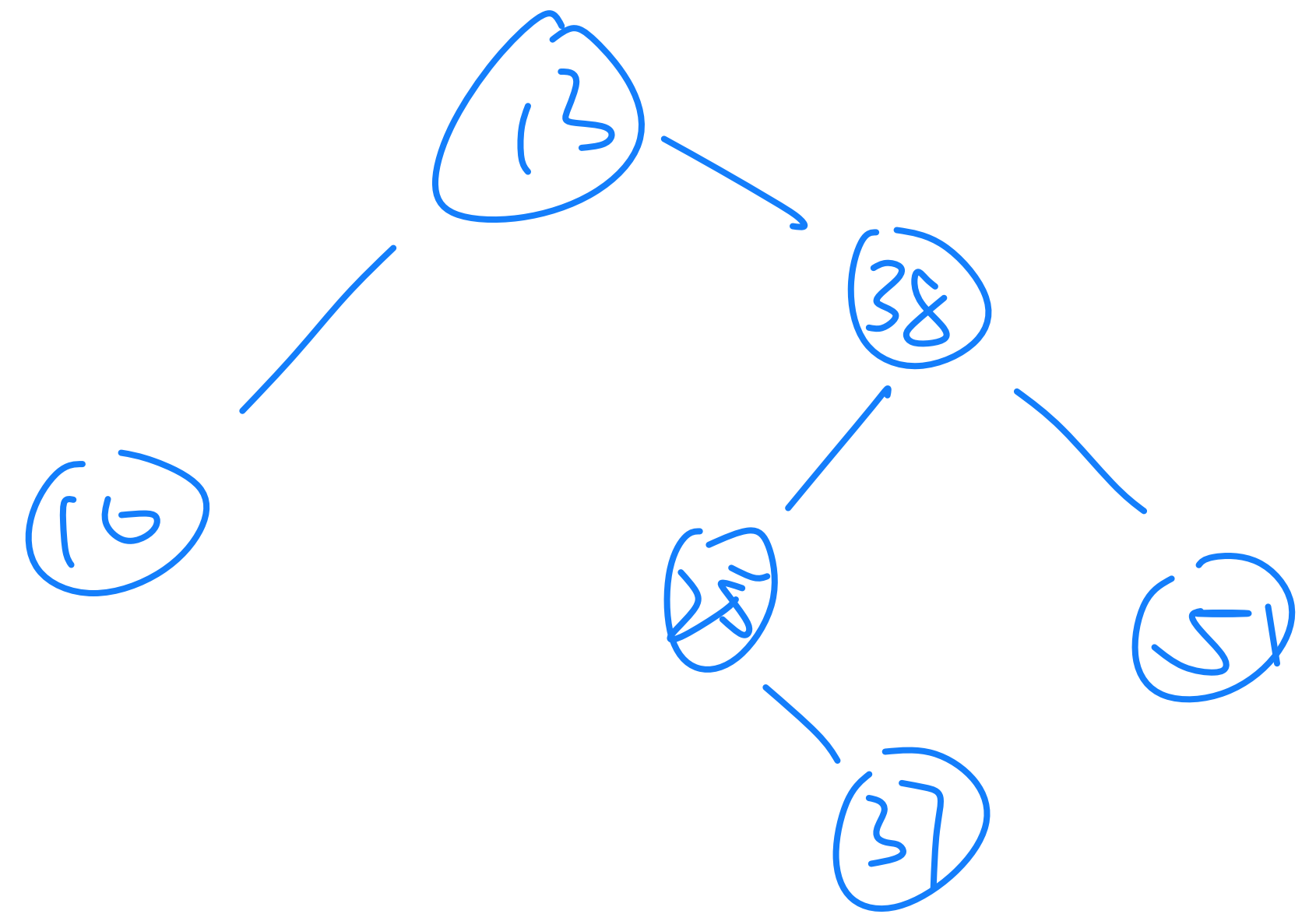
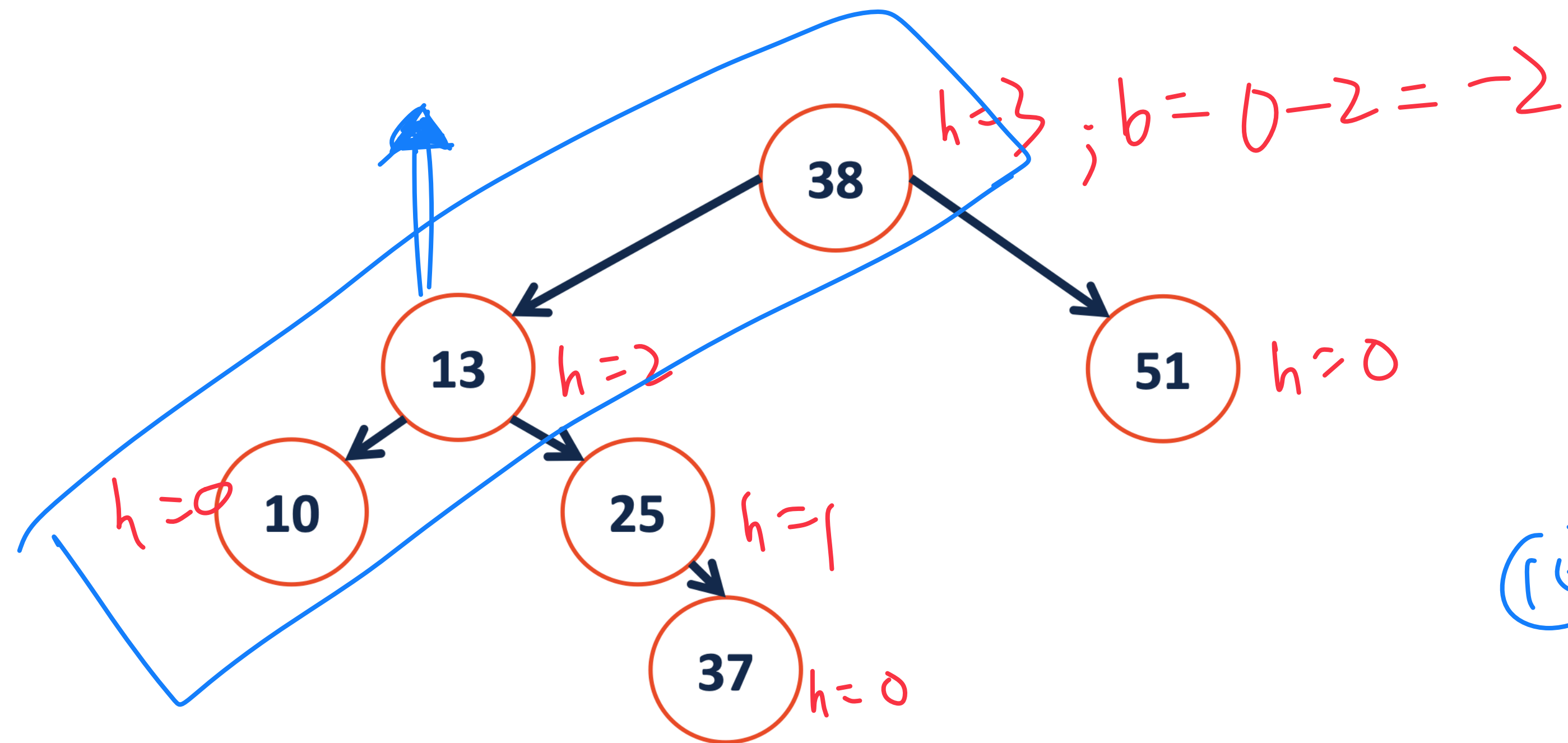
$x \rightarrow \text{right} = y \rightarrow \text{left} ;$

$y \rightarrow \text{left} = x ;$

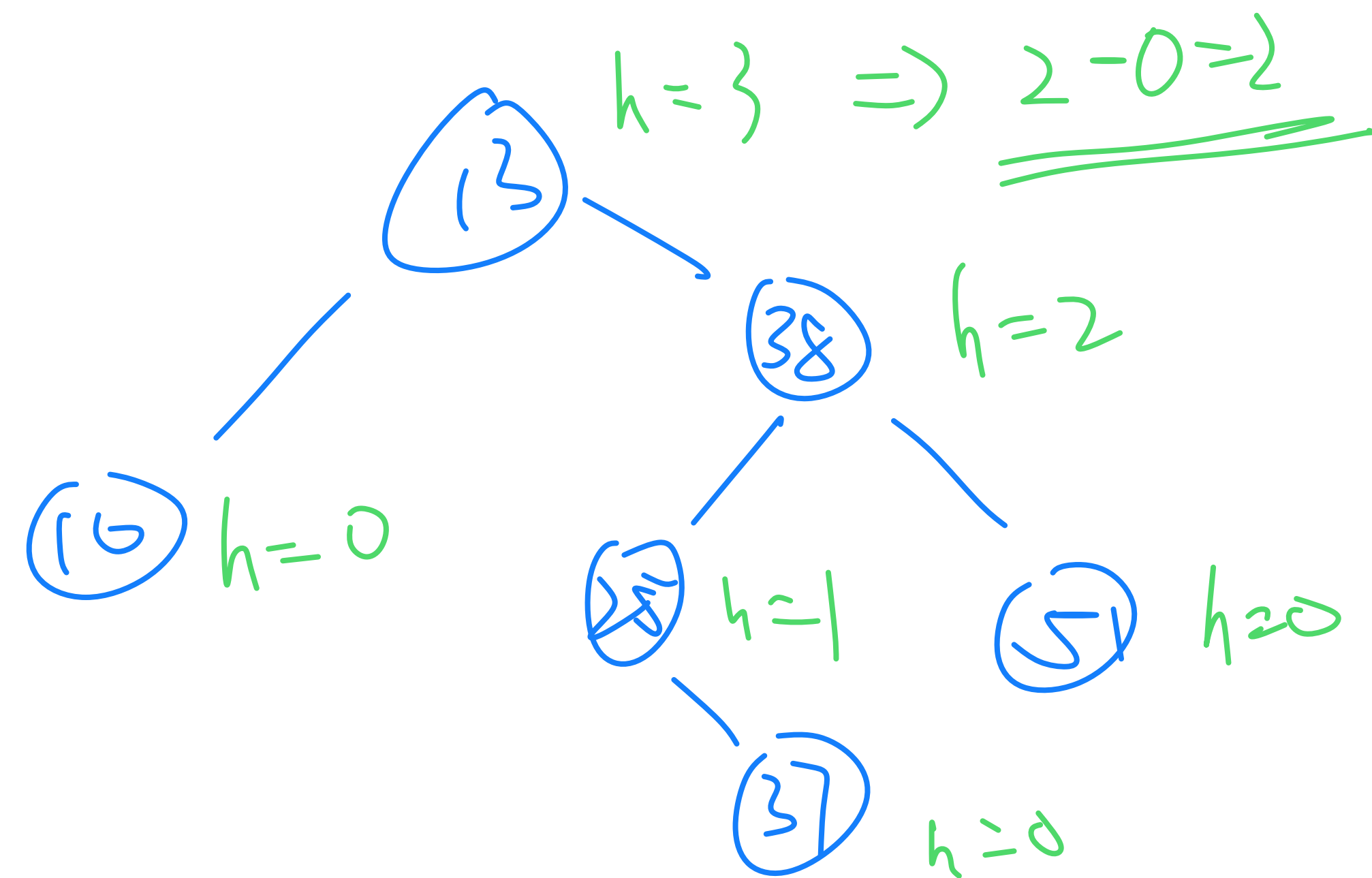
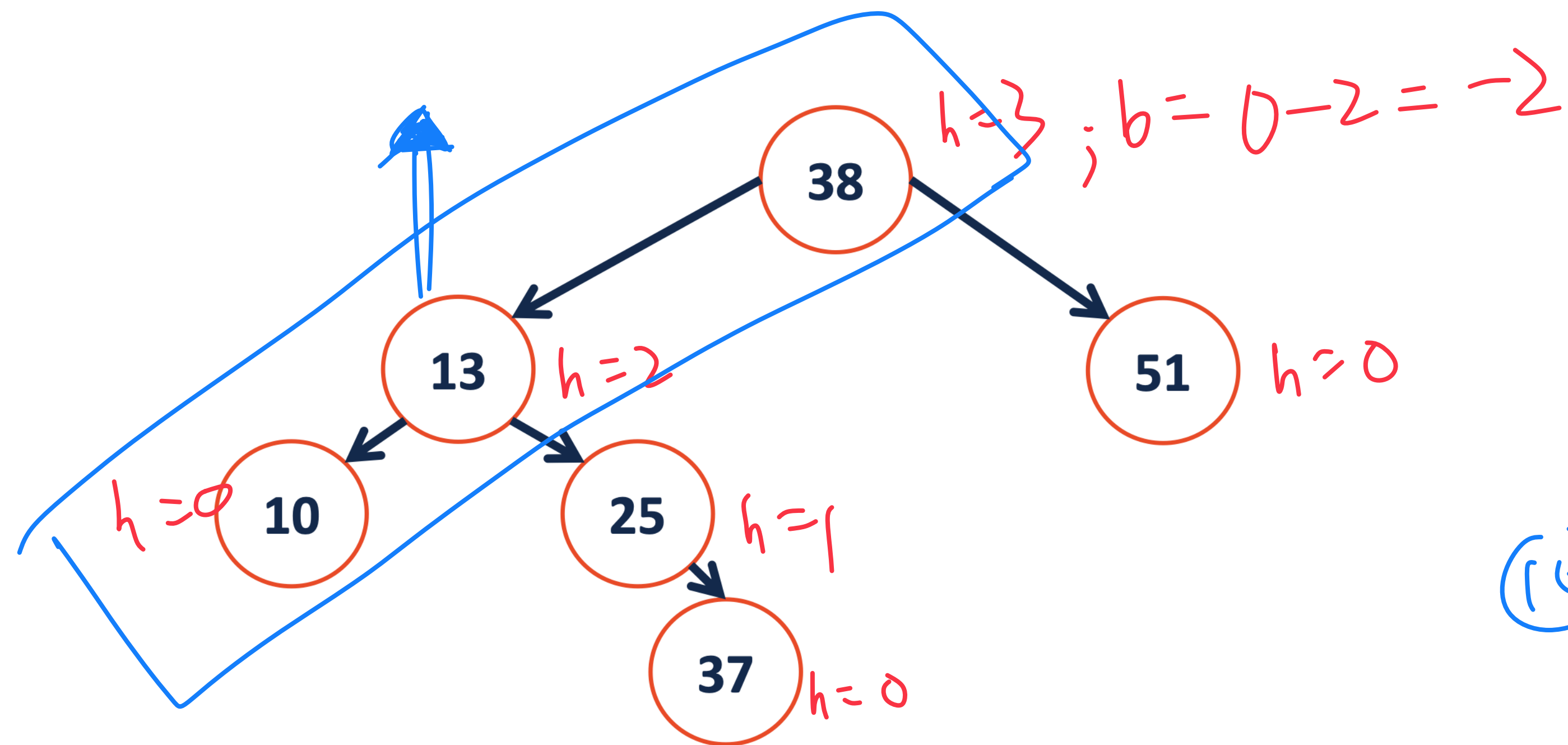
$x = y ;$



- right rotation : similar to left rotation

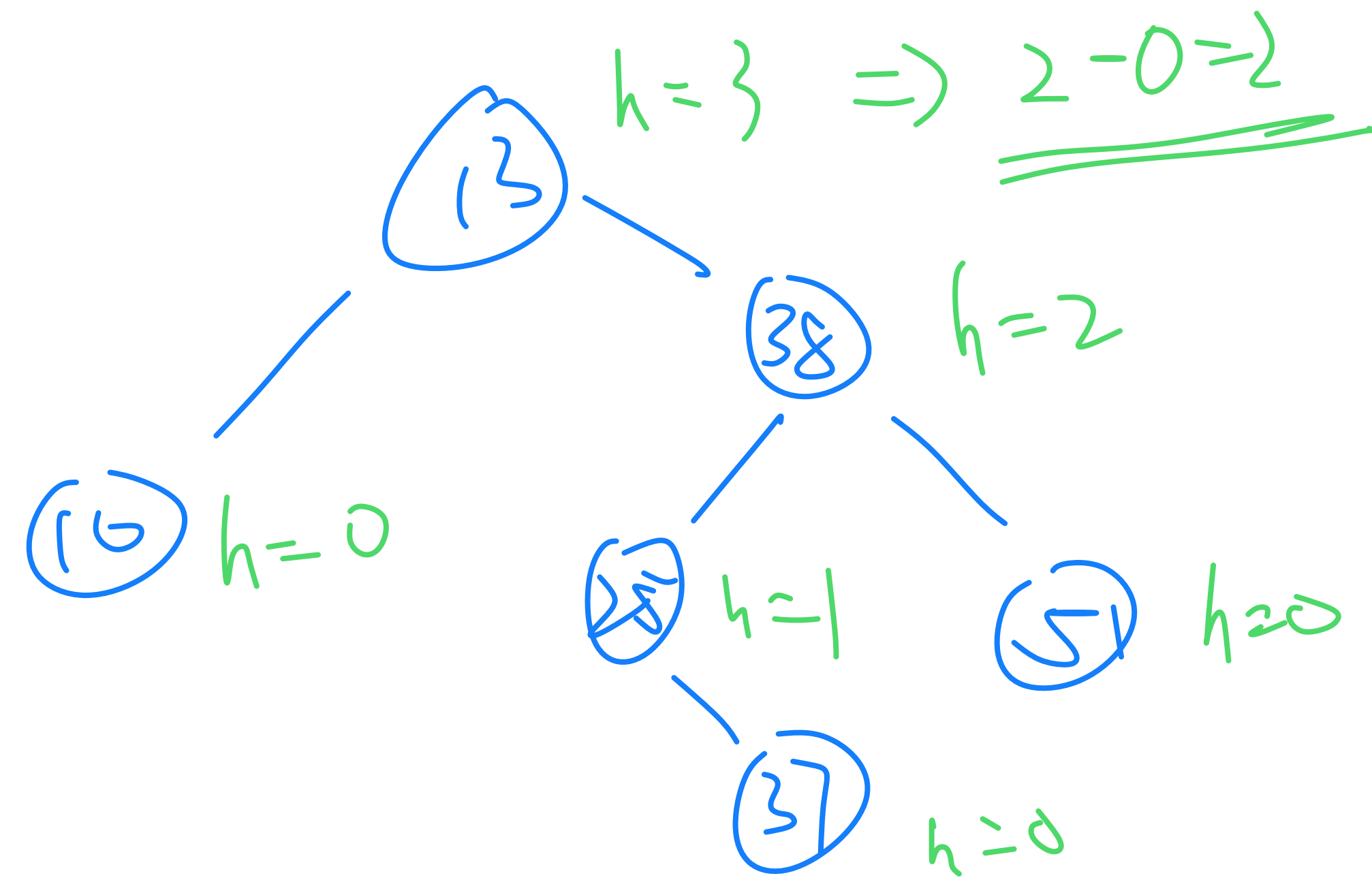
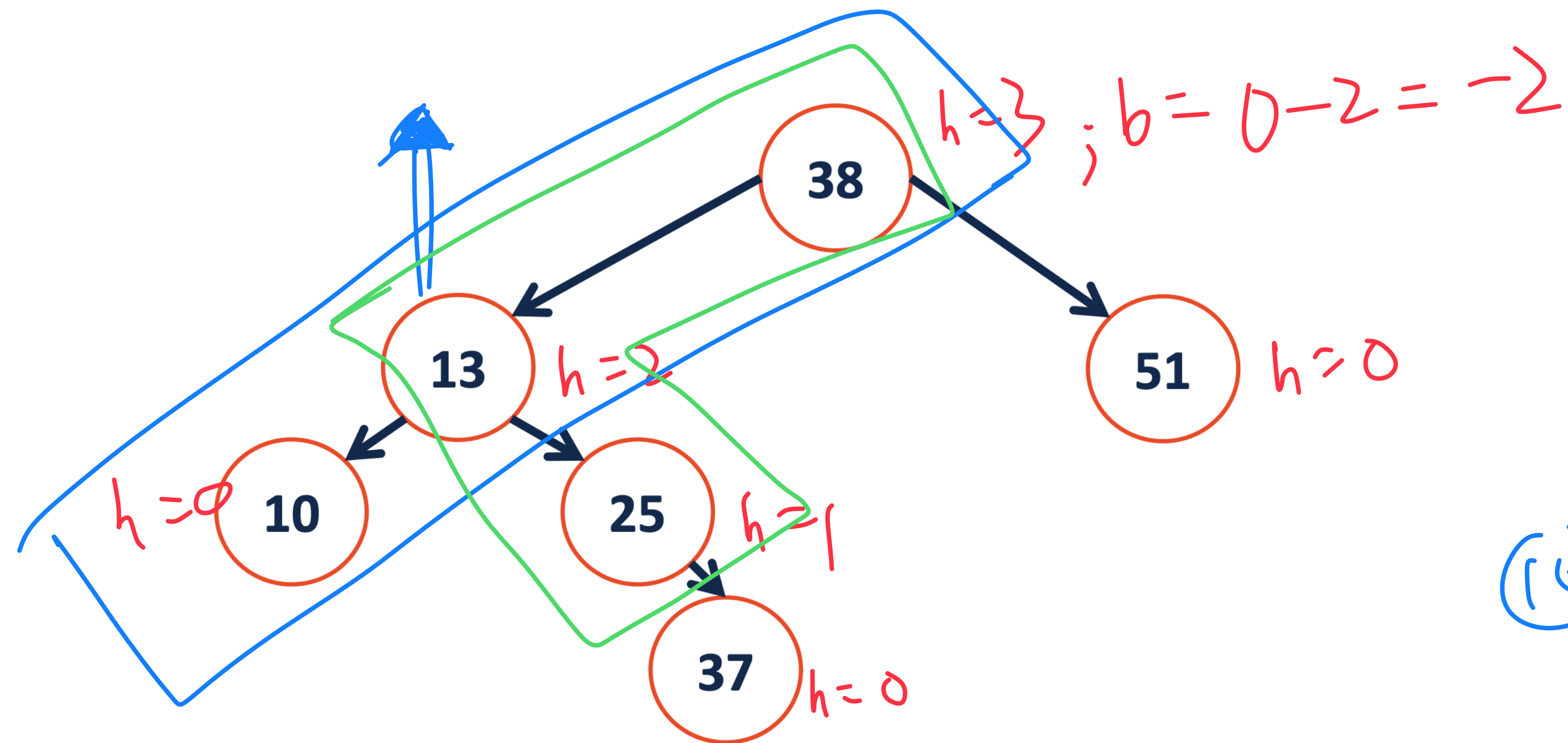


- right rotation : similar to left rotation



Still imbalance ??

- right rotation : similar to left rotation

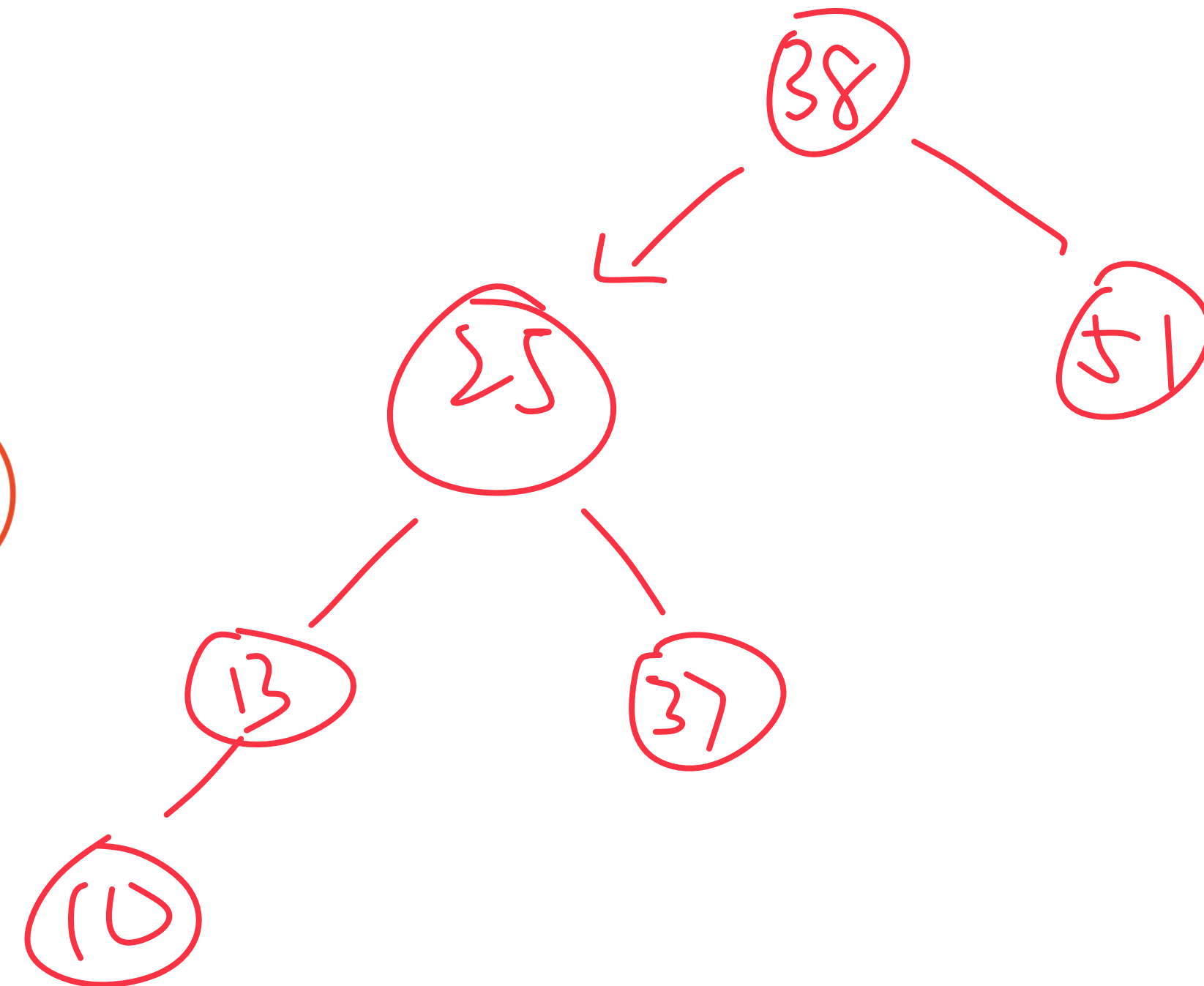
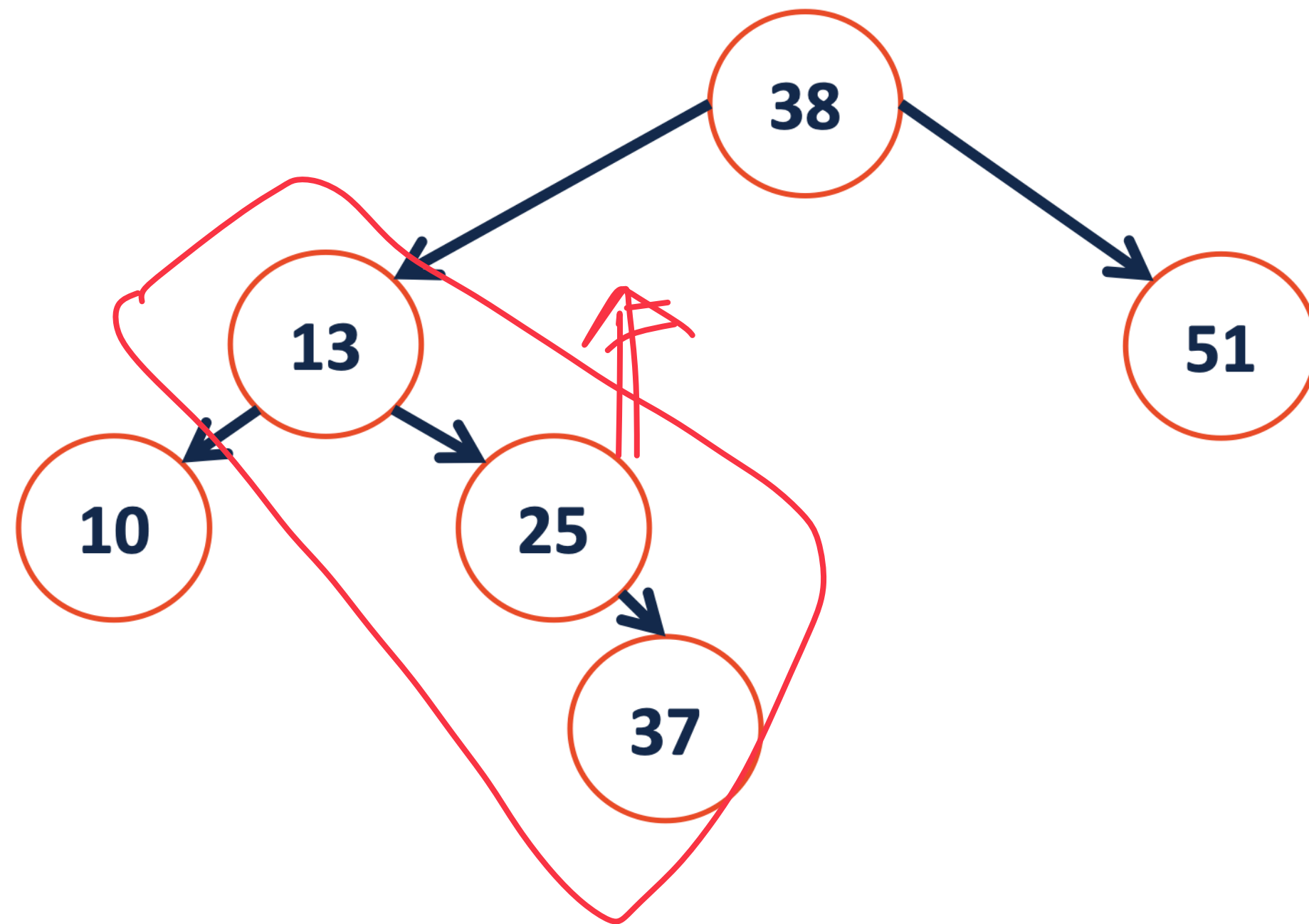


- stick is not the cause

- "Leg" is the cause, '38 $\rightarrow$ 13 $\rightarrow$ 25' Still imbalance ??

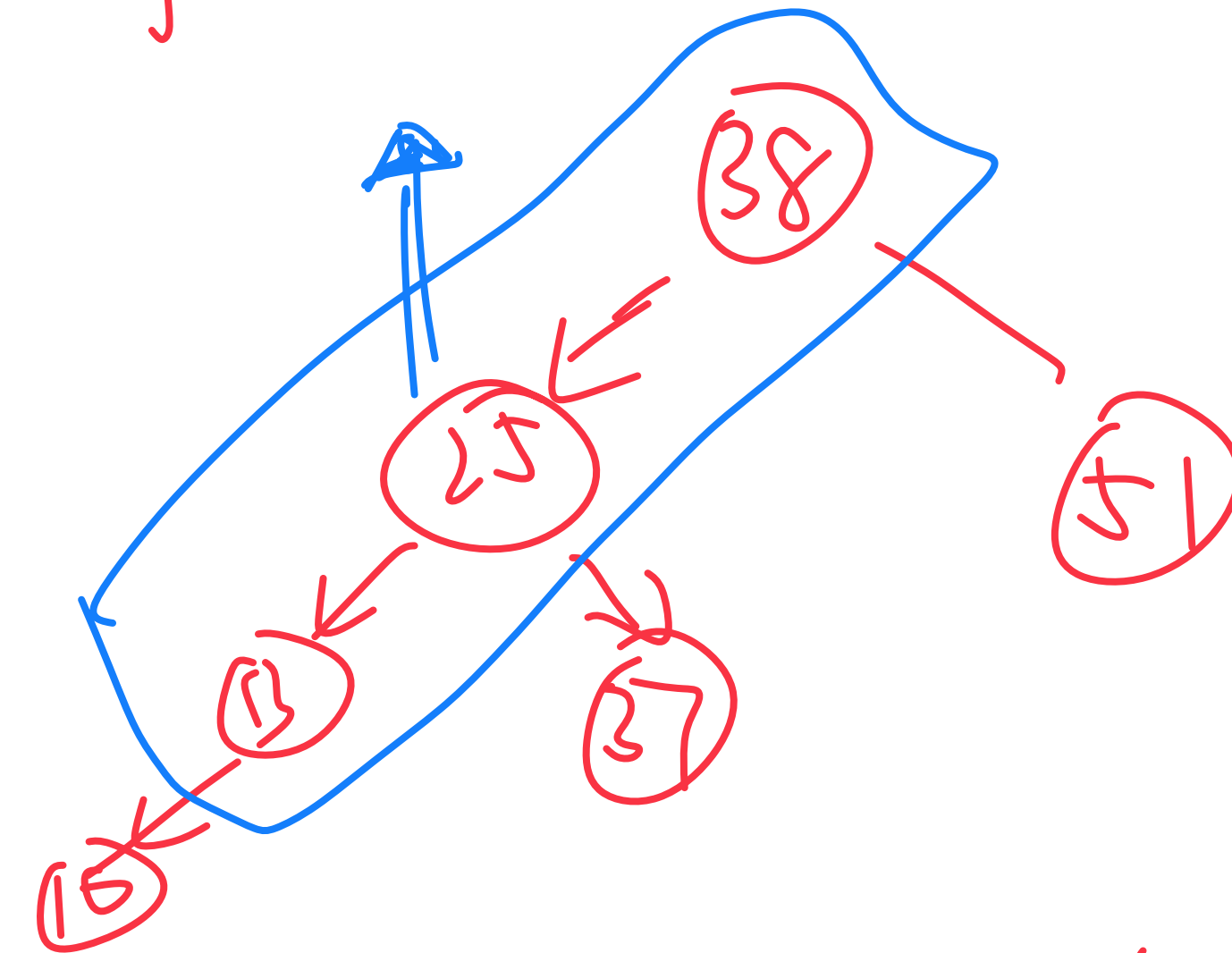
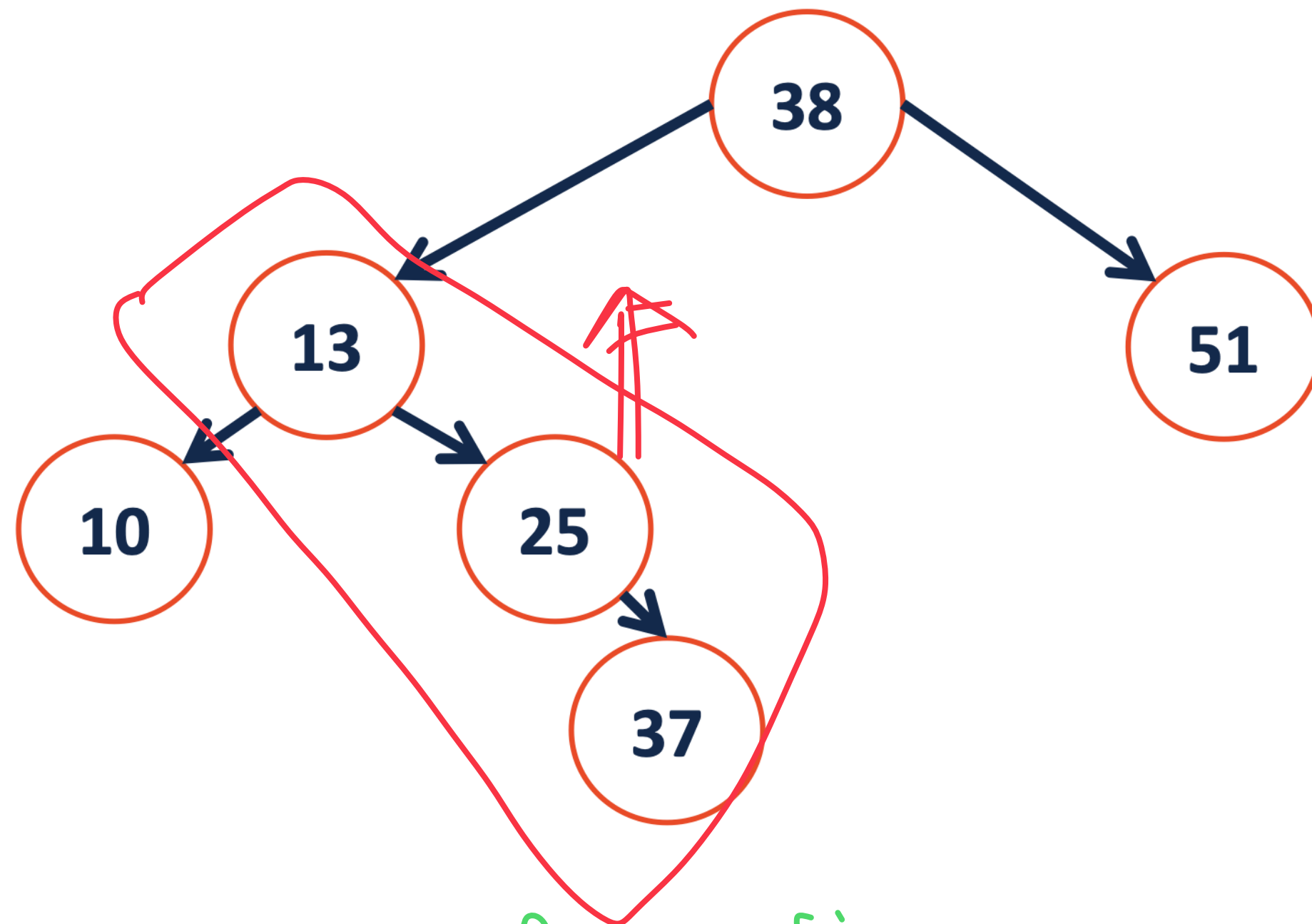
- Two step process

① Left rotation about "13"

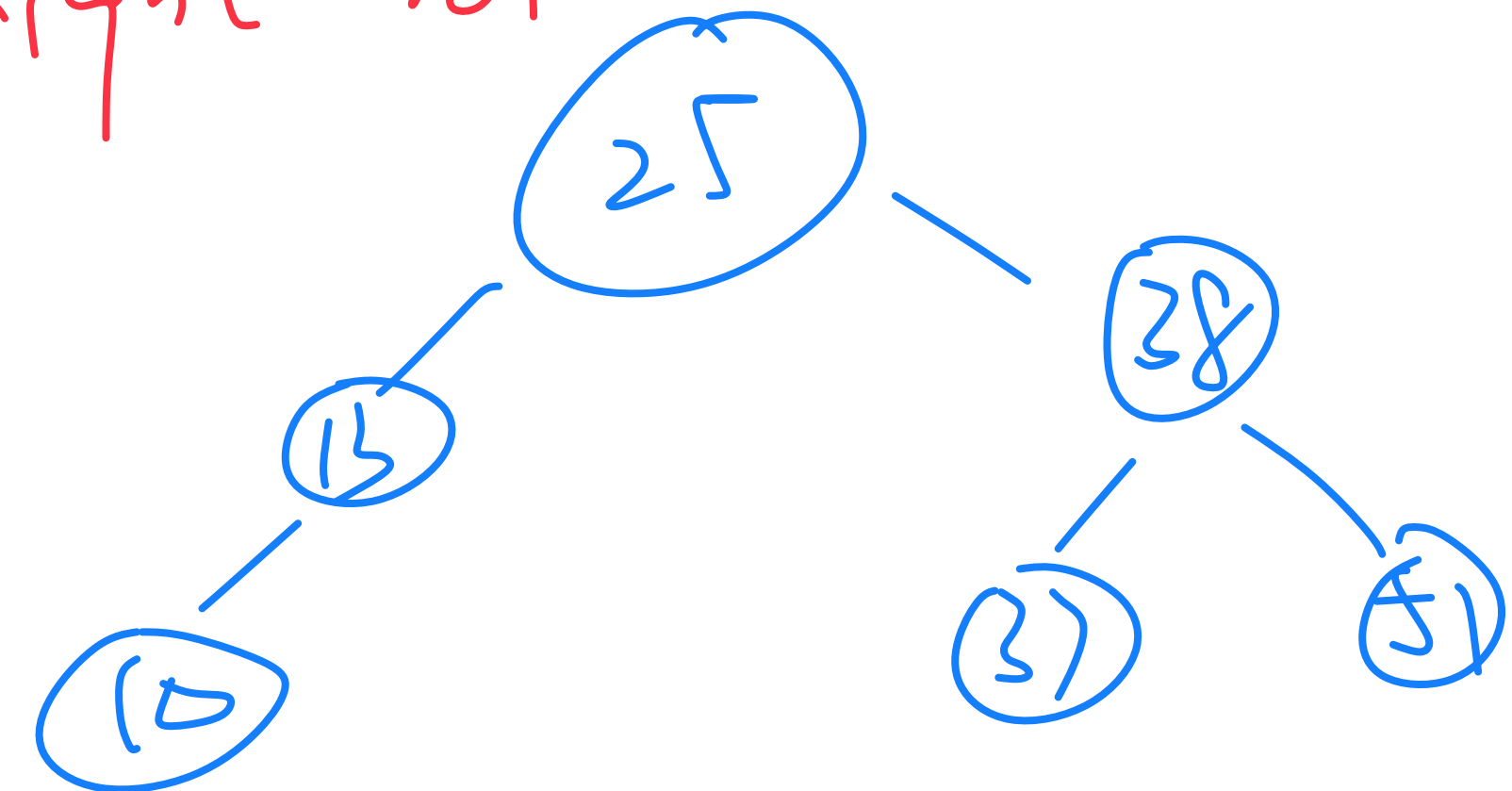


- Two step process

① Left rotation about "13"



② right rotation about "38"



Left-Right Rotation

Balanced??

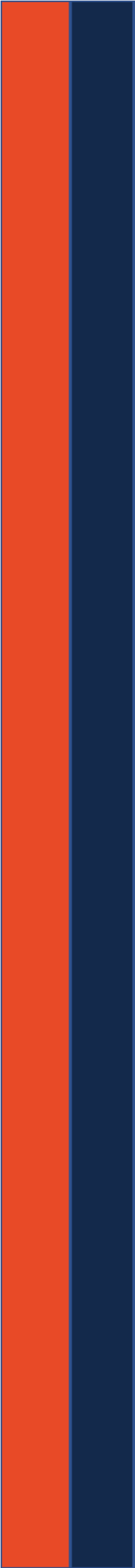
Yes

BST Rotation Summary

- Four kinds of rotations (L, R, LR, RL)
- All rotations are local (subtrees are not impacted)
- All rotations are constant time: $O(1)$
- BST property maintained

GOAL: Create a data structure with
height balanced BSTs

We call these trees:
Adelson-Velskii and Landis $\Rightarrow$ AVL Trees
1962

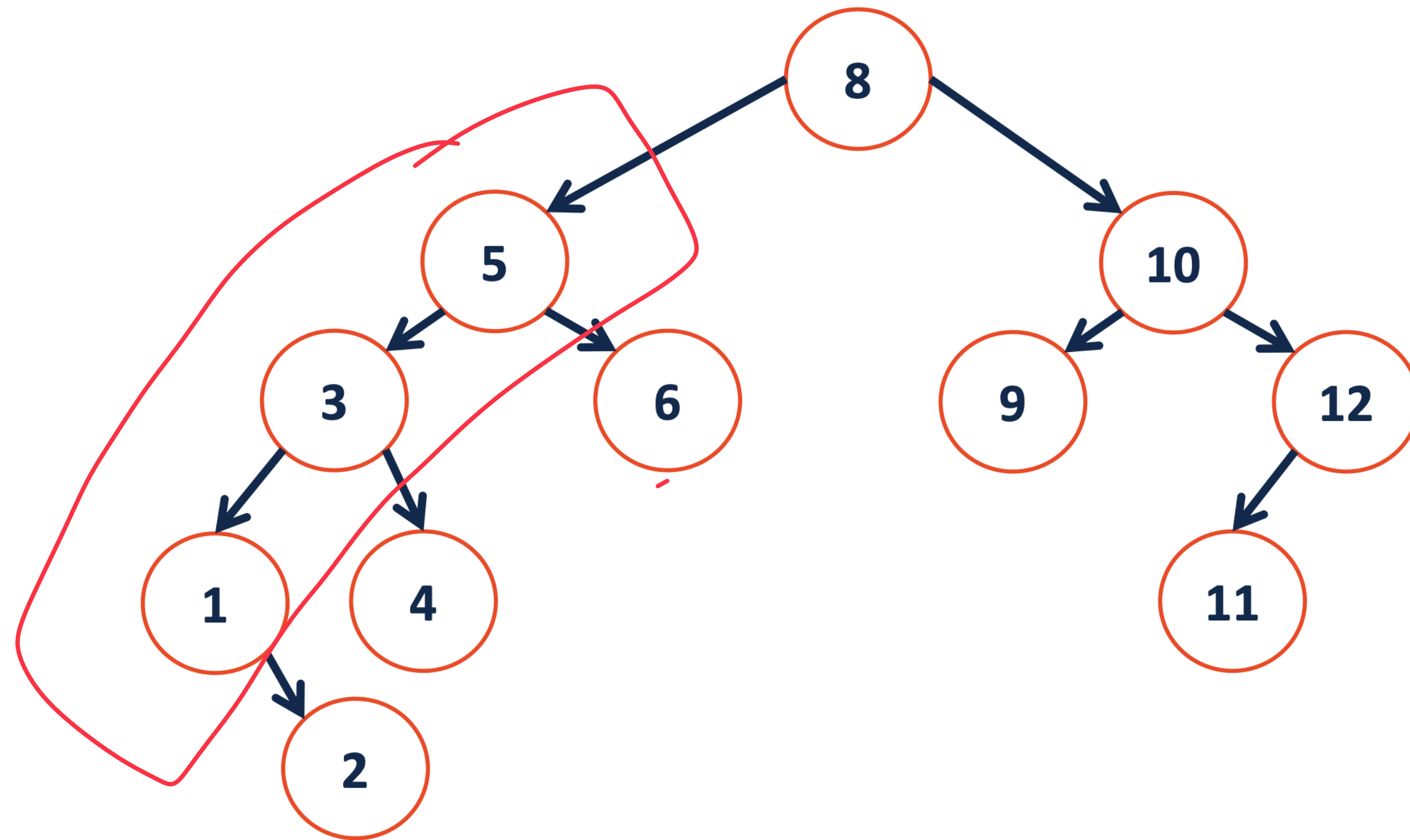


AVL Trees

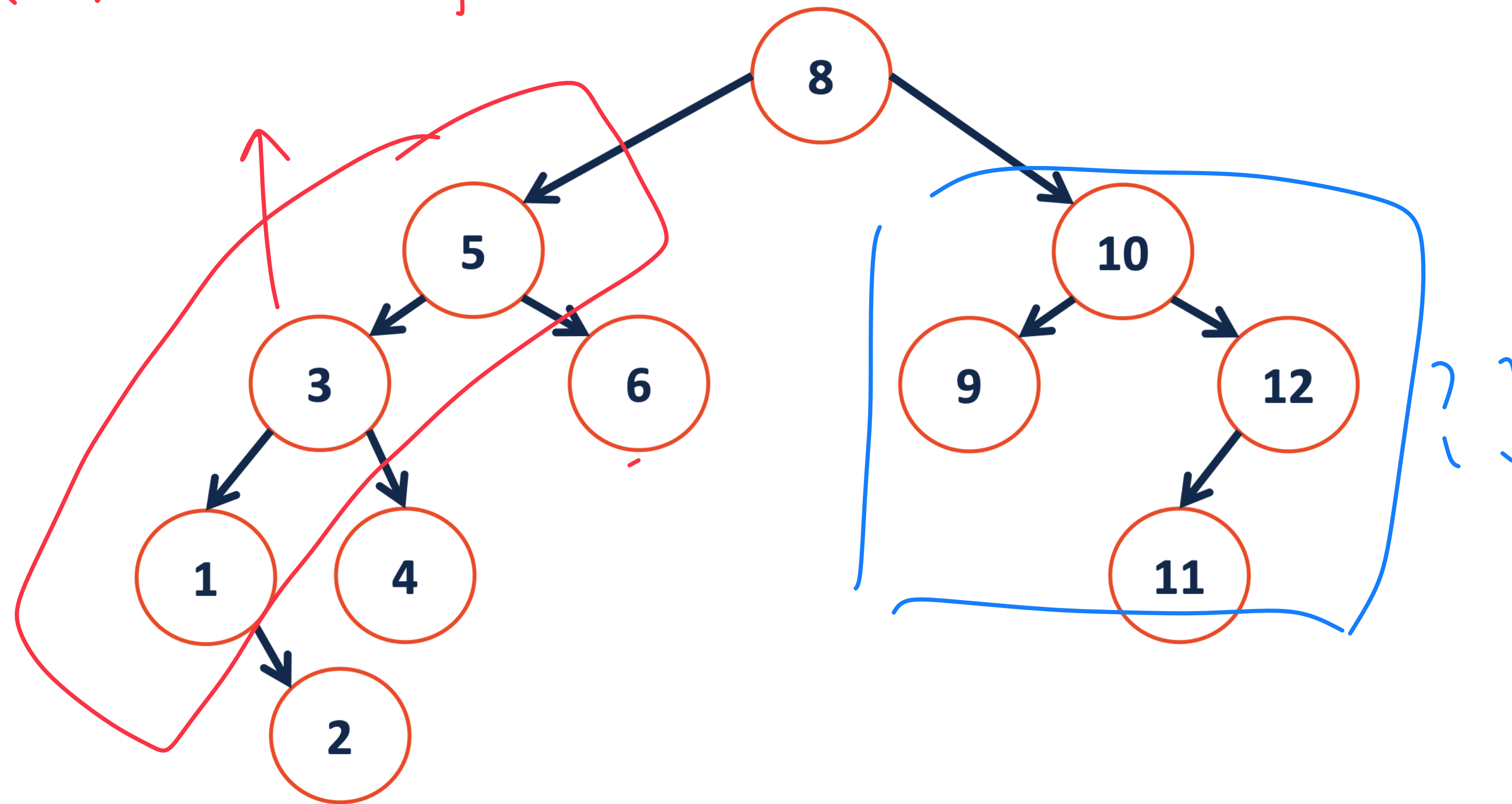
Three issues for consideration:

- Rotations
- Maintaining Height
- Detecting Imbalance

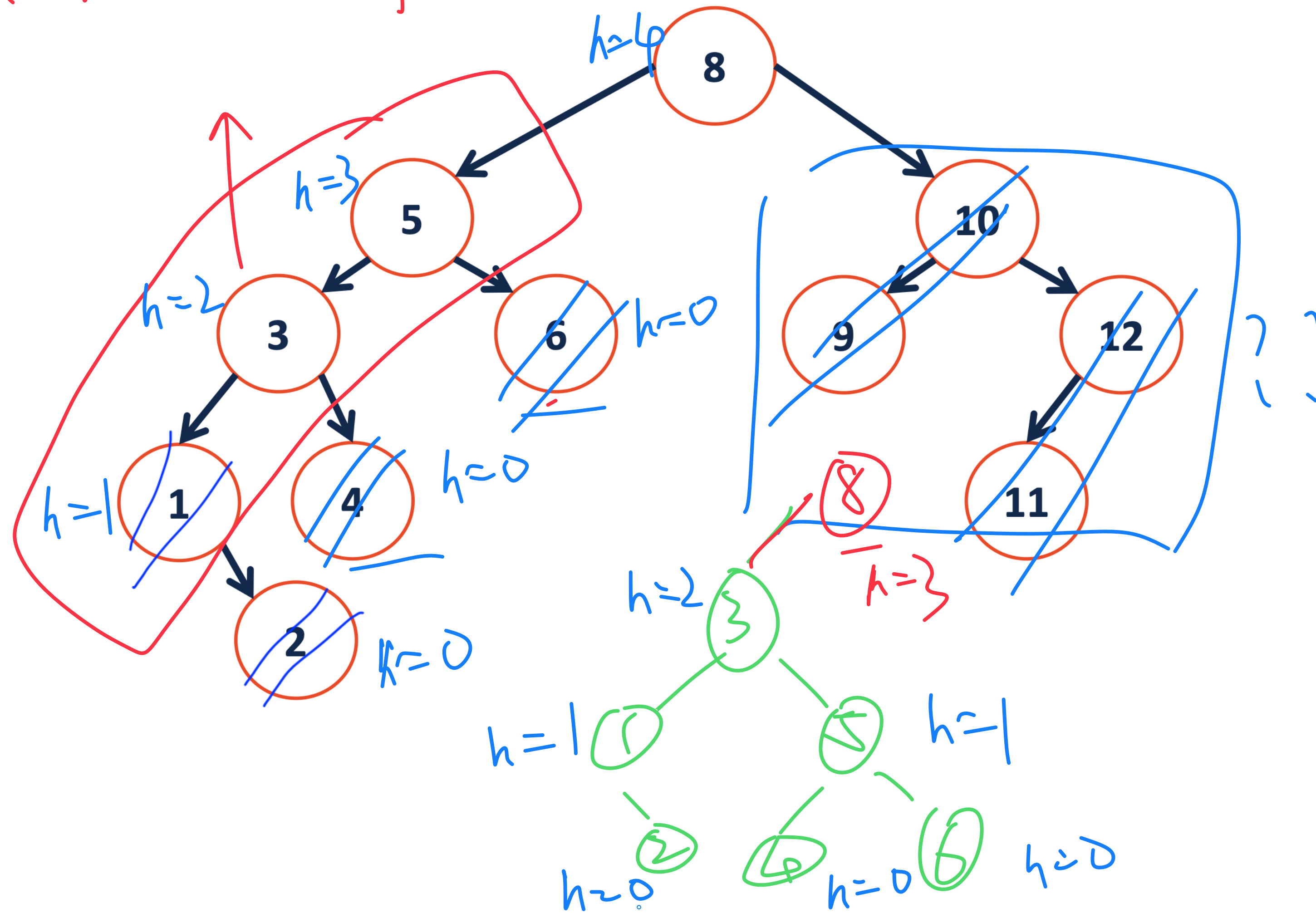
- lowest node of imbalance ?? "5"



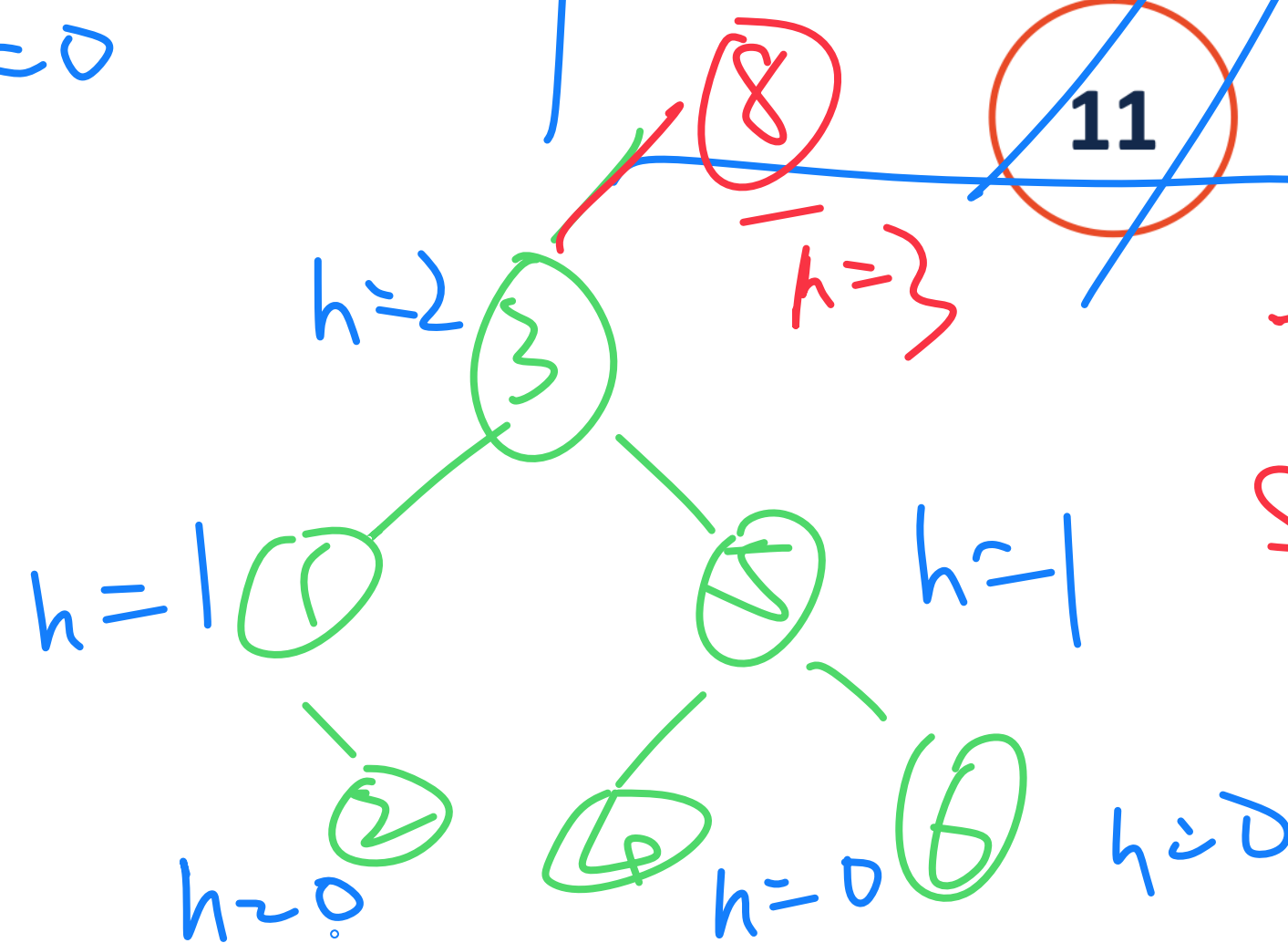
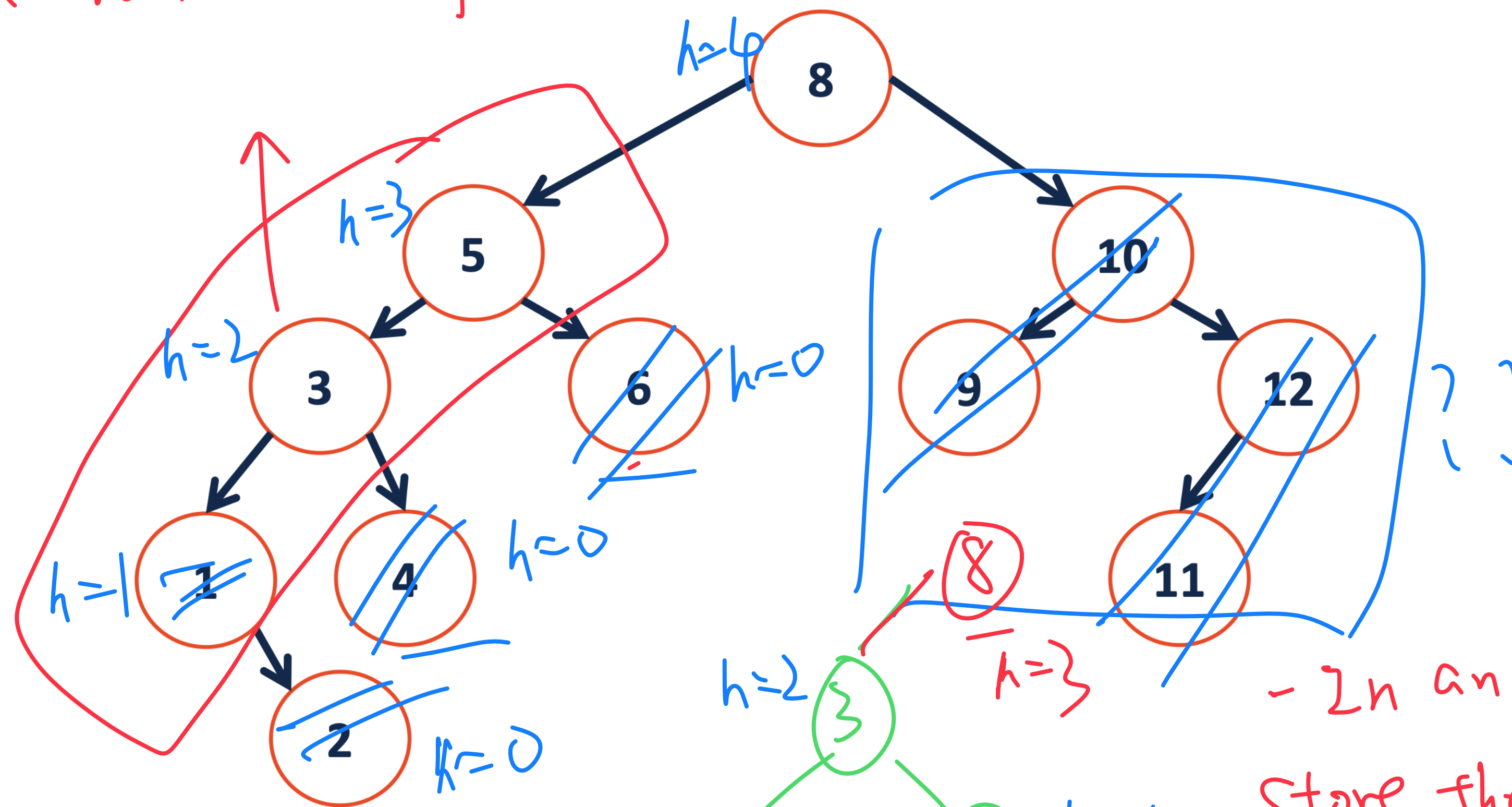
- lowest node of imbalance ?? "5"
- right rotation of "5 → 3 → 1" - Nodes with height change ??



- lowest node of imbalance ?? "5"
- right rotation of "5 → 3 → 1" - Nodes with height change ??

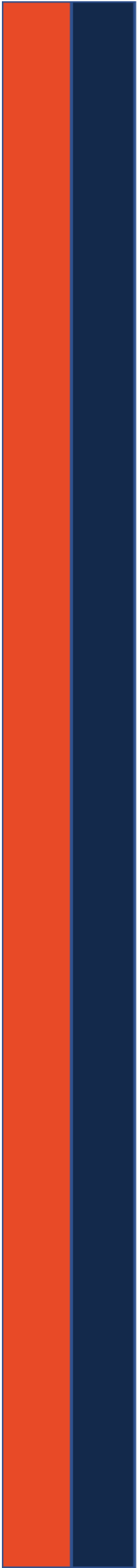


- lowest node of imbalance ?? "5"
- right rotation of "5 → 3 → 1" - Nodes with height change ??



- In an AVL Tree, we will store the height of every node !!!

- height adjustment in const time.



CS 225

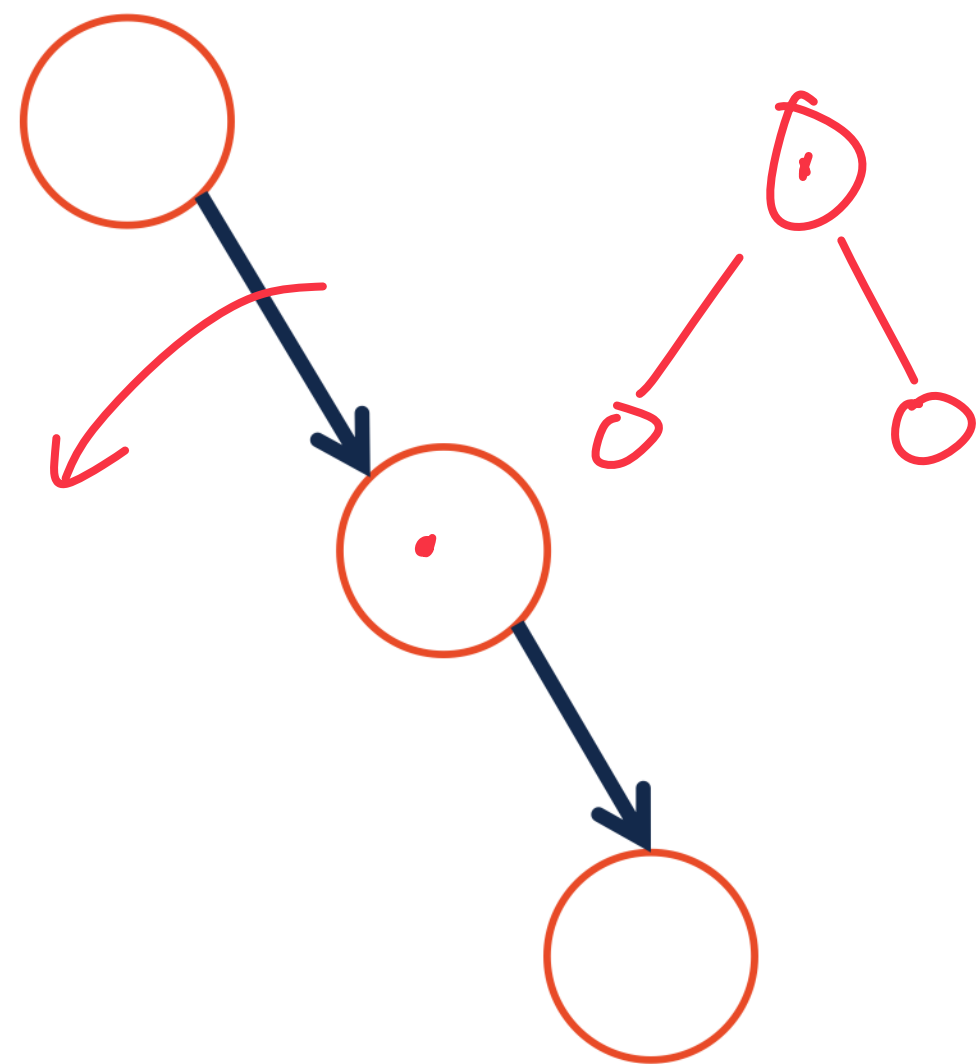
Data Structures

Volodymyr Kindratenko

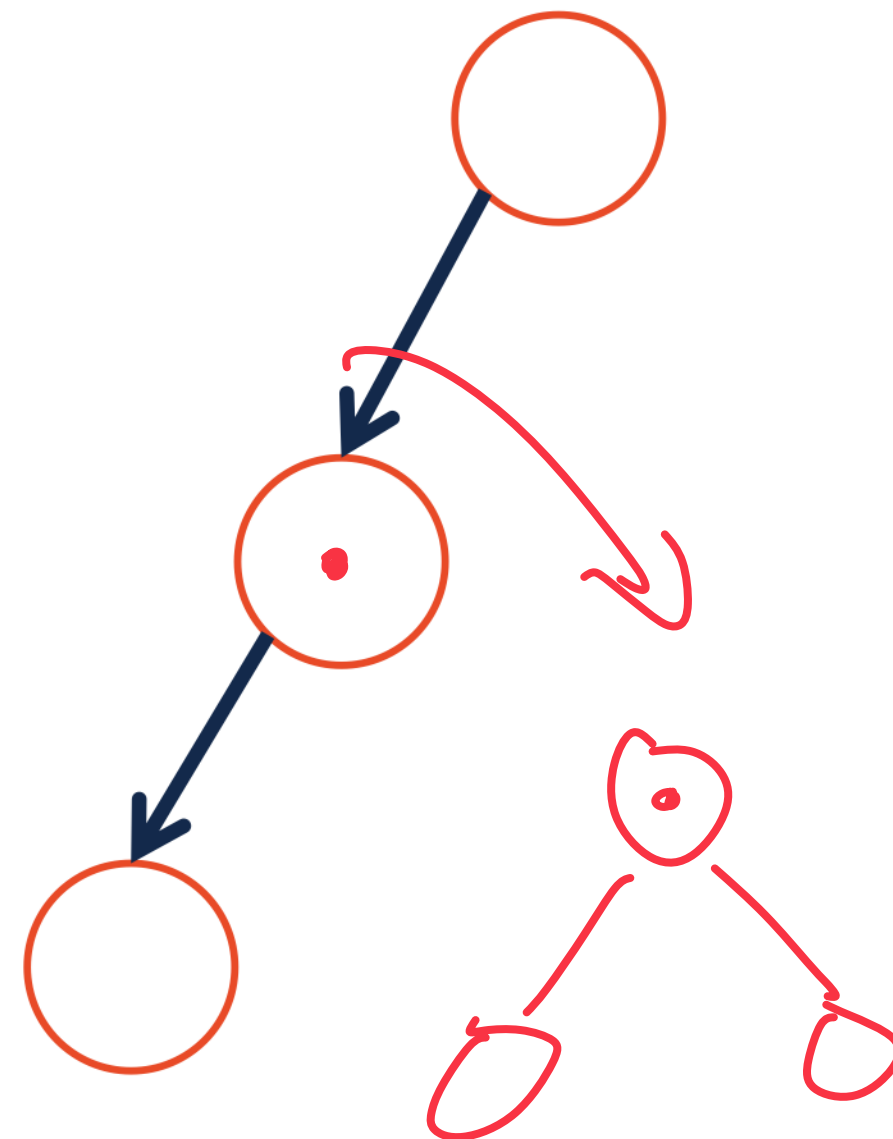
Slides courtesy of Wade Fagen-Ulmschneider

AVL Tree Rotations

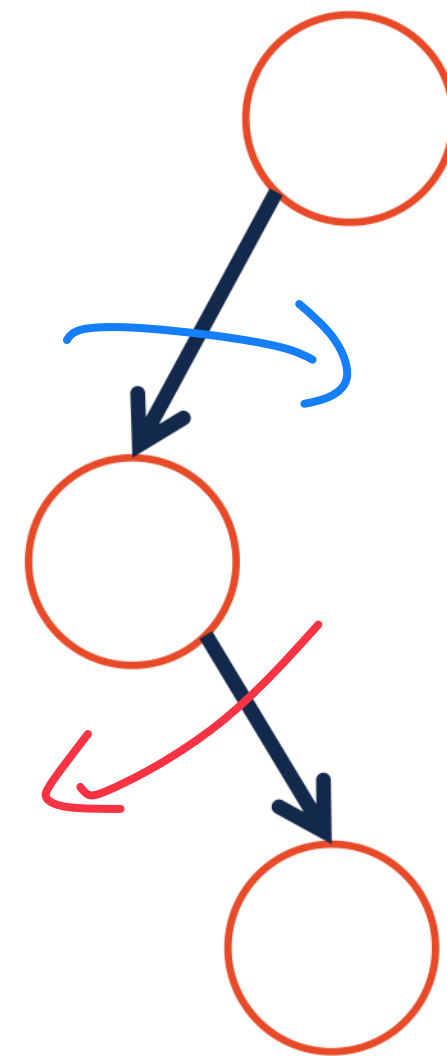
Four templates for rotations:



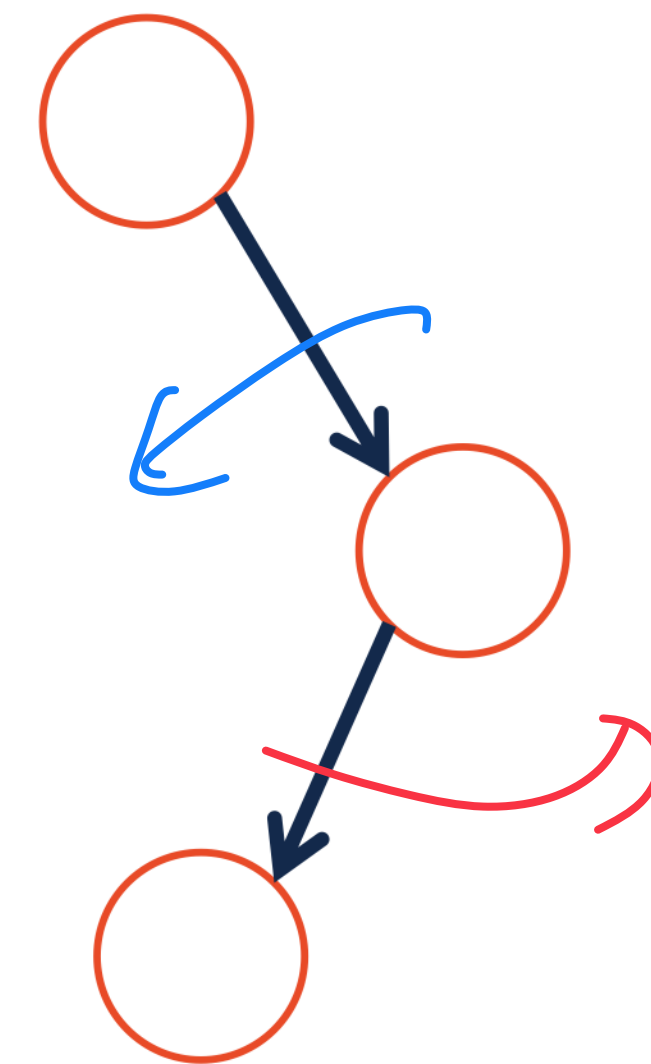
L



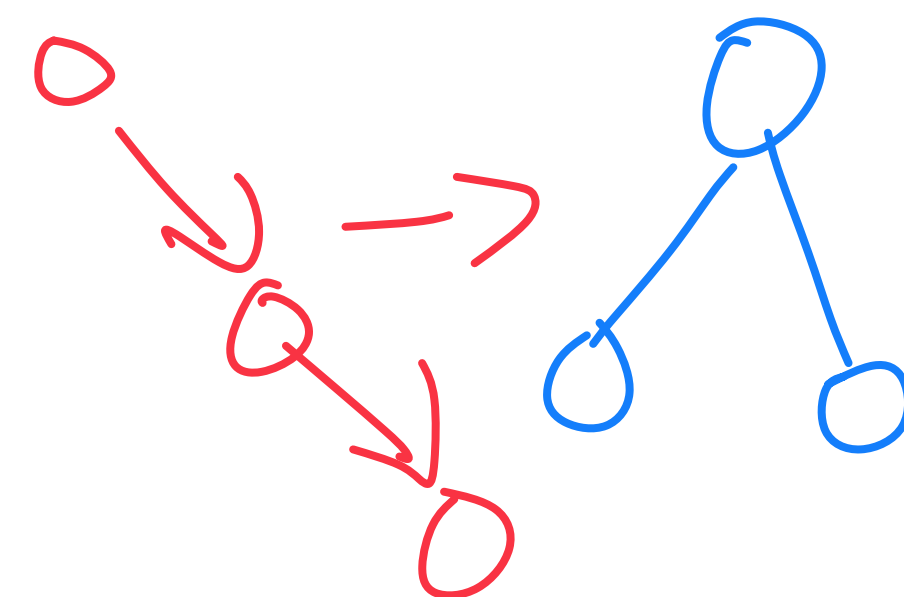
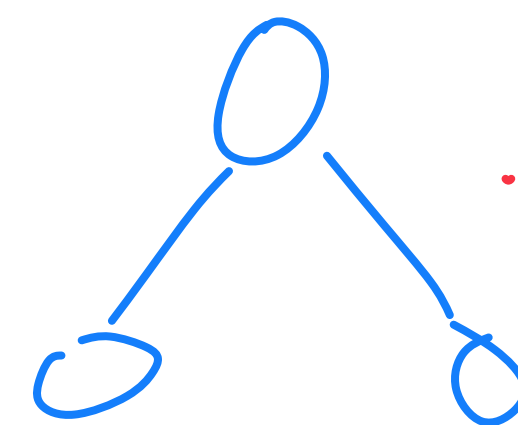
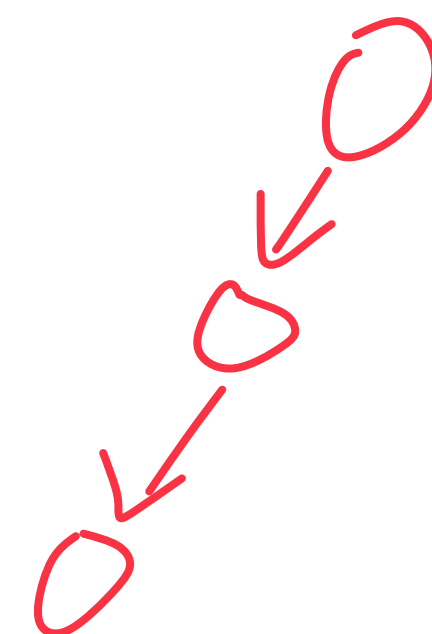
R



L R



R L

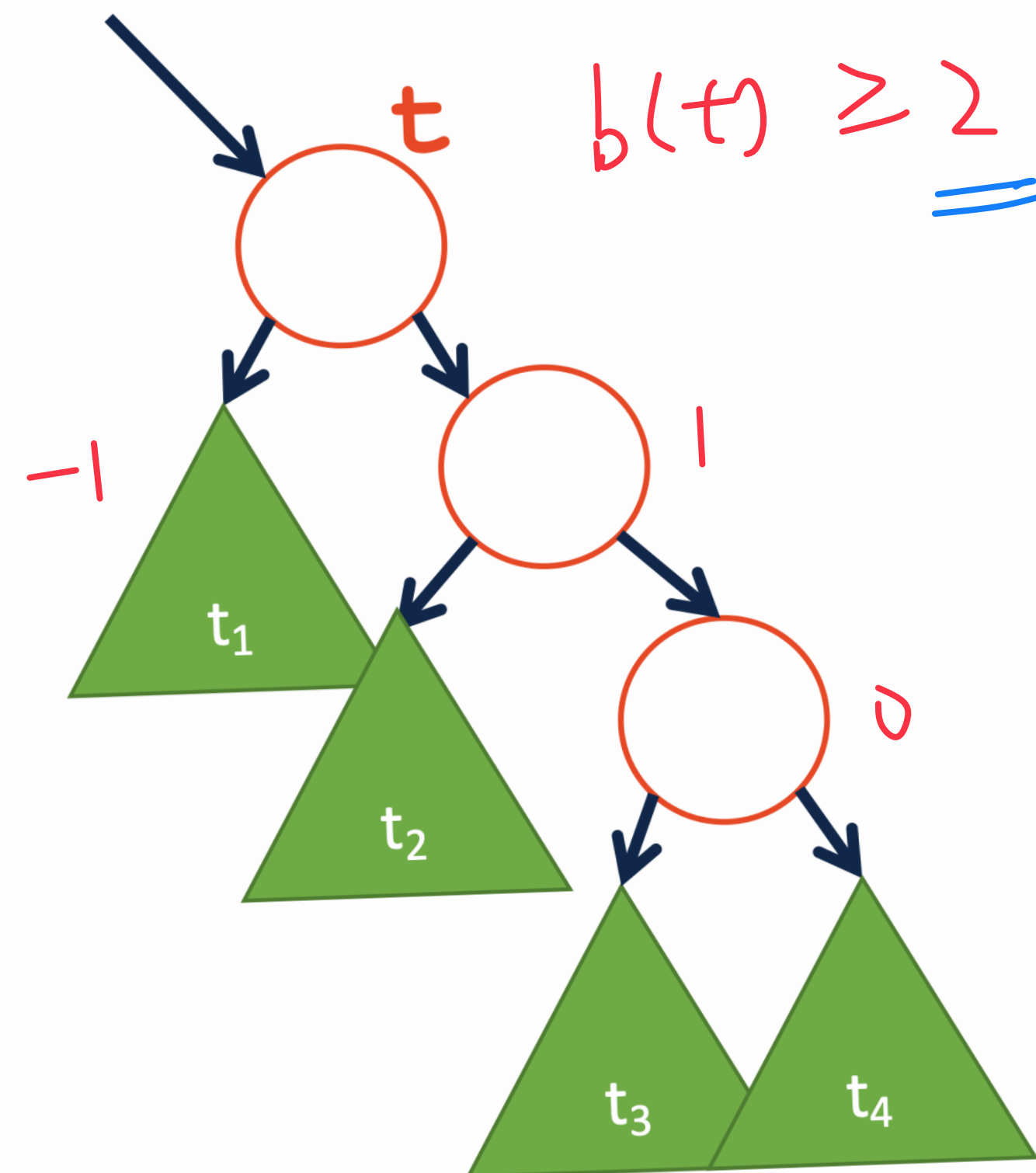


Height-Balanced Tree

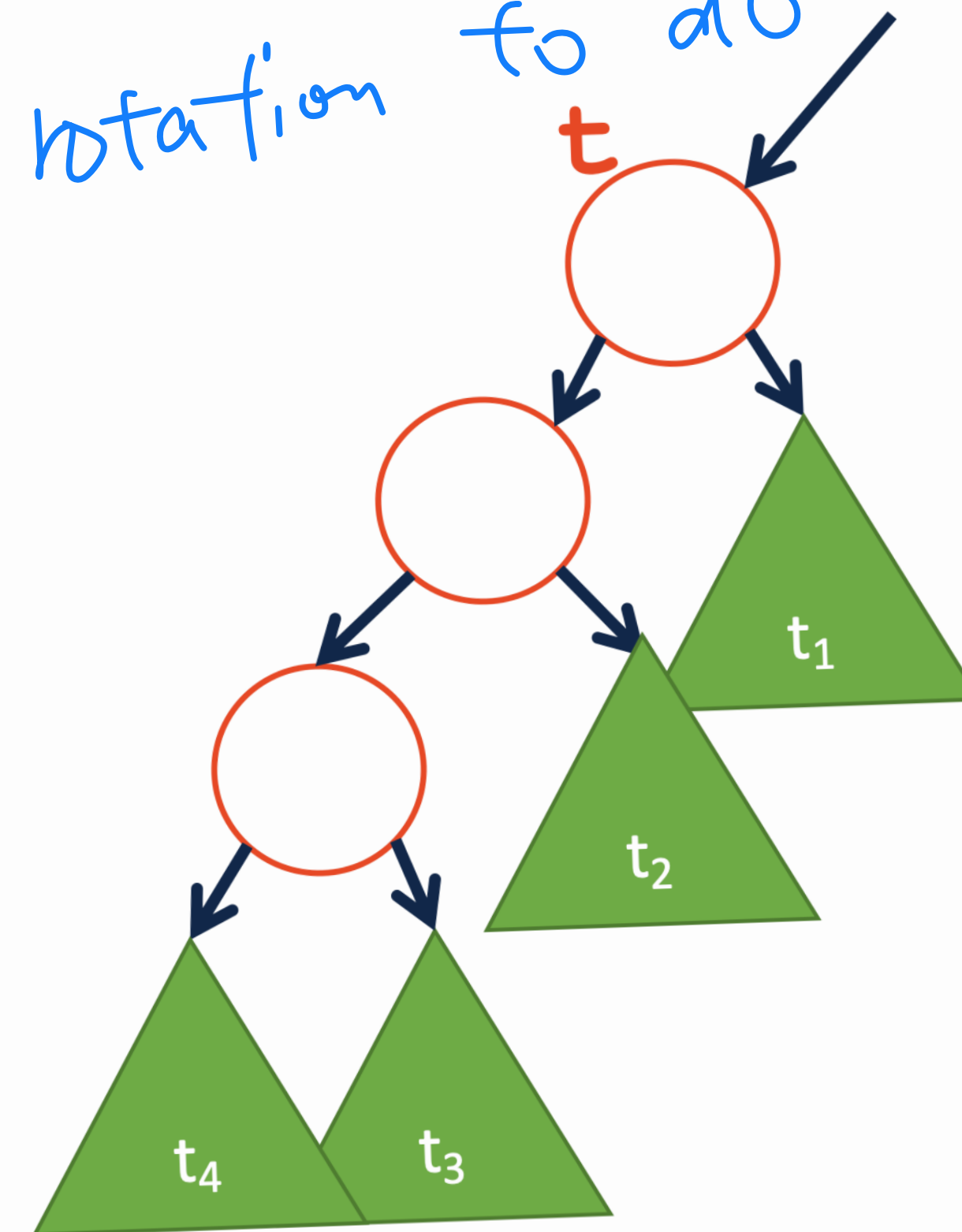
Height balance: $b = \text{height}(T_R) - \text{height}(T_L)$

The balance factor tells us

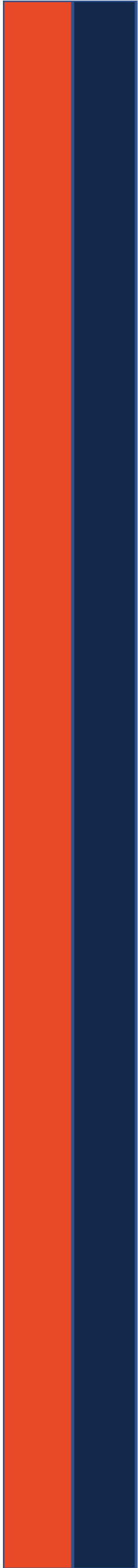
① tree is out of balance or not.
② what rotation to do



← left rotate



→ right rotate.

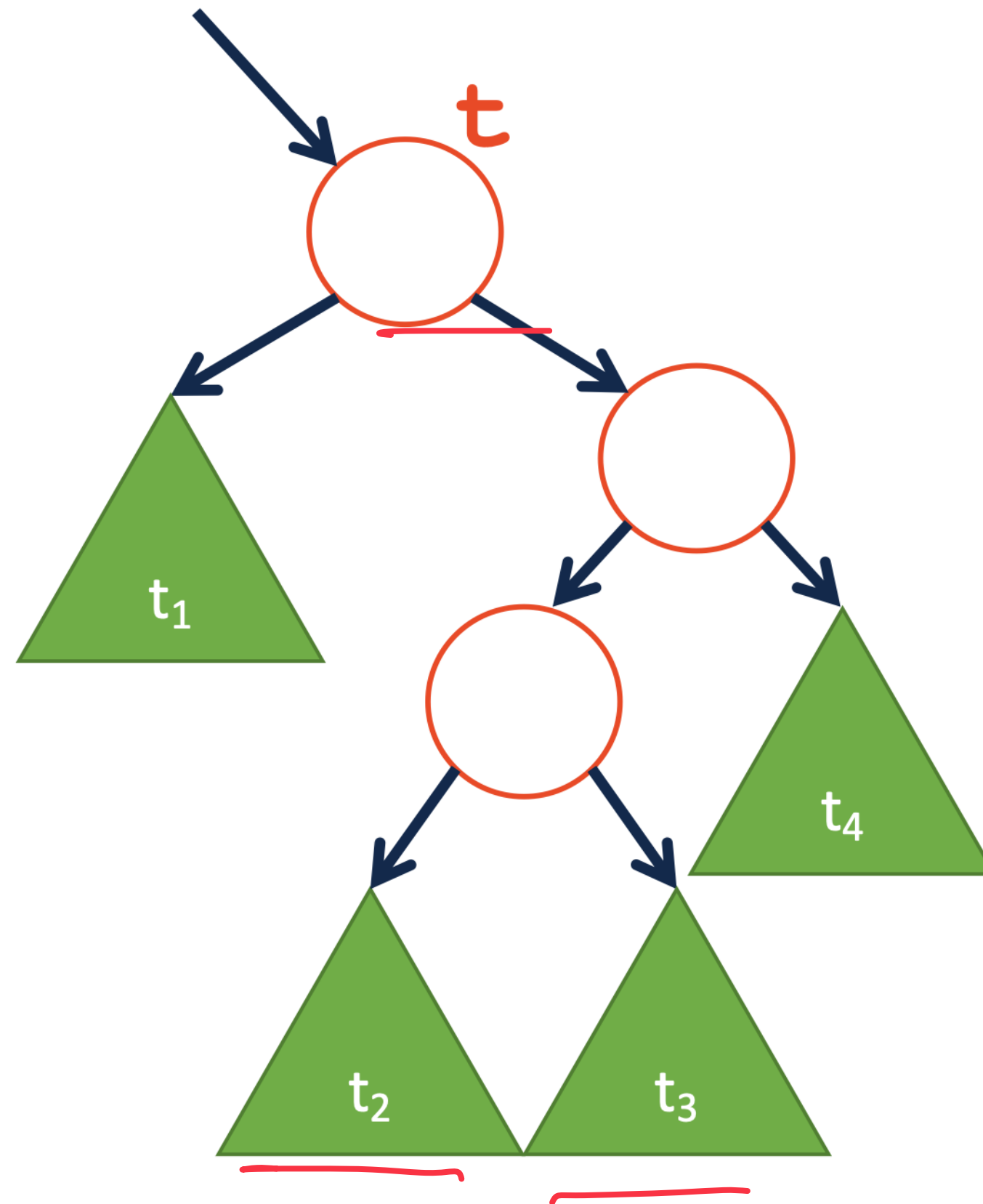


Theorem:

If an insertion occurred in subtrees t_3 or t_4 and an imbalance was detected at t , then a Left rotation about t restores the balance of the tree.

We gauge this by noting the balance factor of **t-right** is .

Finding the Rotation

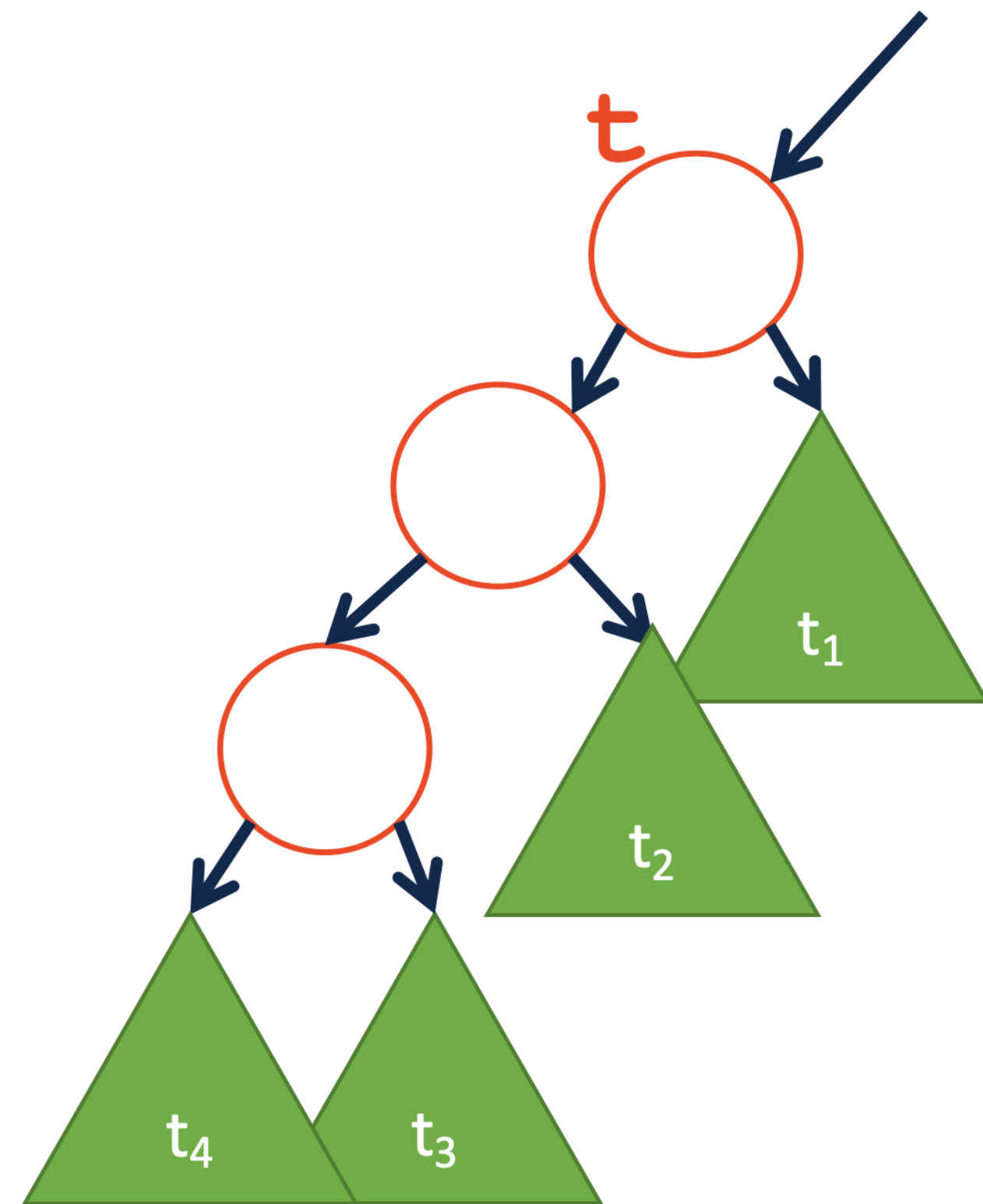


Theorem:

If an insertion occurred in subtrees t_2 or t_3 and an imbalance was detected at t , then a right-left rotation about t restores the balance of the tree. leg

We gauge this by noting the balance factor of $t \rightarrow \text{right}$ is -1.

Finding the Rotation



Theorem:

If an insertion occurred in subtrees t_3 or t_4 and an imbalance was detected at t , then a right rotation about t restores the balance of the tree.

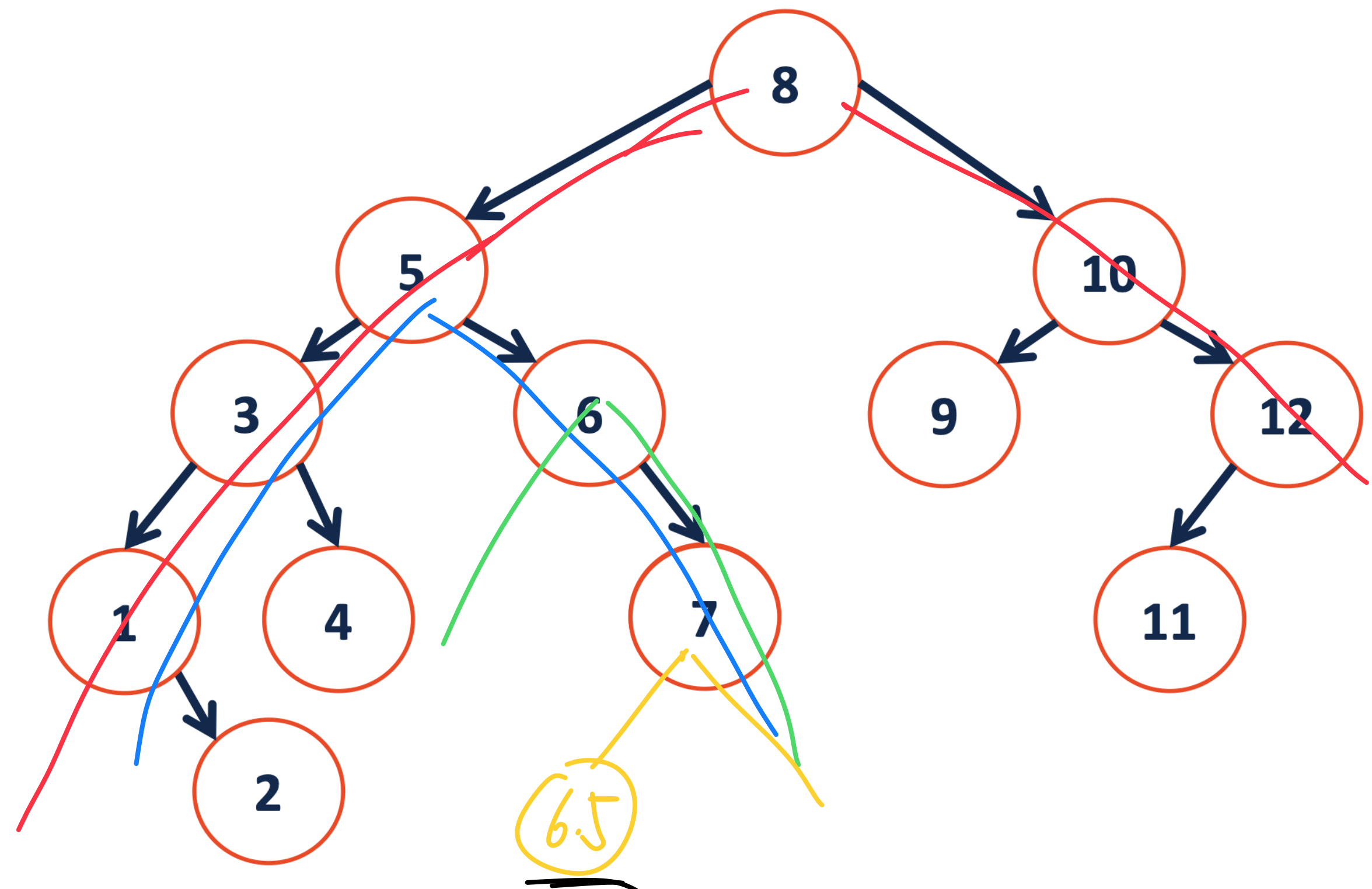
We gauge this by noting the balance factor of $t \rightarrow \text{left}$ is -1.

Insertion into an AVL Tree

Insert (pseudo code):

- 1: Insert at proper place ✓
- 2: Check for imbalance ✓
- 3: Rotate, if necessary ✓
- 4: Update height ✓

insert(6.5)
 $\log(n)$
✓ Recursively.



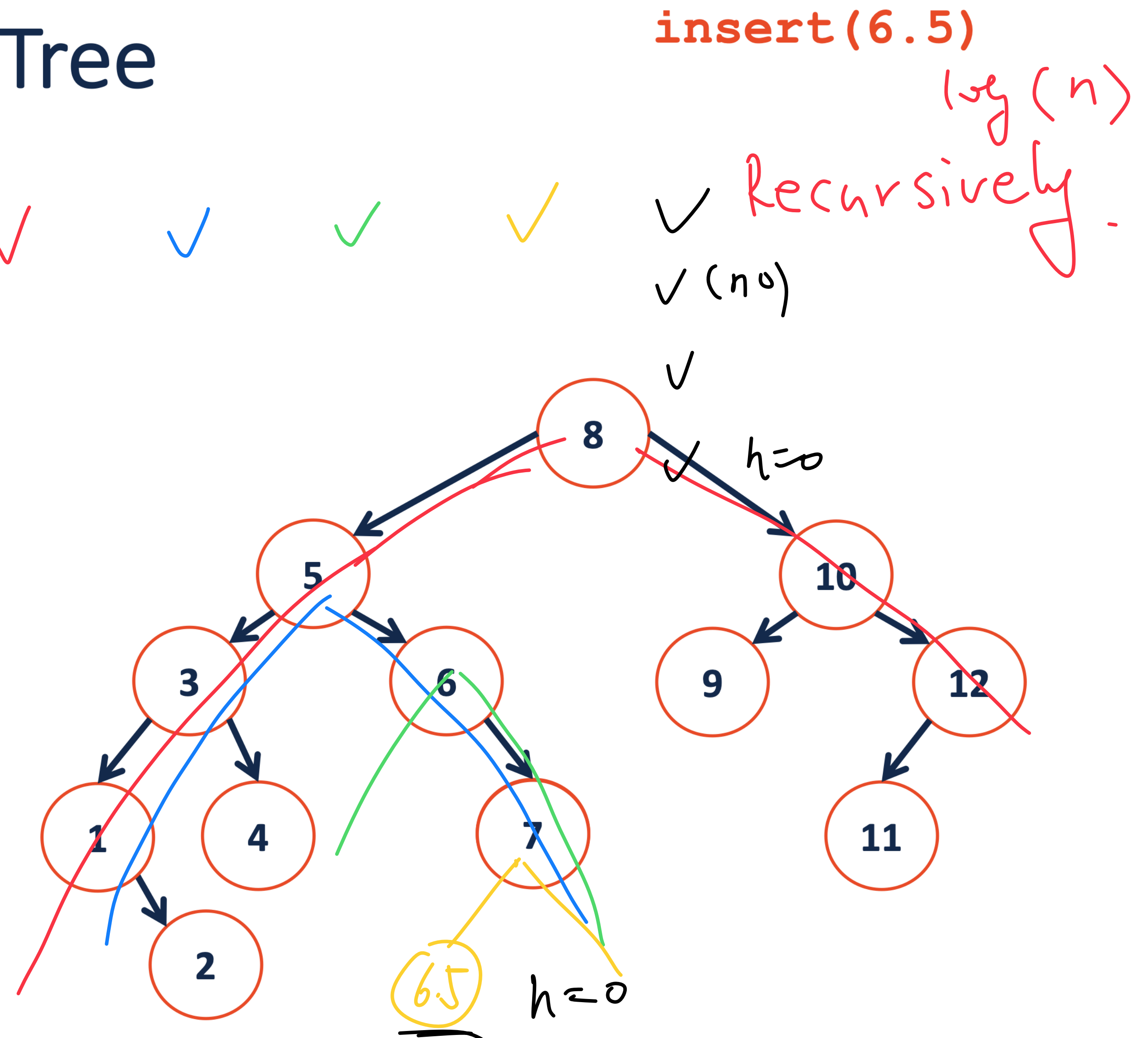
```
1 struct TreeNode {
2     T key;
3     unsigned height;
4     TreeNode *left;
5     TreeNode *right;
6 };
```


Insertion into an AVL Tree

Insert (pseudo code):

- 1: Insert at proper place ✓
- 2: Check for imbalance ✓
- 3: Rotate, if necessary ✓
- 4: Update height ✓

```
1 struct TreeNode {  
2     T key;  
3     unsigned height;  
4     TreeNode *left;  
5     TreeNode *right;  
6 };
```

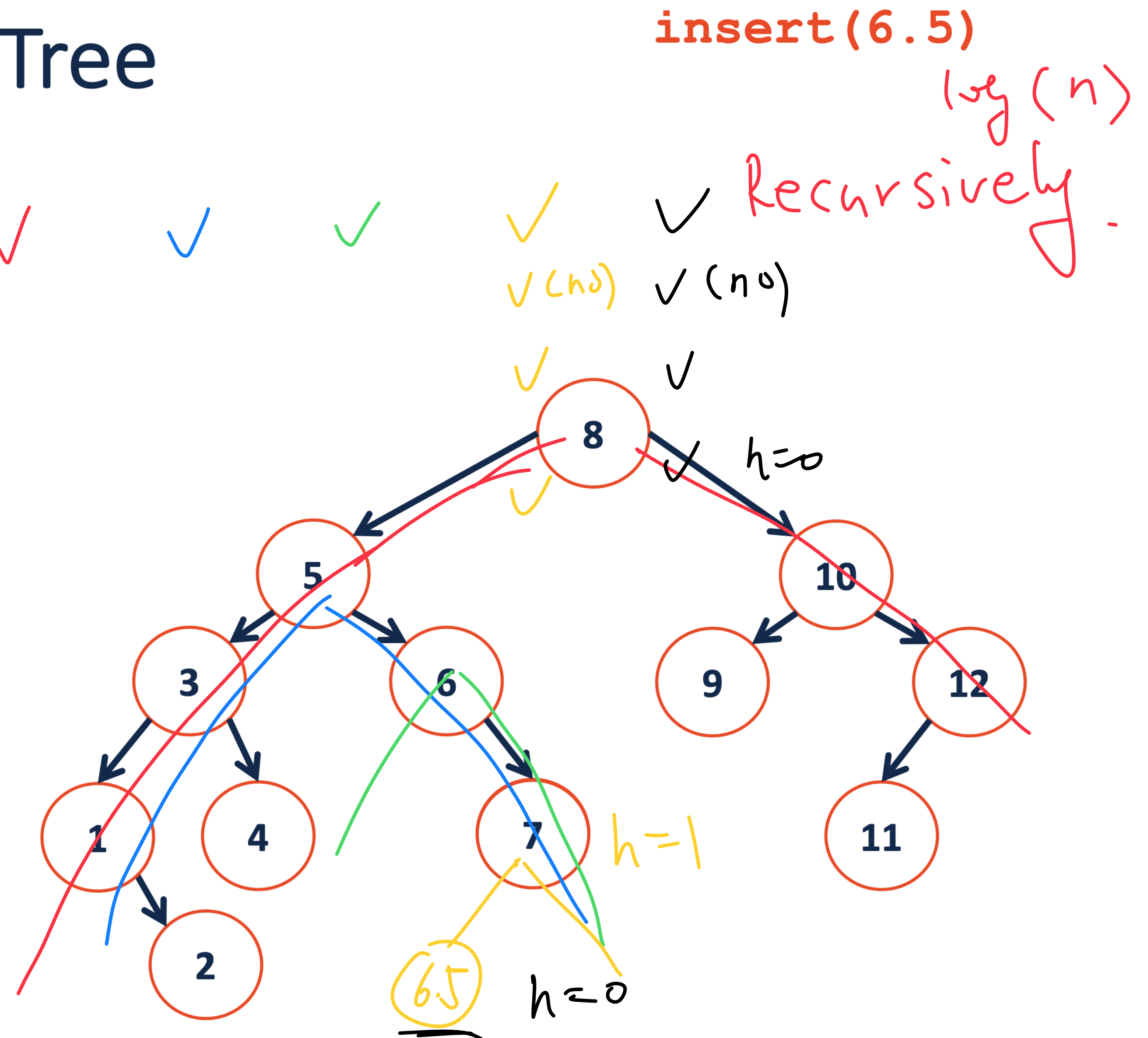


Insertion into an AVL Tree

Insert (pseudo code):

- 1: Insert at proper place
- 2: Check for imbalance
- 3: Rotate, if necessary
- 4: Update height

```
1 struct TreeNode {  
2     T key;  
3     unsigned height;  
4     TreeNode *left;  
5     TreeNode *right;  
6 };
```

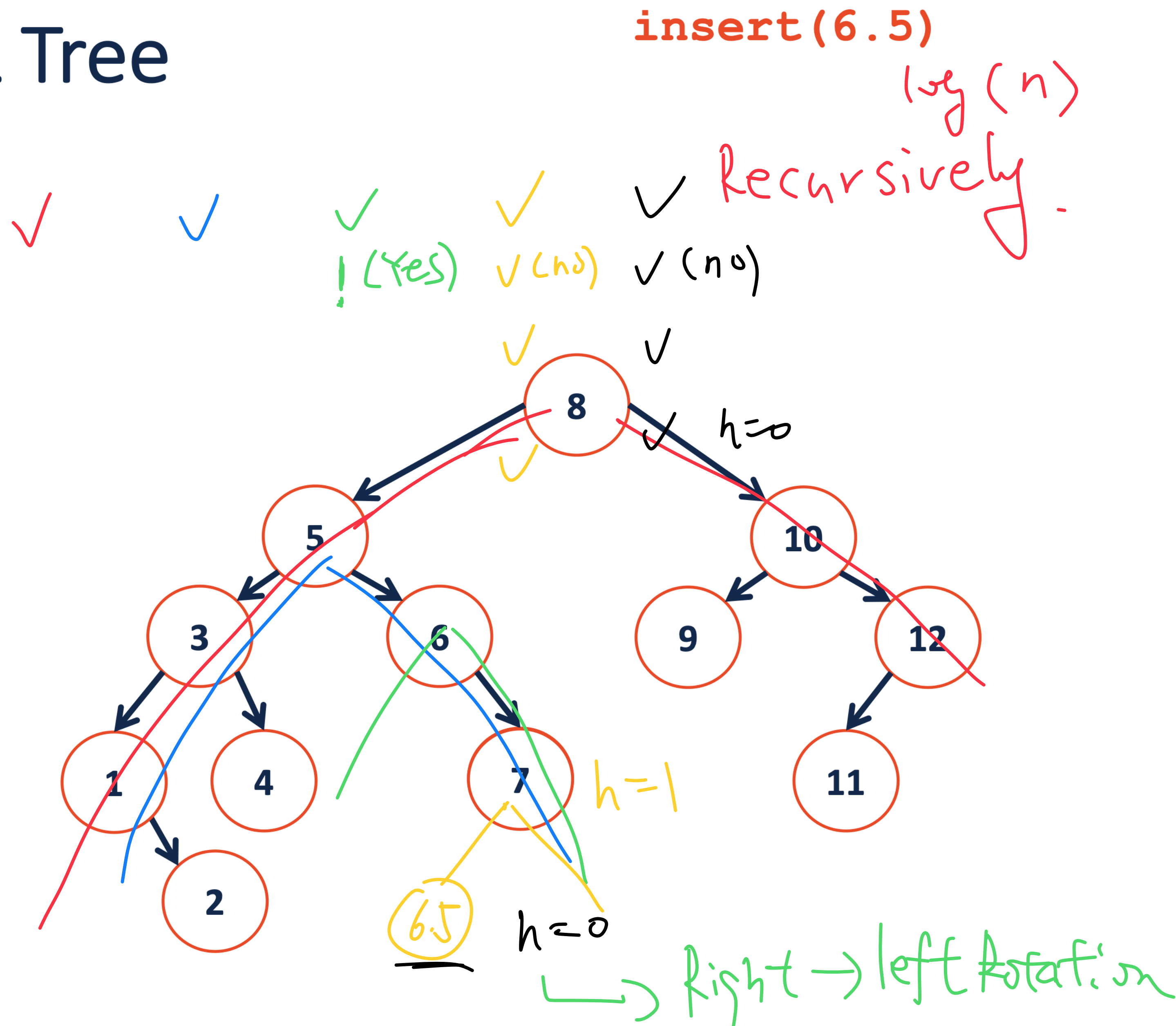


Insertion into an AVL Tree

Insert (pseudo code):

- 1: Insert at proper place
- 2: Check for imbalance
- 3: Rotate, if necessary
- 4: Update height

```
1 struct TreeNode {  
2     T key;  
3     unsigned height;  
4     TreeNode *left;  
5     TreeNode *right;  
6 };
```



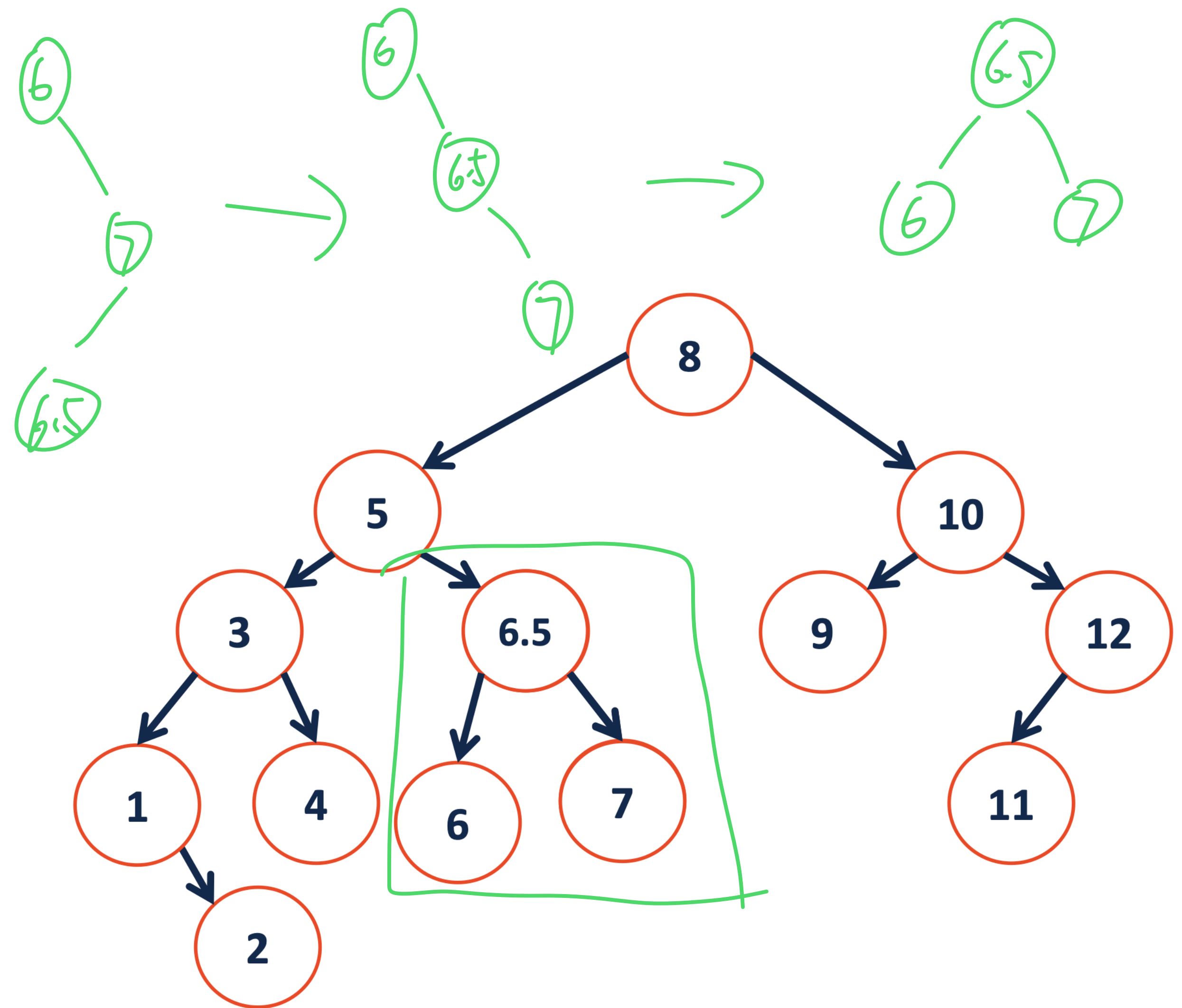
Insertion into an AVL Tree

Insert (pseudo code):

- 1: Insert at proper place
- 2: Check for imbalance
- 3: Rotate, if necessary
- 4: Update height

```
1 struct TreeNode {  
2     T key;  
3     unsigned height;  
4     TreeNode *left;  
5     TreeNode *right;  
6 };
```

insert(6.5)

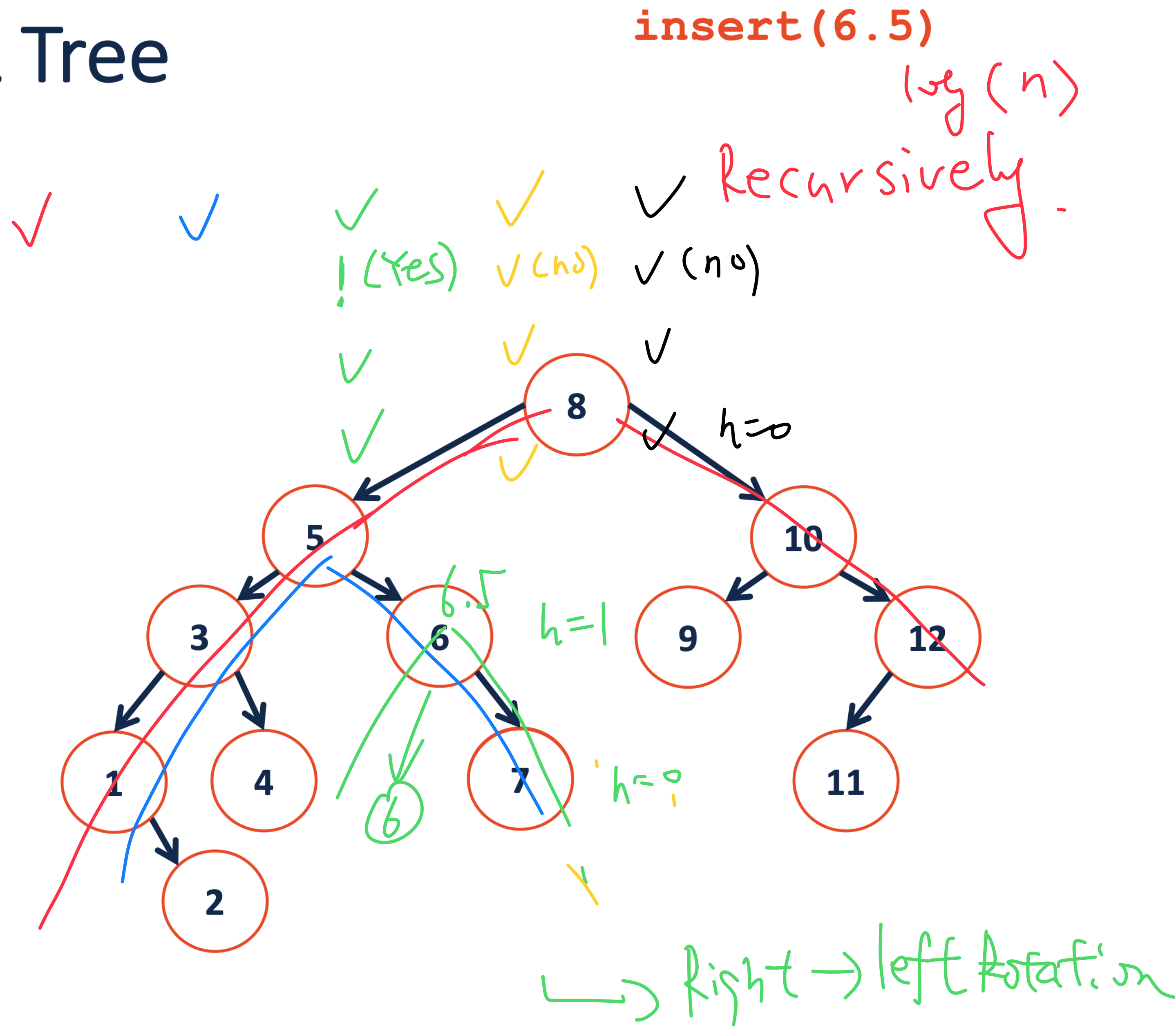


Insertion into an AVL Tree

Insert (pseudo code):

- 1: Insert at proper place
- 2: Check for imbalance
- 3: Rotate, if necessary
- 4: Update height

```
1 struct TreeNode {  
2     T key;  
3     unsigned height;  
4     TreeNode *left;  
5     TreeNode *right;  
6 };
```

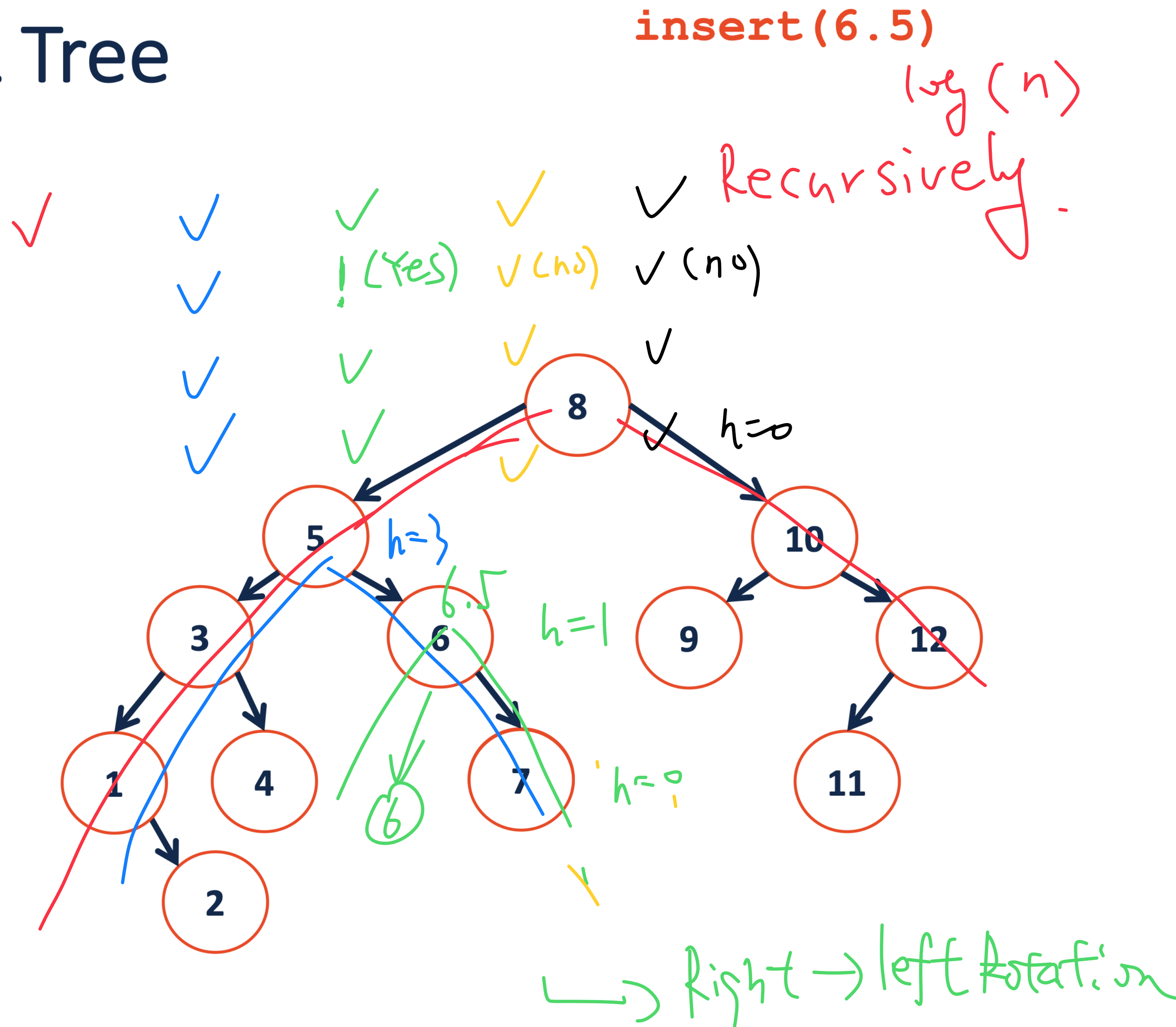


Insertion into an AVL Tree

Insert (pseudo code):

- 1: Insert at proper place
- 2: Check for imbalance
- 3: Rotate, if necessary
- 4: Update height

```
1 struct TreeNode {  
2     T key;  
3     unsigned height;  
4     TreeNode *left;  
5     TreeNode *right;  
6 };
```

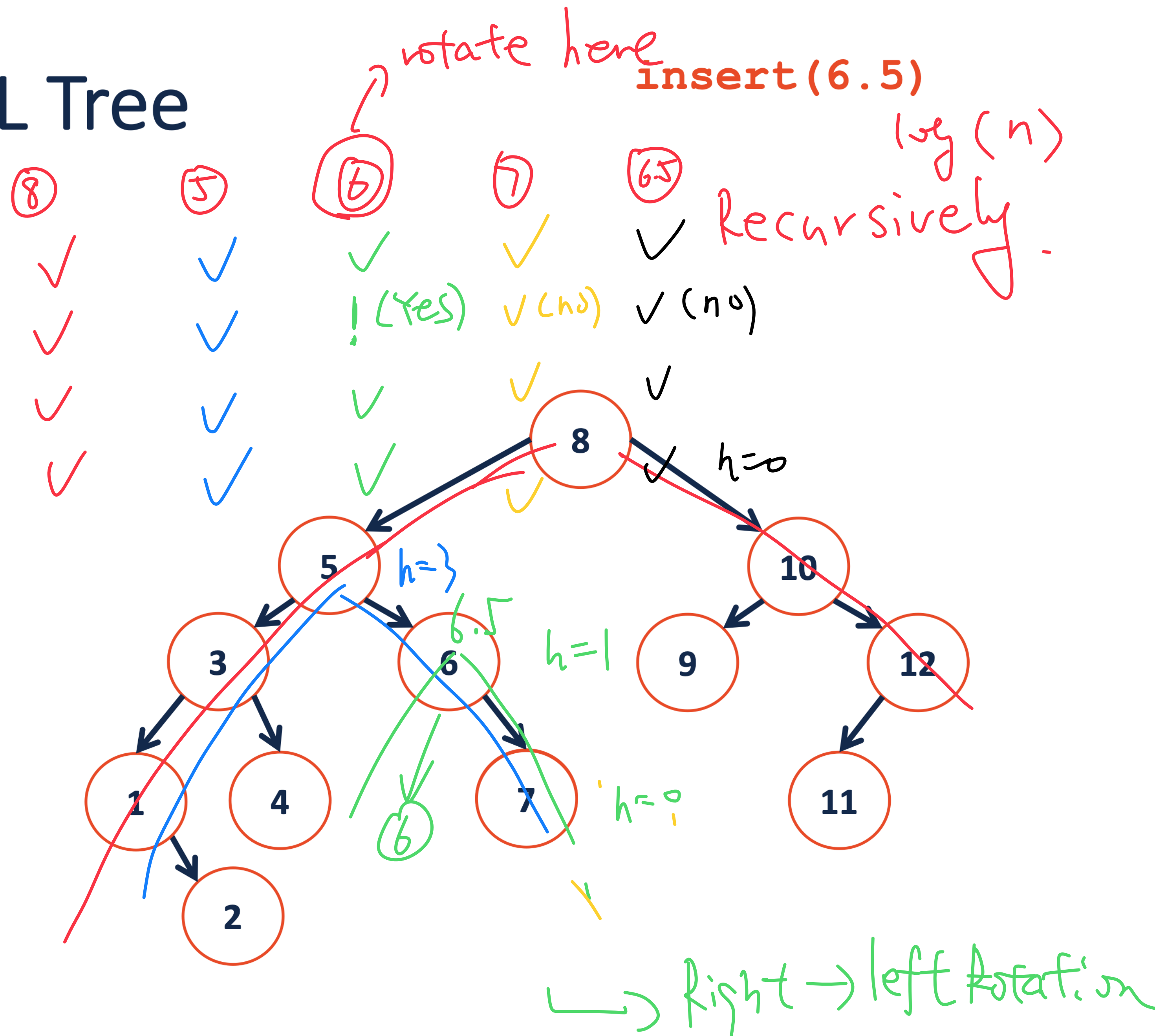


Insertion into an AVL Tree

Insert (pseudo code):

- 1: Insert at proper place
- 2: Check for imbalance
- 3: Rotate, if necessary
- 4: Update height

```
1 struct TreeNode {  
2     T key;  
3     unsigned height;  
4     TreeNode *left;  
5     TreeNode *right;  
6 };
```



```

1  template <class T> void AVLTree<T>::insert_(const T & x, treeNode<T> * & t) {
2      if( t == NULL ) {
3          t = new TreeNode<T>( x, 0, NULL, NULL);
4      }
5
6      else if( x < t->key ) {
7          insert_( x, t->left );
8          int balance = height(t->right) - height(t->left);
9          int leftBalance = height(t->left->right) - height(t->left->left);
10         if ( balance == -2 ) {
11             if ( leftBalance == -1 ) { rotate_____ ( t ); }
12             else { rotate_____ ( t ); }
13         }
14     }
15
16     else if( x > t->key ) {
17         insert_( x, t->right );
18         int balance = height(t->right) - height(t->left);
19         int rightBalance = height(t->right->right) - height(t->right->left);
20         if( balance == 2 ) {
21             if( rightBalance == 1 ) { rotate_____ ( t ); }
22             else { rotate_____ ( t ); }
23         }
24     }
25
26     t->height = 1 + max(height(t->left), height(t->right));
27 }

```

key

current root


```

1  template <class T> void AVLTree<T>::insert_(const T & x, treeNode<T> * & t) {
2      if( t == NULL ) {
3          t = new TreeNode<T>( x, 0, NULL, NULL);
4      }
5
6      else if( x < t->key ) {
7          → insert_( x, t->left );
8          int balance = height(t->right) - height(t->left);
9          → { int leftBalance = height(t->left->right) - height(t->left->left);
10             if ( balance == -2 ) {
11                 if ( leftBalance == -1 ) { rotate_____ ( t ); }
12                 else { rotate_____ ( t ); }
13             }
14         }
15
16         else if( x > t->key ) {
17             insert_( x, t->right );
18             int balance = height(t->right) - height(t->left);
19             int rightBalance = height(t->right->right) - height(t->right->left);
20             if( balance == 2 ) {
21                 if( rightBalance == 1 ) { rotate_____ ( t ); }
22                 else { rotate_____ ( t ); }
23             }
24         }
25
26         t->height = 1 + max(height(t->left), height(t->right));
27     }

```

key
 current root
 Base case
 Insertion
 check imbalance

```

1  template <class T> void AVLTree<T>::insert_(const T & x, treeNode<T> * & t) {
2      if( t == NULL ) {
3          t = new TreeNode<T>( x, 0, NULL, NULL);
4      } Base case
5
6      else if( x < t->key ) {
7          → insert_( x, t->left ); Insertion
8          int balance = height(t->right) - height(t->left);
9          → { int leftBalance = height(t->left->right) - height(t->left->left); check imbalance
10             if ( balance == -2 ) {
11                 if ( leftBalance == -1 ) { rotate Right ( t ); }
12                 else { rotate Left Right ( t ); }
13             }
14         }
15
16         else if( x > t->key ) {
17             insert_( x, t->right );
18             int balance = height(t->right) - height(t->left);
19             int rightBalance = height(t->right->right) - height(t->right->left);
20             if( balance == 2 ) {
21                 if( rightBalance == 1 ) { rotate _____ ( t ); }
22                 else { rotate _____ ( t ); }
23             }
24         }
25
26         t->height = 1 + max(height(t->left), height(t->right));
27     }

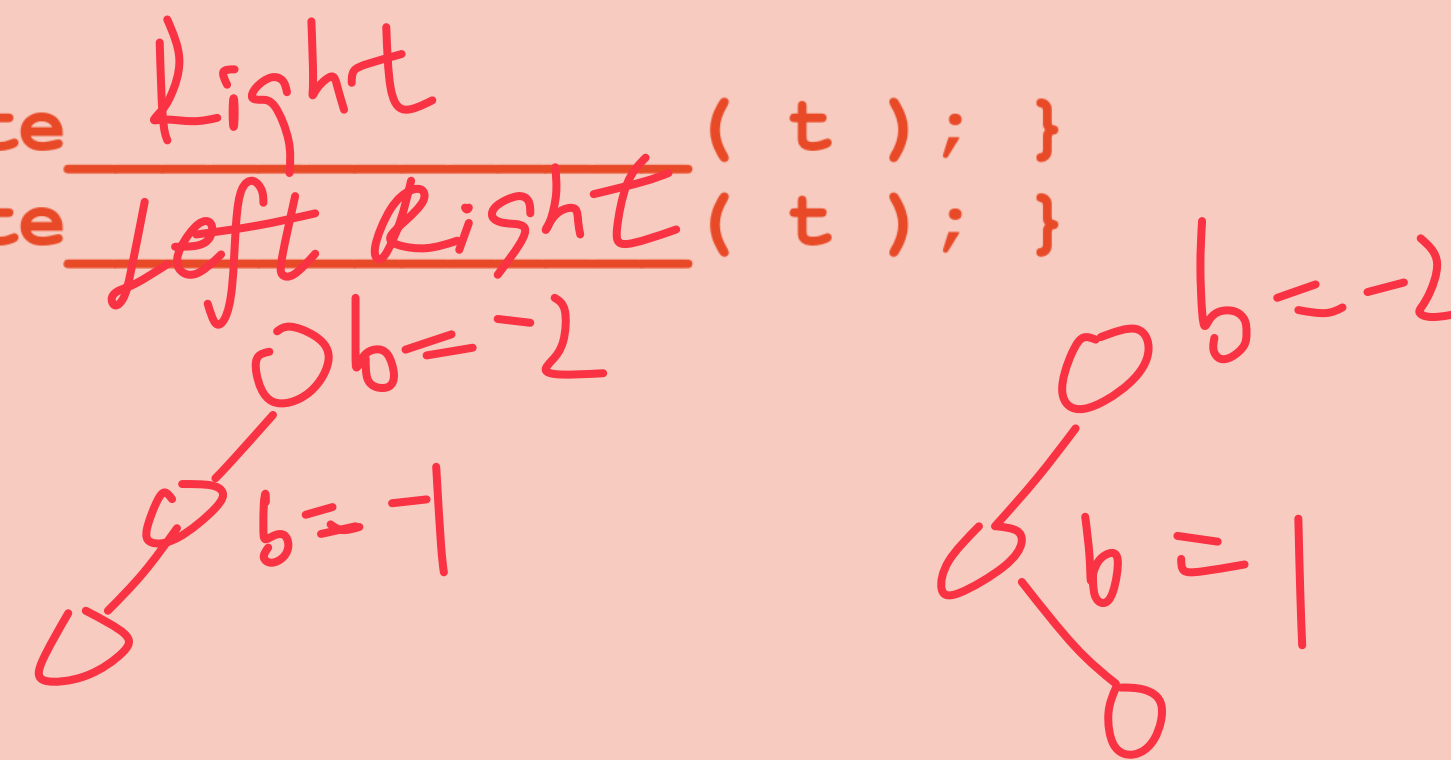
```

key

current root

check imbalance

rotation




```

1  template <class T> void AVLTree<T>::insert_(const T & x, treeNode<T> * & t) {
2      if( t == NULL ) {
3          t = new TreeNode<T>( x, 0, NULL, NULL);
4      }
5
6      else if( x < t->key ) {
7          insert_( x, t->left );
8          int balance = height(t->right) - height(t->left);
9          int leftBalance = height(t->left->right) - height(t->left->left);
10         if ( balance == -2 ) {
11             if ( leftBalance == -1 ) { rotate ( t ); }
12             else { rotate ( t ); }
13         }
14
15         else if( x > t->key ) {
16             insert_( x, t->right );
17             int balance = height(t->right) - height(t->left);
18             int rightBalance = height(t->right->right) - height(t->right->left);
19             if( balance == 2 ) {
20                 if( rightBalance == 1 ) { rotate ( t ); }
21                 else { rotate ( t ); }
22             }
23         }
24     }
25
26     t->height = 1 + max(height(t->left), height(t->right));
27 }

```

key

current root

Base case

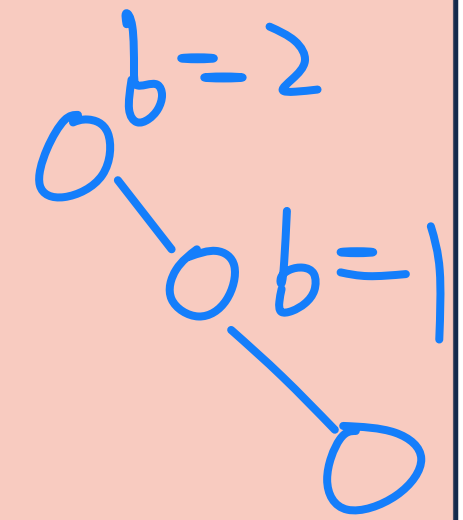
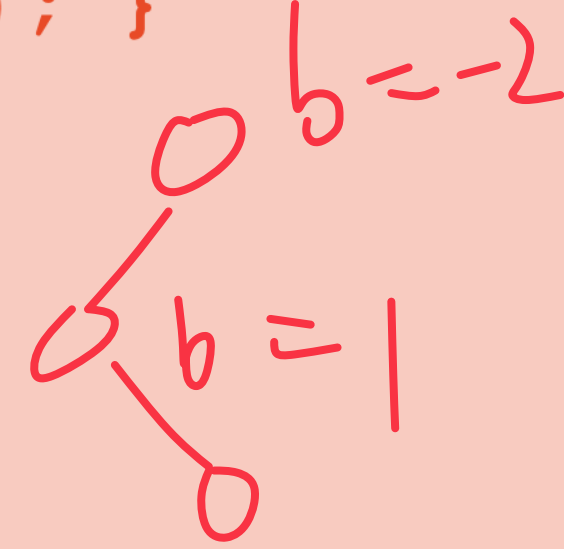
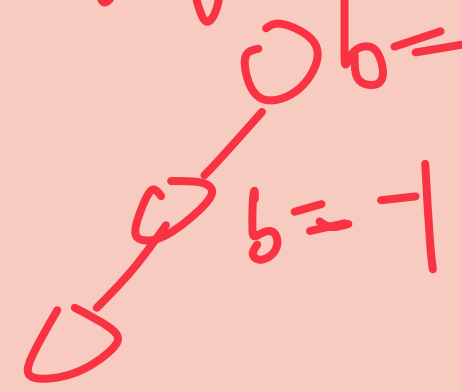
Insertion

check imbalance

Right

Left Right

rotation



Left

Right Left



update height

```

1  template <class T> void AVLTree<T>::insert_(const T & x, treeNode<T> * & t) {
2      if( t == NULL ) {
3          t = new TreeNode<T>( x, 0, NULL, NULL);
4      }
5
6      else if( x < t->key ) {
7          insert_( x, t->left );
8          int balance = height(t->right) - height(t->left);
9          int leftBalance = height(t->left->right) - height(t->left->left);
10         if ( balance == -2 ) {
11             if ( leftBalance == -1 ) { rotate
12             else { rotate
13         }
14     }
15
16     else if( x > t->key ) {
17         insert_( x, t->right );
18         int balance = height(t->right) - height(t->left);
19         int rightBalance = height(t->right->right) - height(t->right->left);
20         if( balance == 2 ) {
21             if( rightBalance == 1 ) { rotate
22             else { rotate
23         }
24     }
25
26     t->height = 1 + max(height(t->left), height(t->right));
27 }

```

key

current not
might have
null

Just use $\rightarrow$ height ??

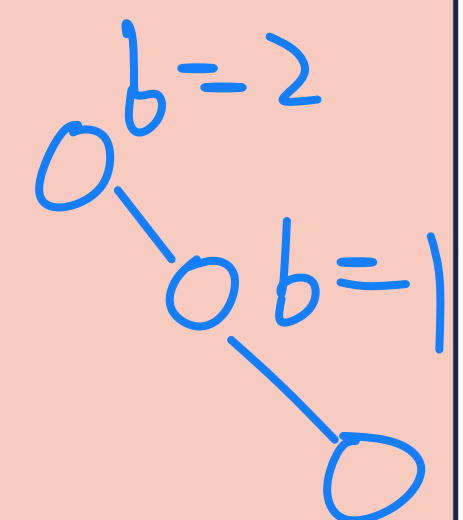
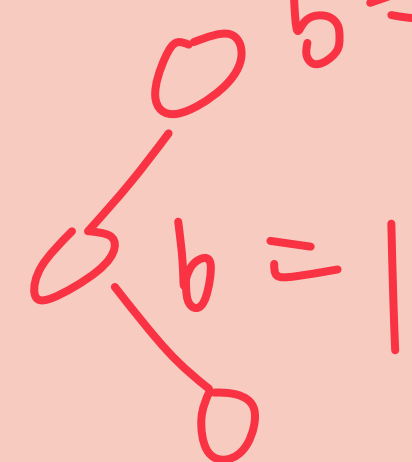
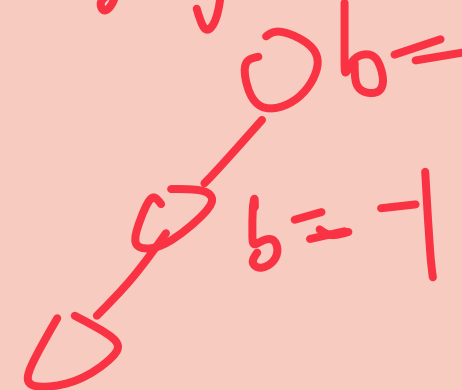
Insertion

check imbalance

Right

Avoid seg fault

rotation



Left

Right Left



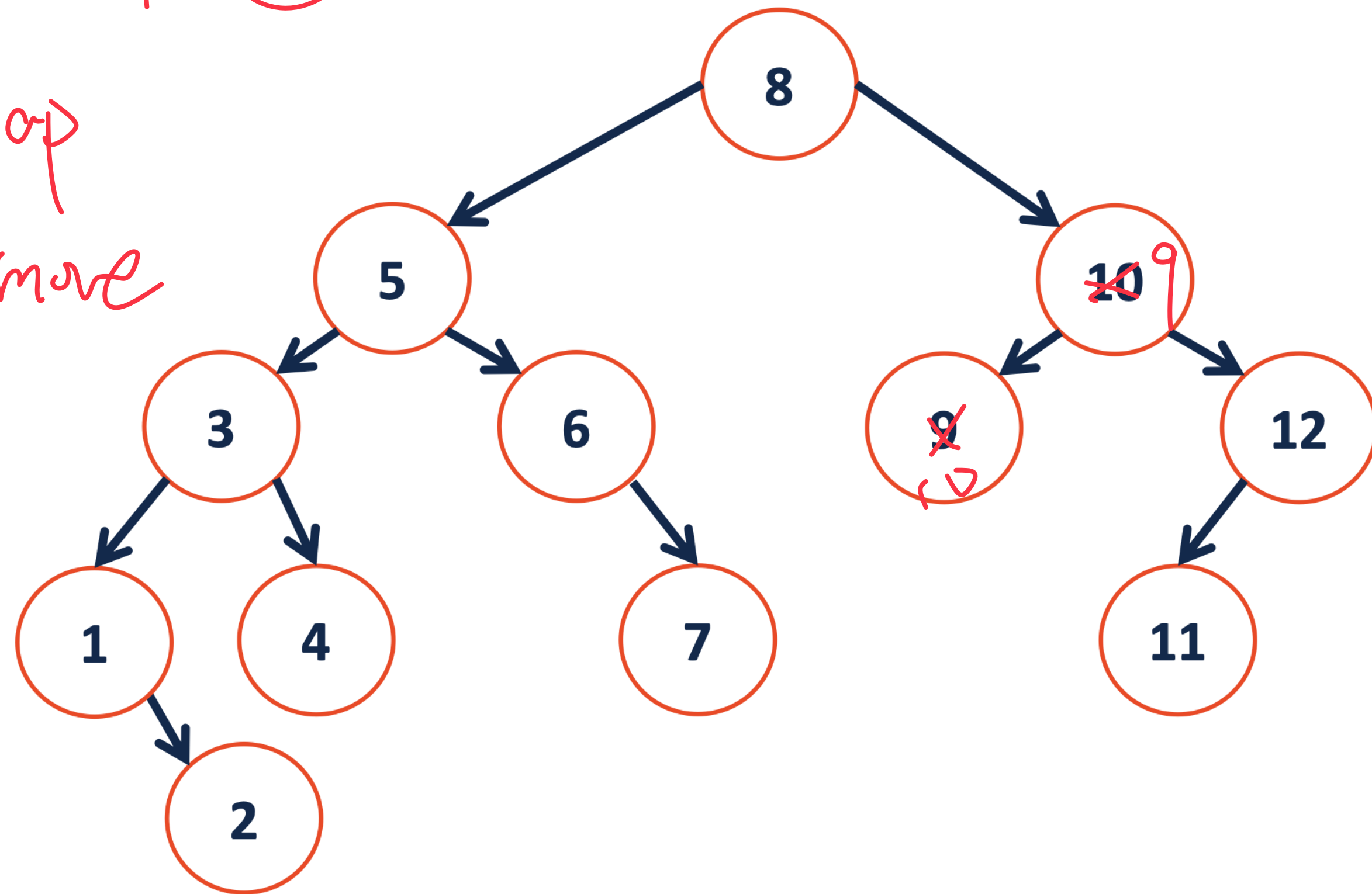
update height

Remove 10

① Find IOP: 9

② Swap

③ Remove

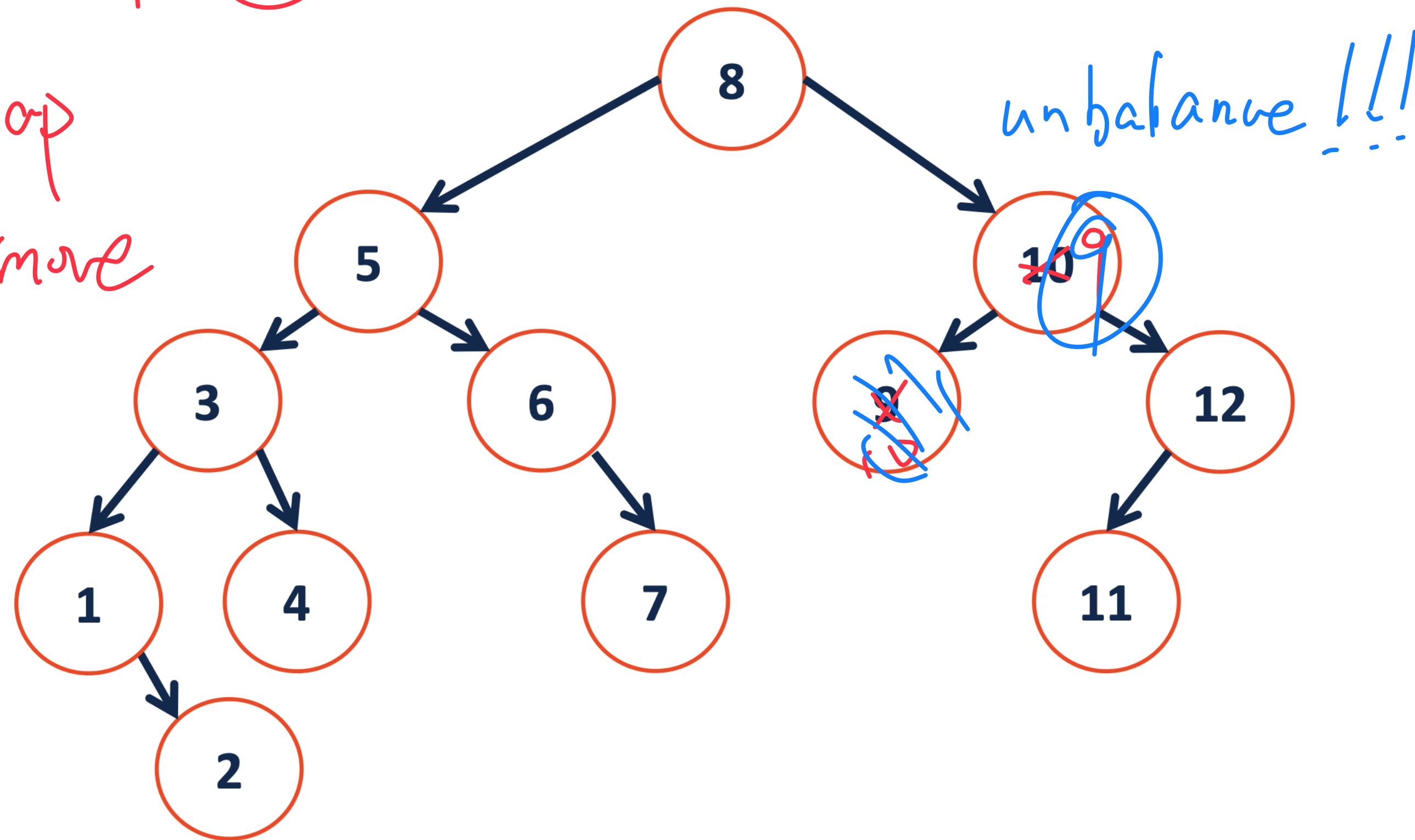


Remove 10

① Find IOP: 9

② Swap

③ Remove



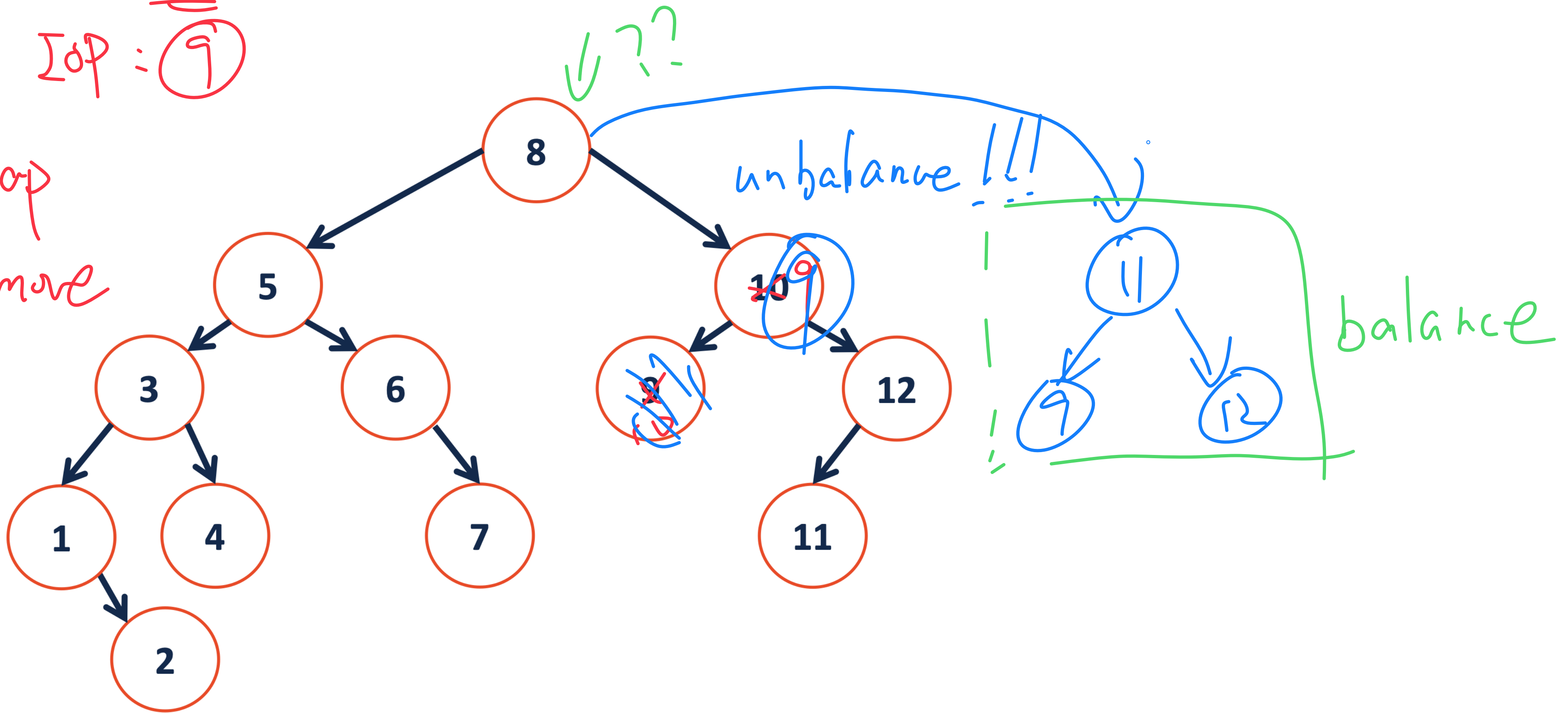
Rotate

Remove 10

① Find IOP: 9

② Swap

② to move

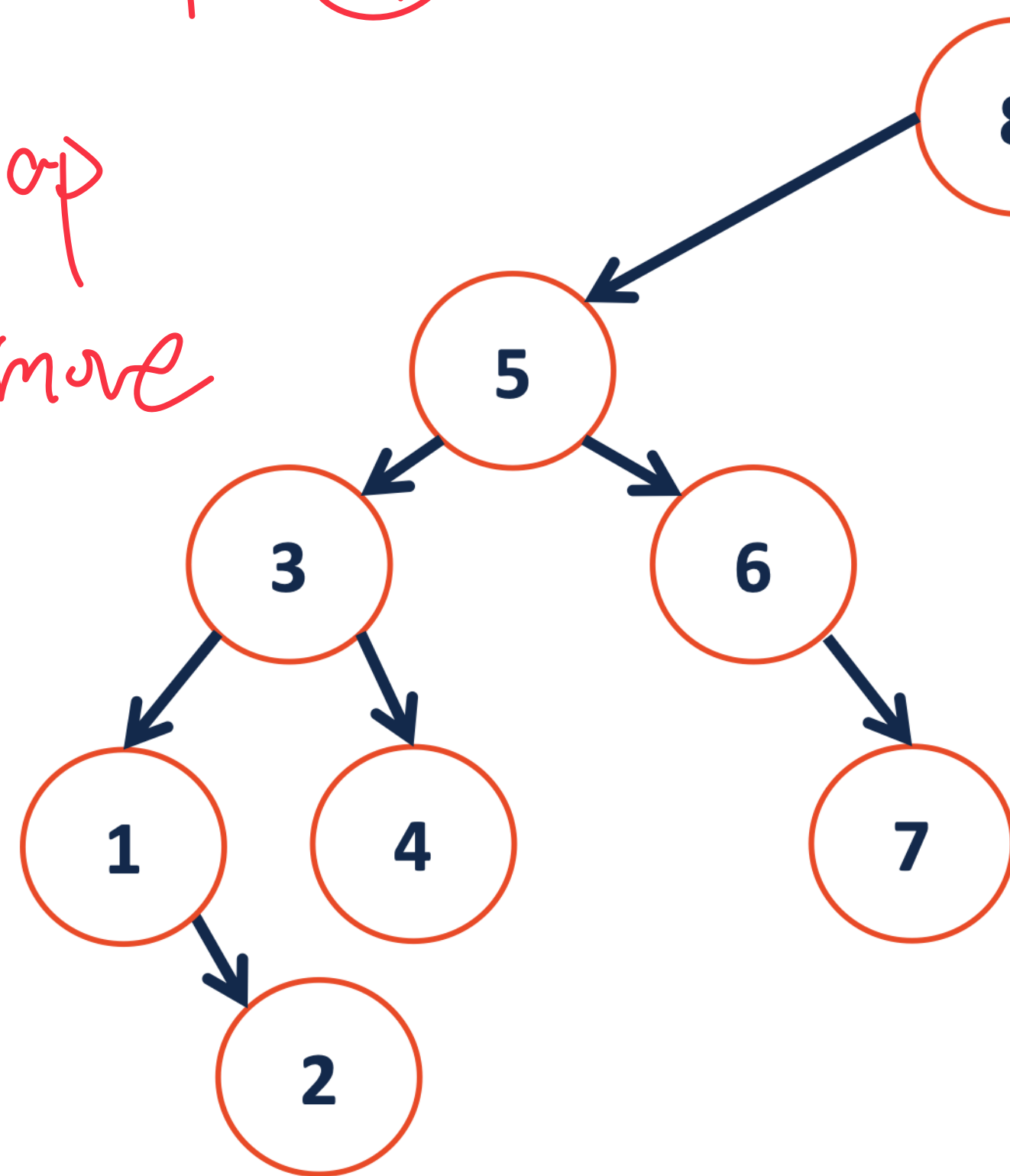


Rotate

Remove 10
① Find IOP: 9

② Swap

③ Remove



imbalanced, rotate.

unbalanced!!!

balance

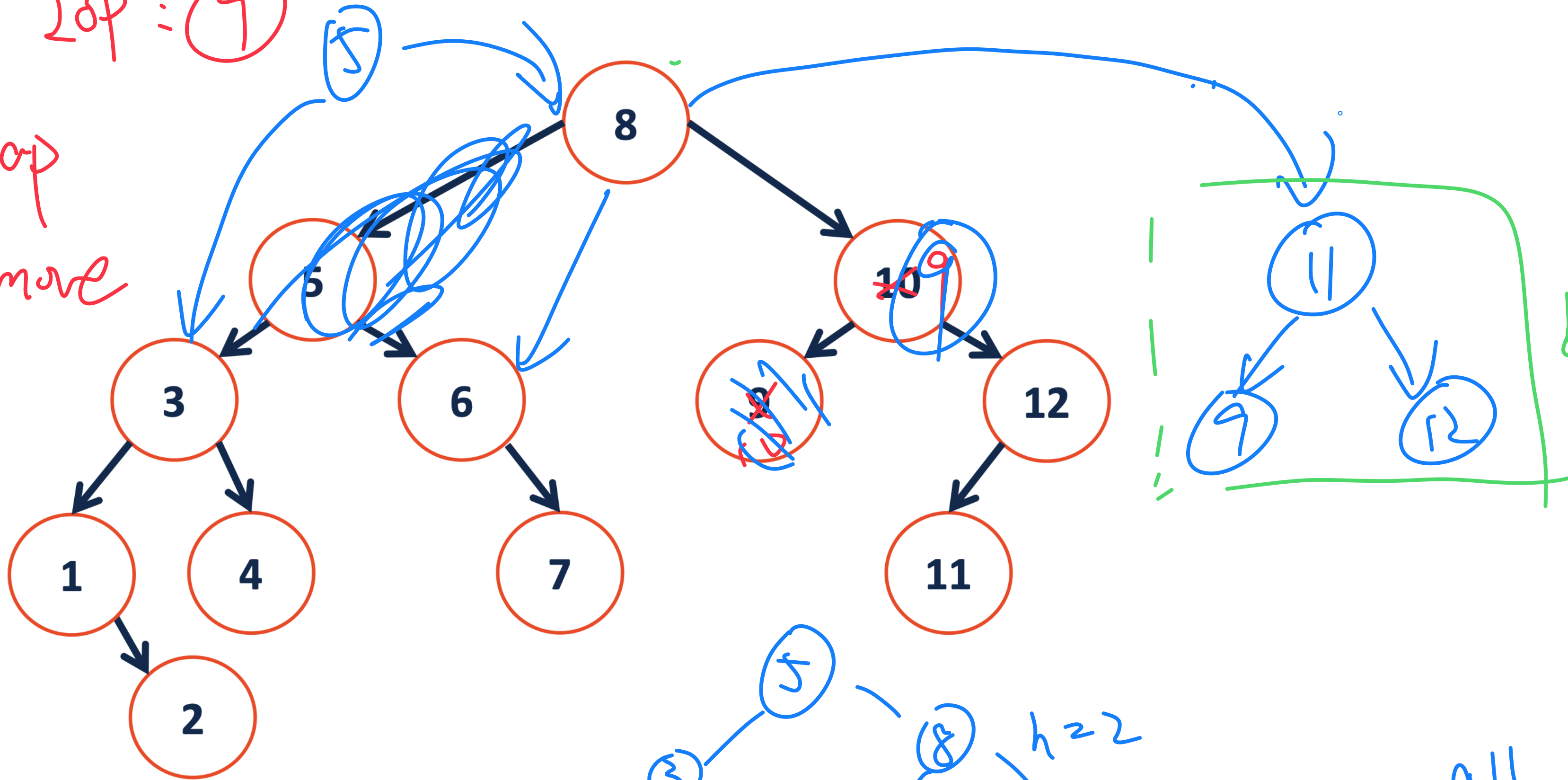
Rotate

Remove 10

① Find IOP: 9

② Swap

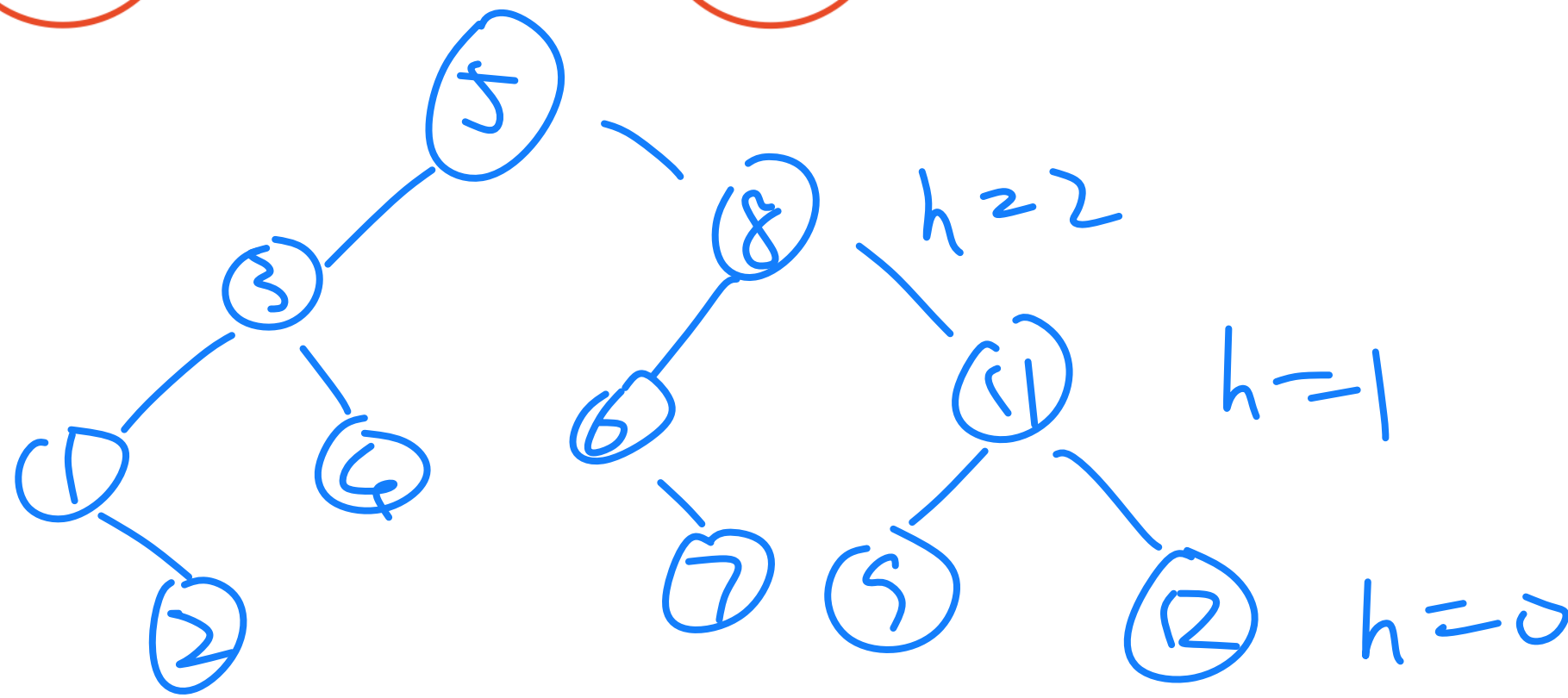
② to move



balance

all balance.

Rotate



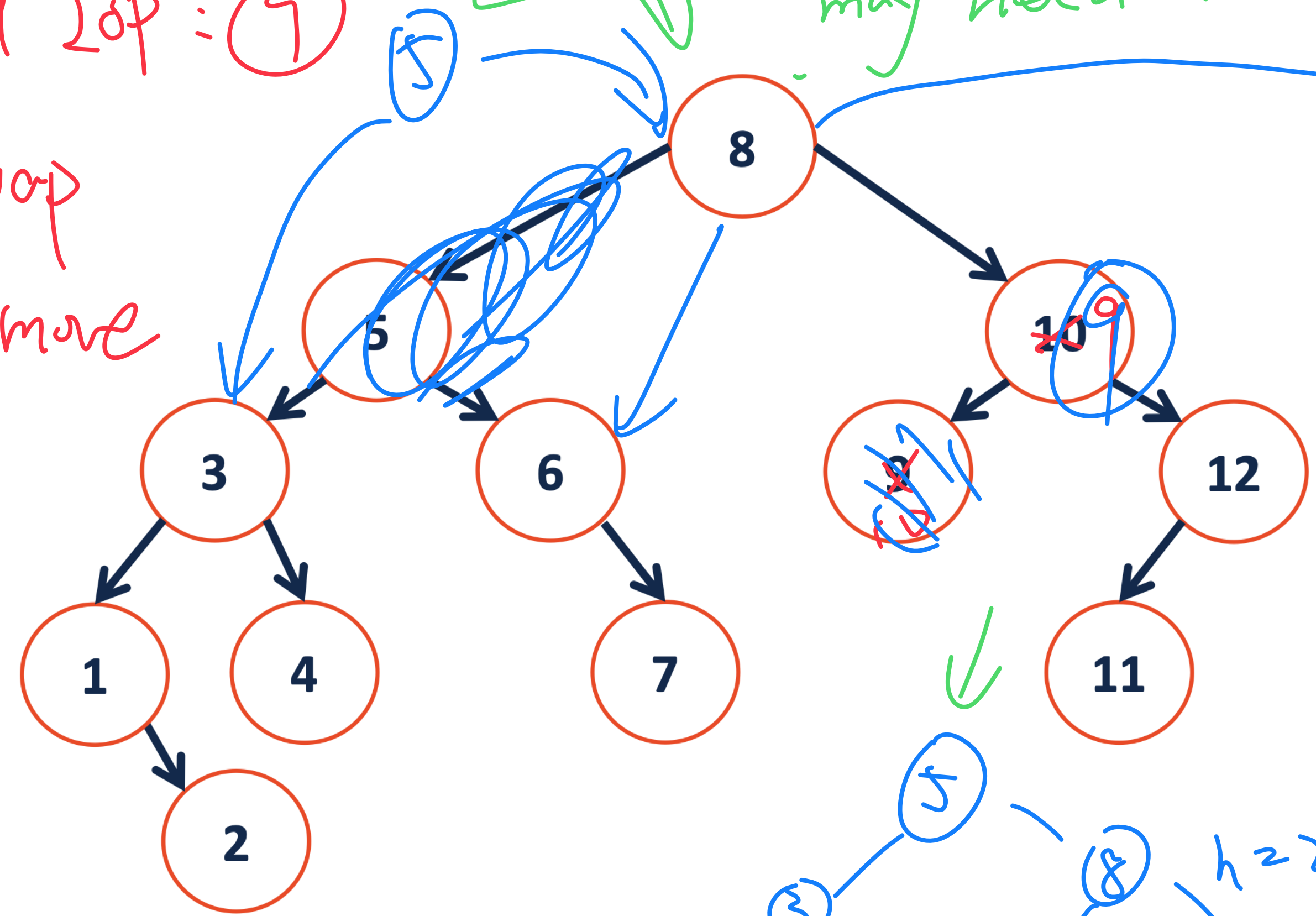
Remove 10

① Find IOP: 9

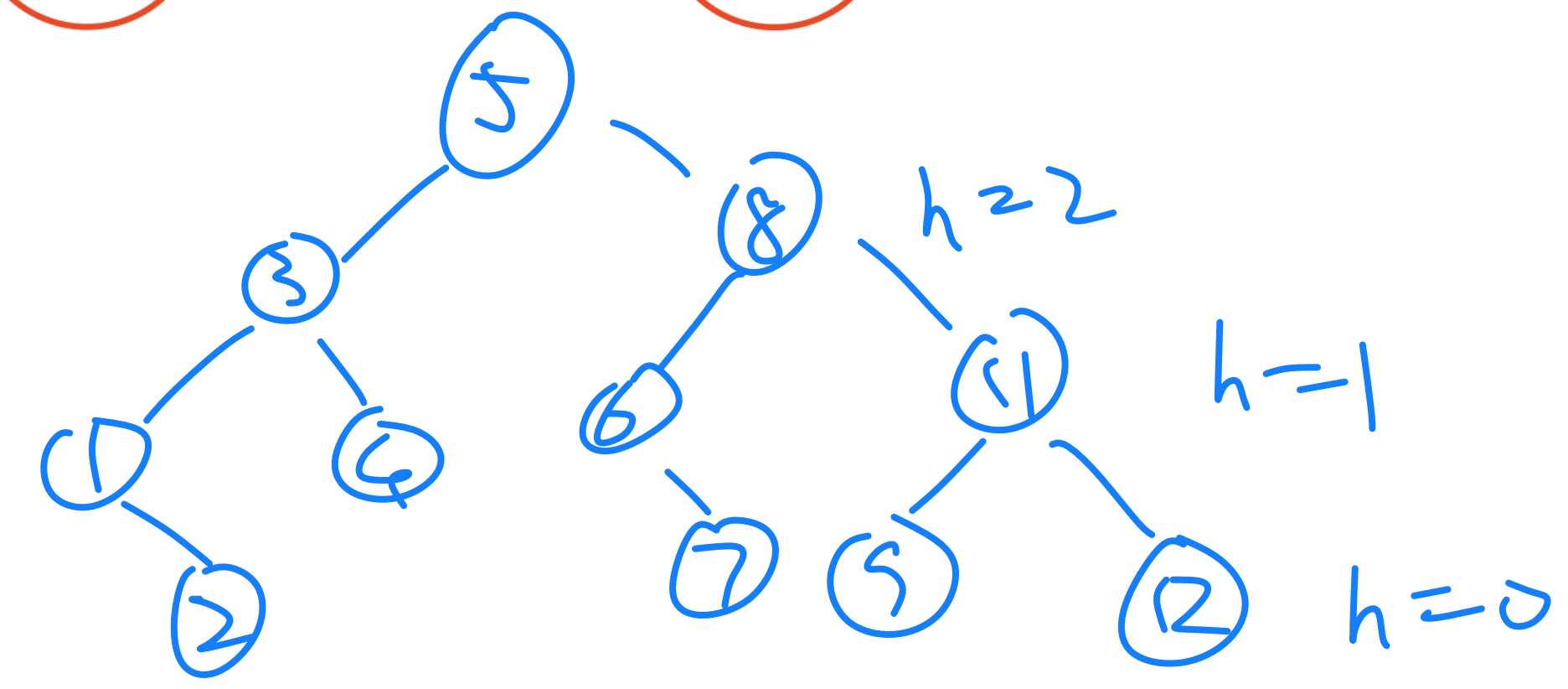
② Swap

③ Remove

if 8 is just a subtree of a bigger tree, we may need to rebalance all part until root!



Rotate



all balance.

Remove 10

① Find IOP: 9

② Swap

③ Remove

if 8 is just a subtree of a bigger tree, we may need to rebalance all part until root!

→ remove

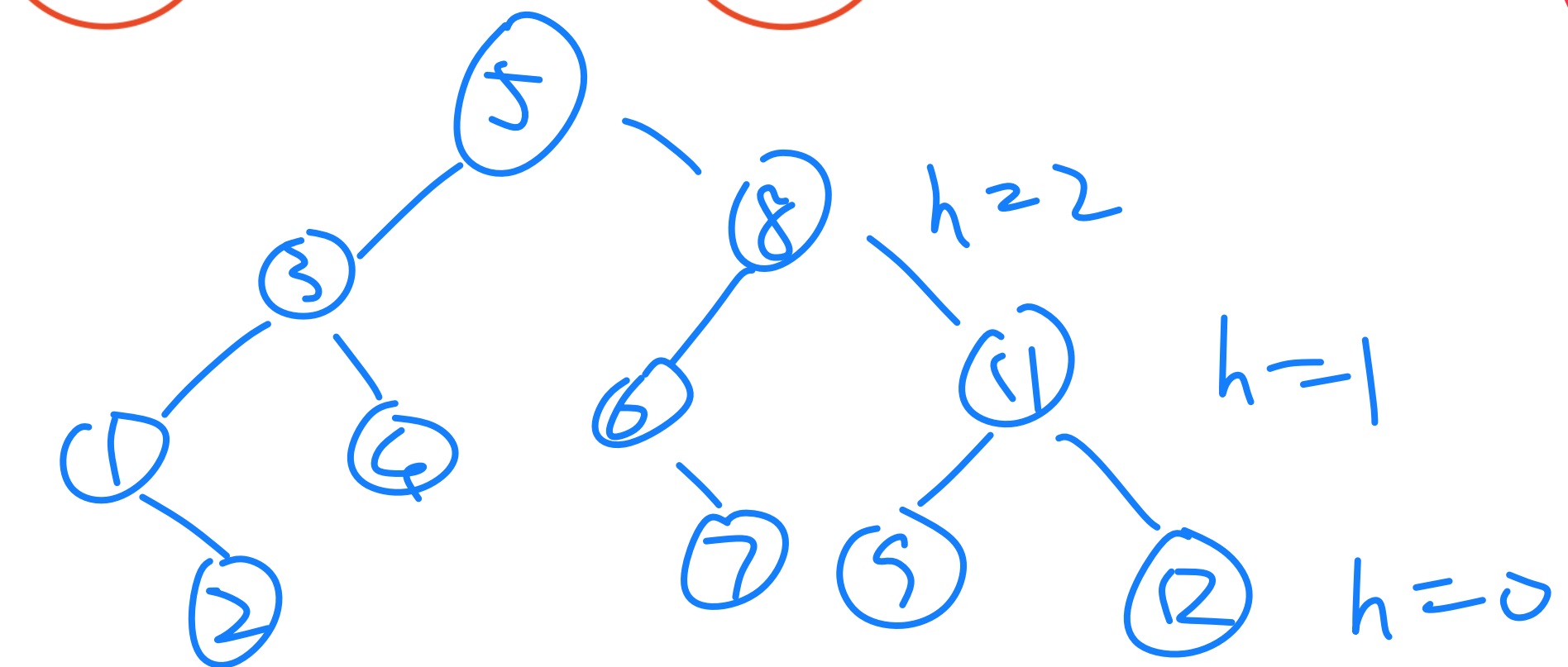
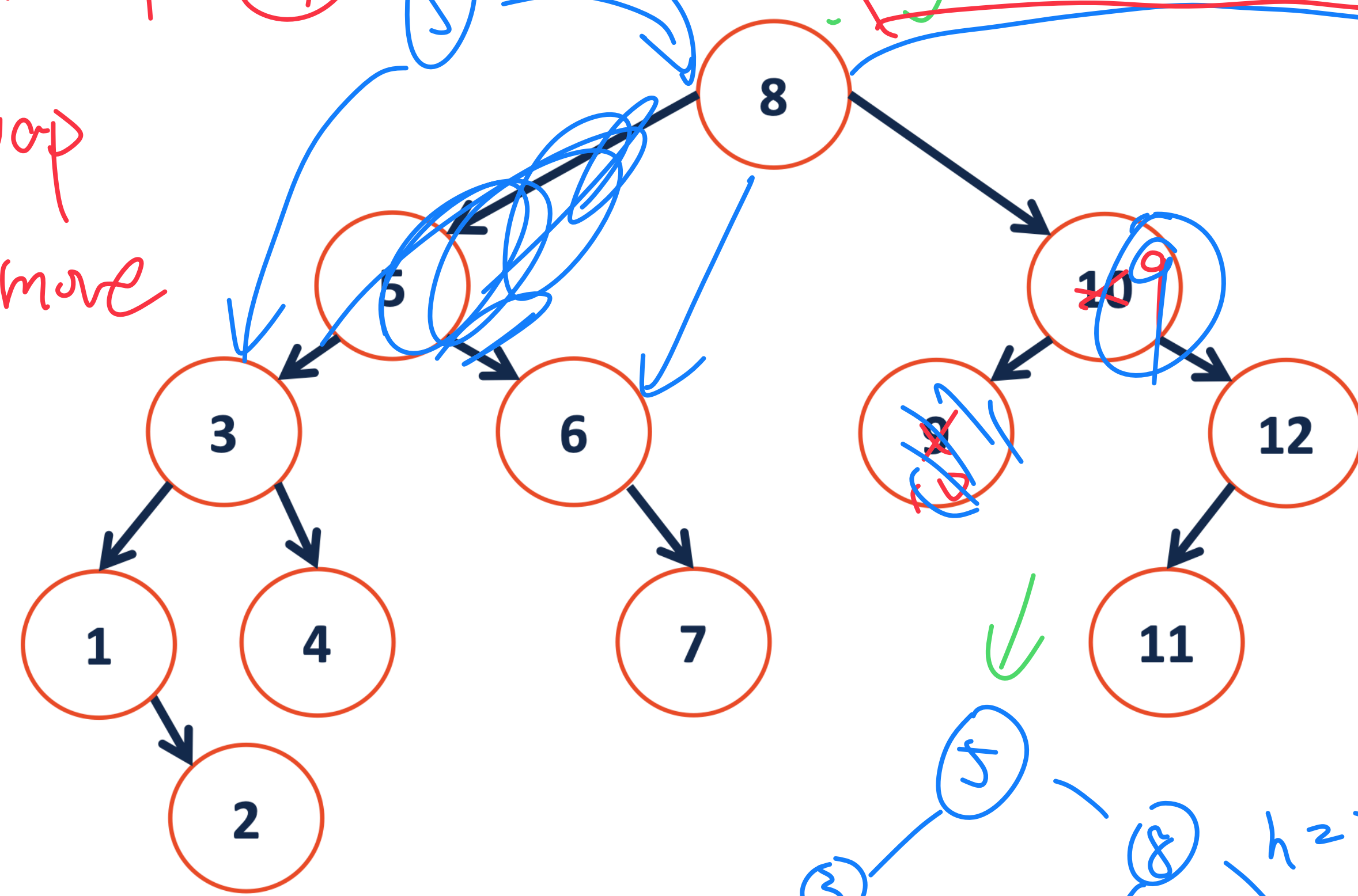
balance

Insert:

balance the exact place

all balance.

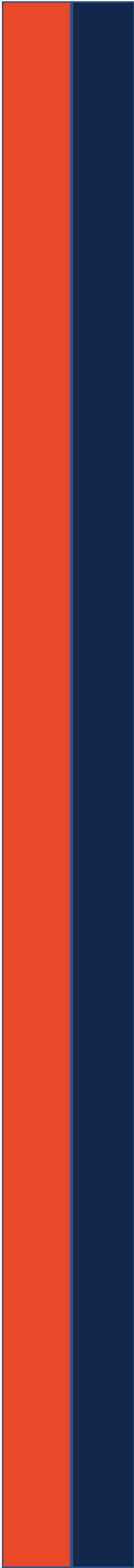
Rotate



AVL Tree Analysis

We know: insert, remove and find runs in: $O(h)$.

We will argue that: $h =$ $O(\log n)$.



CS 225

Data Structures

Volodymyr Kindratenko

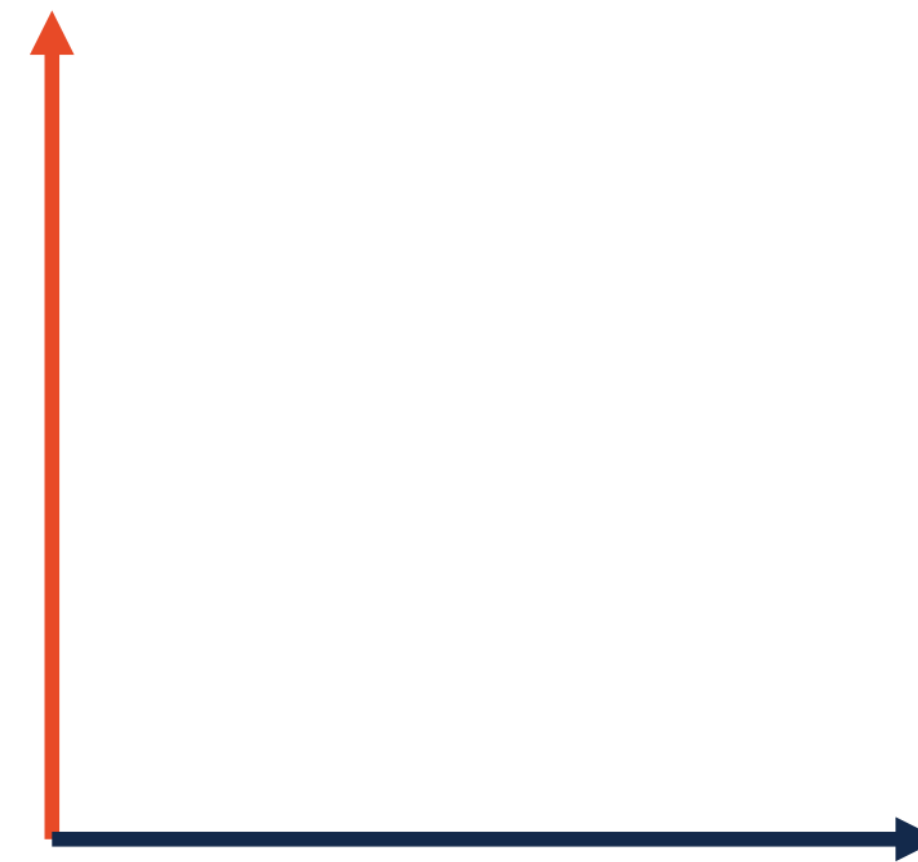
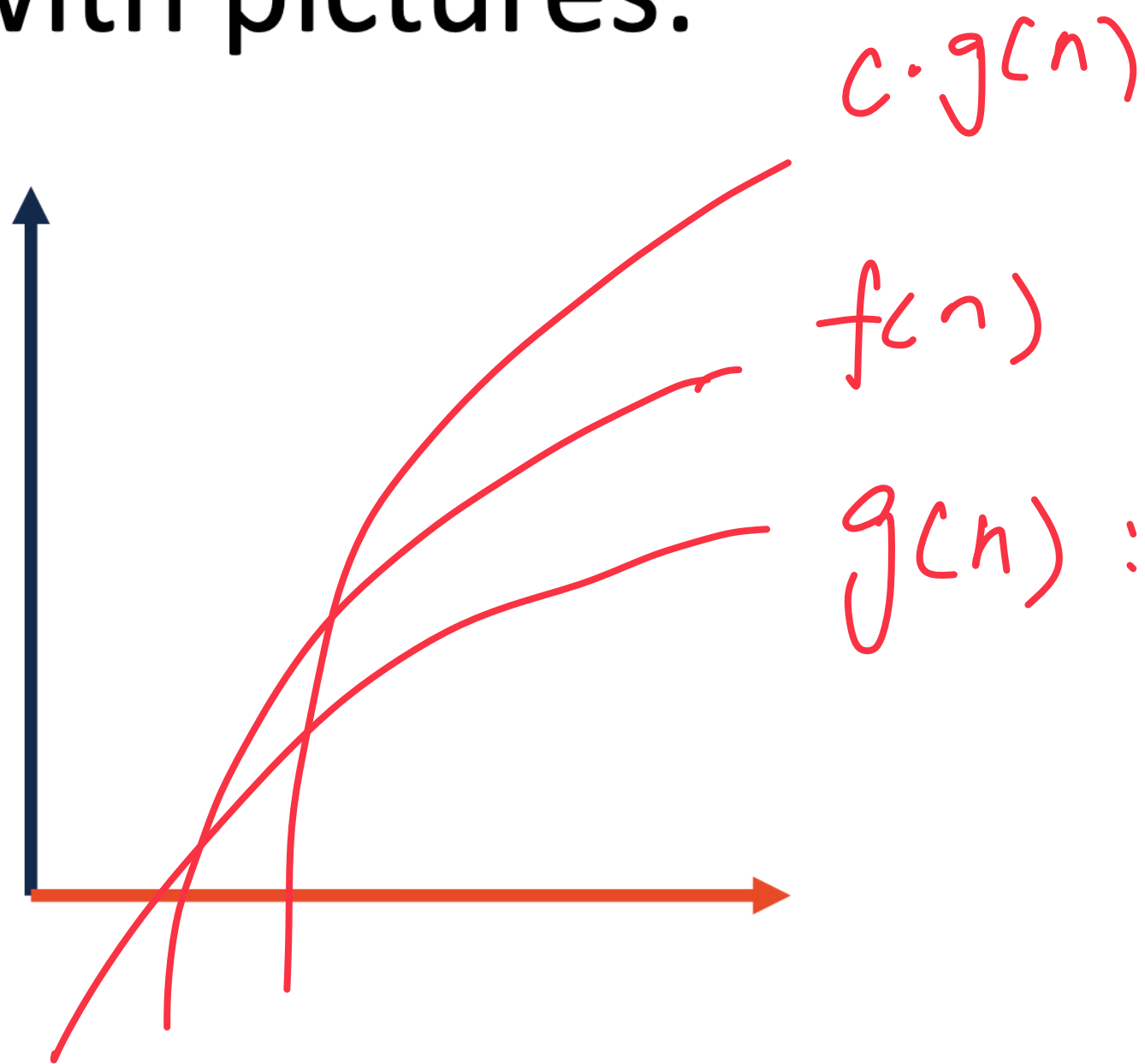
Slides courtesy of Wade Fagen-Ulmschneider

AVL Tree Analysis

Definition of big-O:

$f(n) = O(g(n))$, iff $\exists c, k$, s.t.: $f(n) \leq c \cdot g(n)$, $\forall n > k$

...or, with pictures:

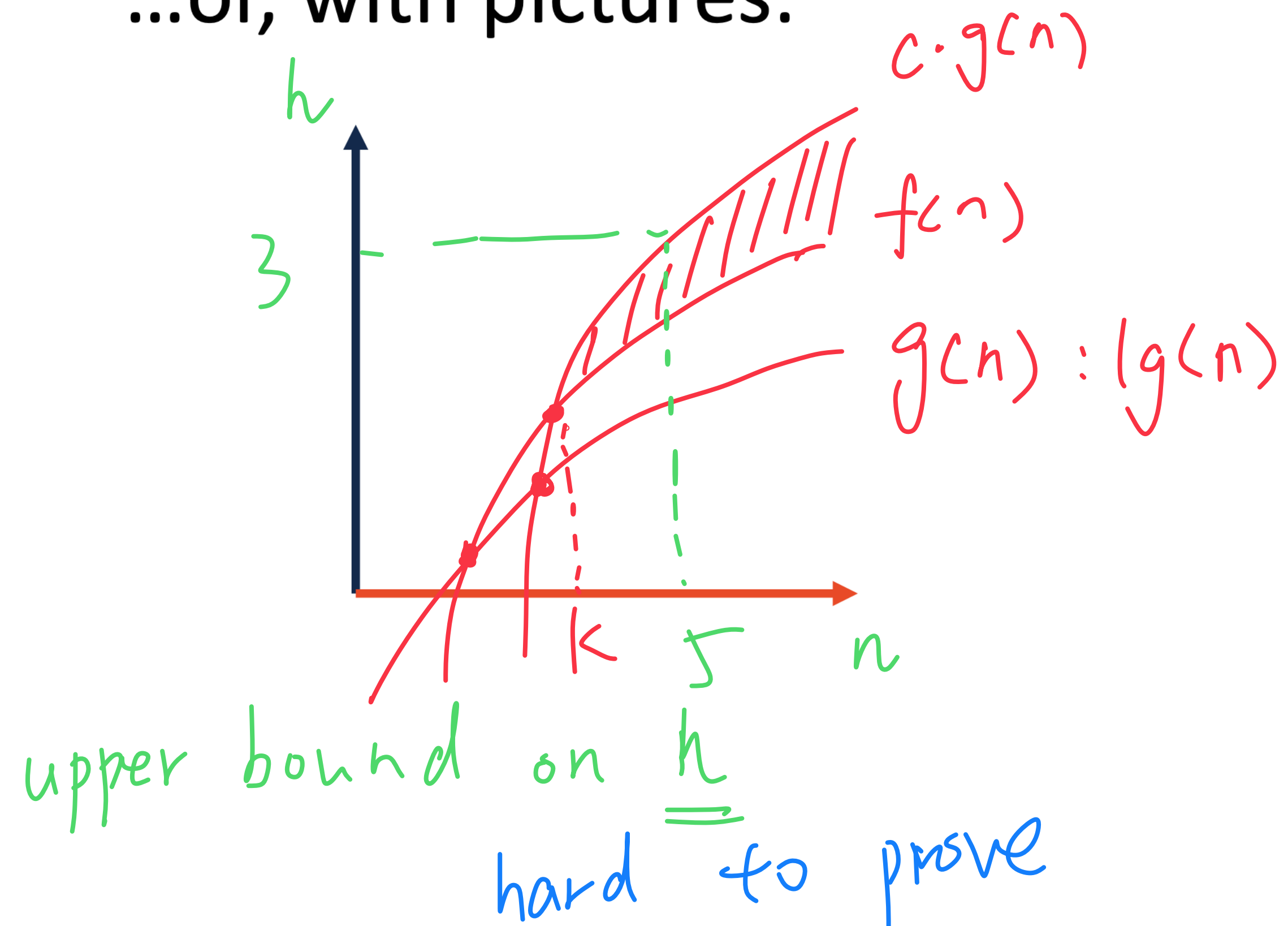


AVL Tree Analysis

Definition of big-O:

$f(n) = O(g(n))$, iff $\exists c, k$, s.t.: $f(n) \leq c \cdot g(n)$, $\forall n > k$
for $n=5$, h must ≤ 3

...or, with pictures:

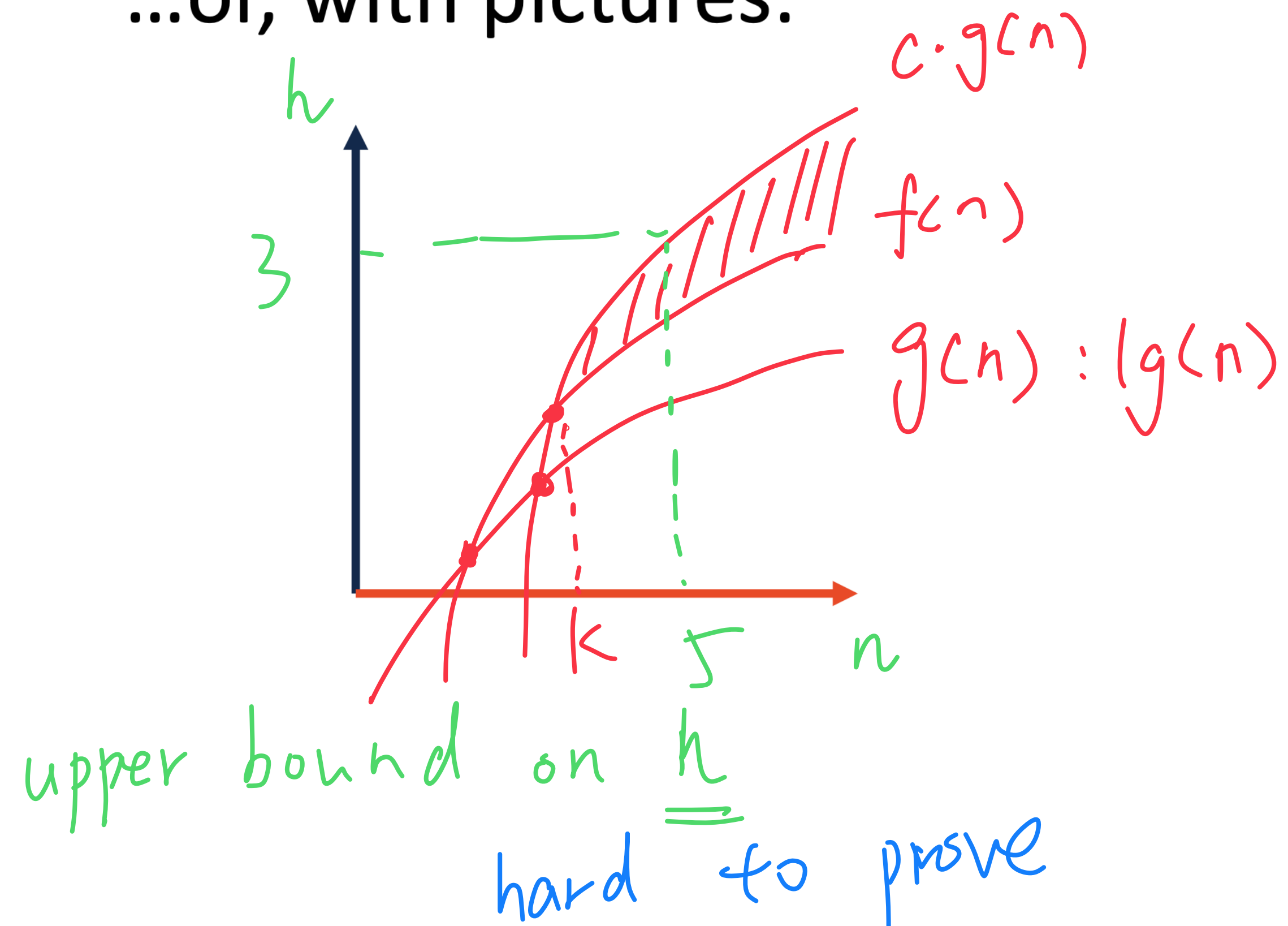


AVL Tree Analysis

Definition of big-O:

$f(n) = O(g(n))$, iff $\exists c, k$, s.t.: $f(n) \leq c \cdot g(n)$, $\forall n > k$
for $n=5$, h must ≤ 3

...or, with pictures:

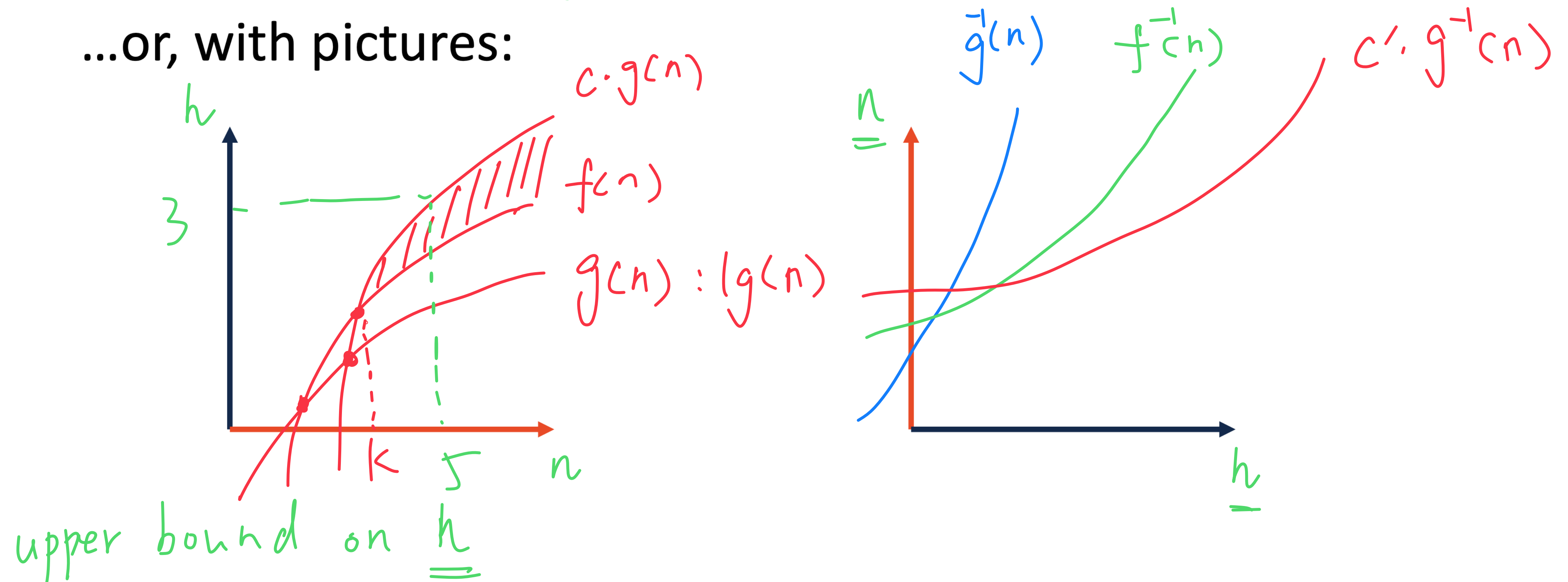


AVL Tree Analysis

Definition of big-O:

$f(n) = O(g(n))$, iff $\exists c, k$, s.t.: $f(n) \leq c \cdot g(n)$, $\forall n > k$
for $n=5$, h must ≤ 3

...or, with pictures:

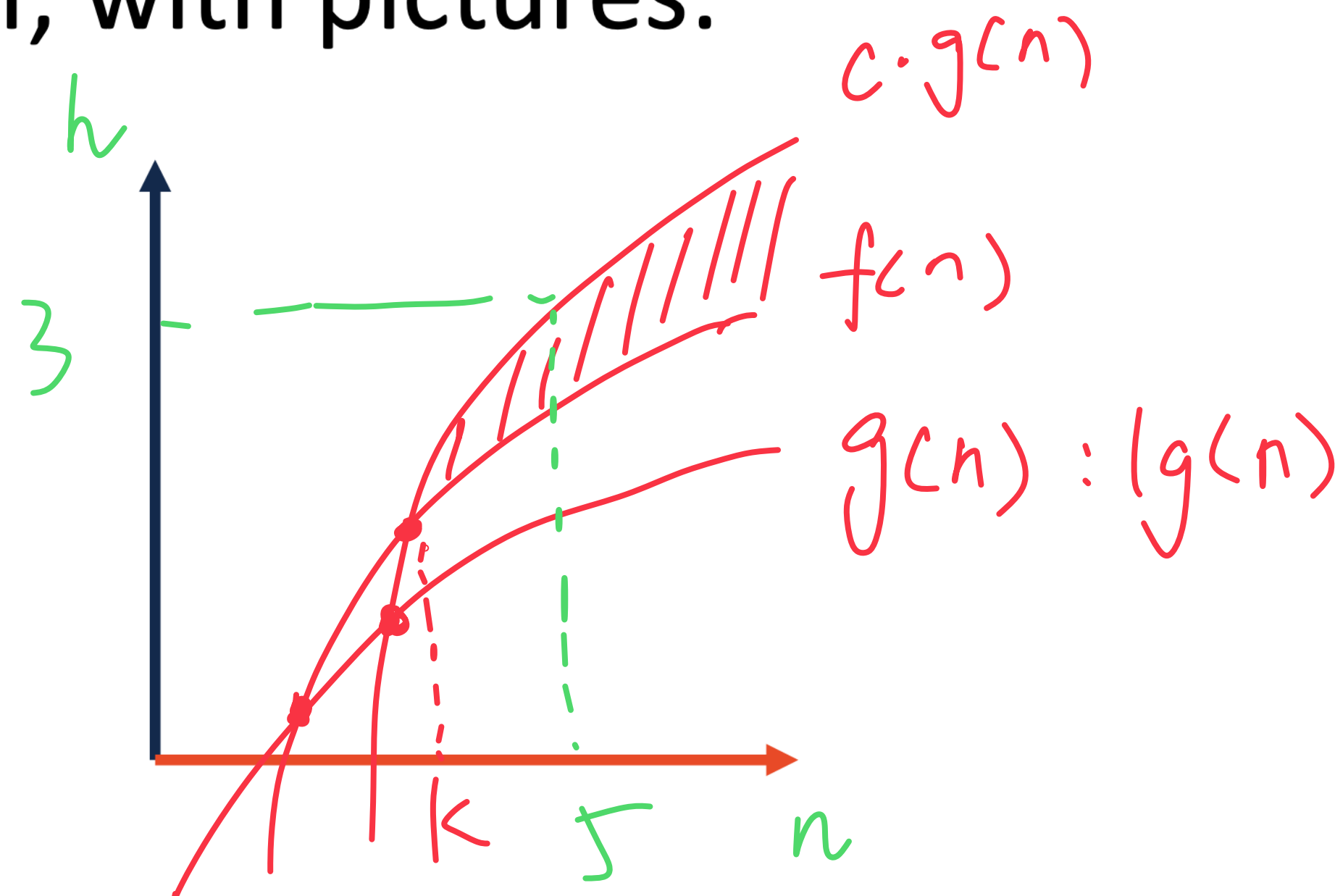


AVL Tree Analysis

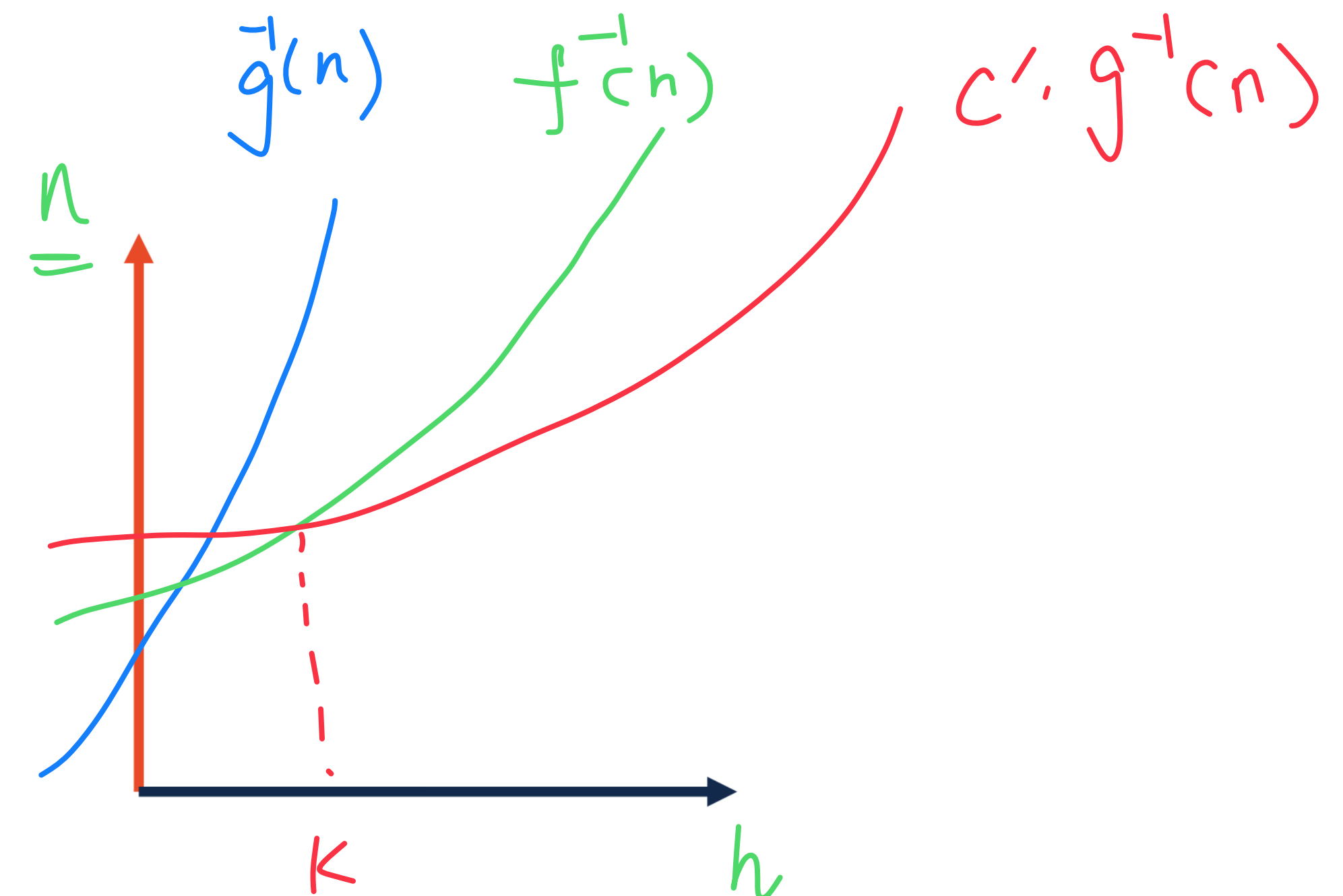
Definition of big-O:

$f(n) = O(g(n))$, iff $\exists c, k$, s.t.: $f(n) \leq c \cdot g(n)$, $\forall n > k$
 for $n = 5$, h must ≤ 3

...or, with pictures:



upper bound on h



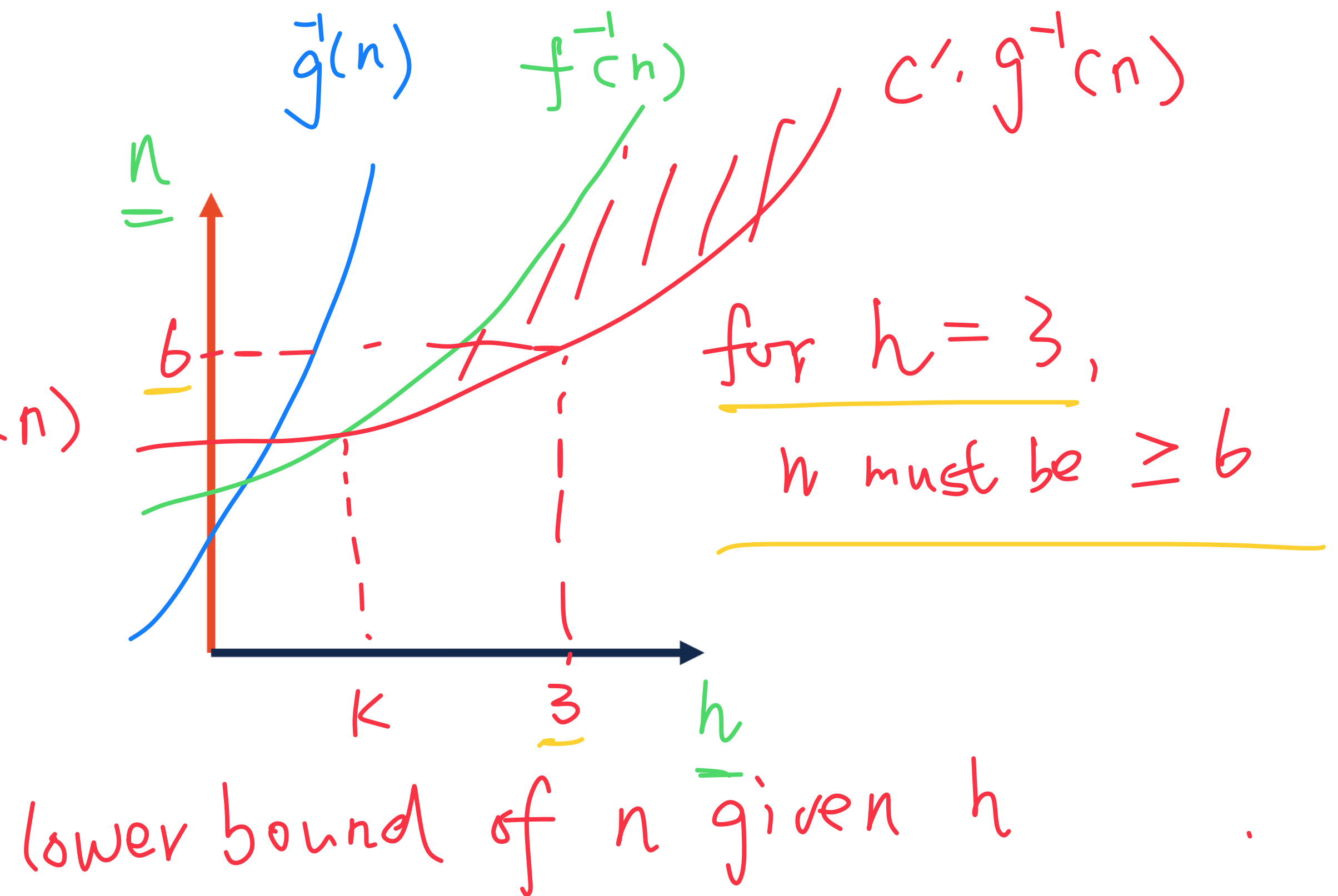
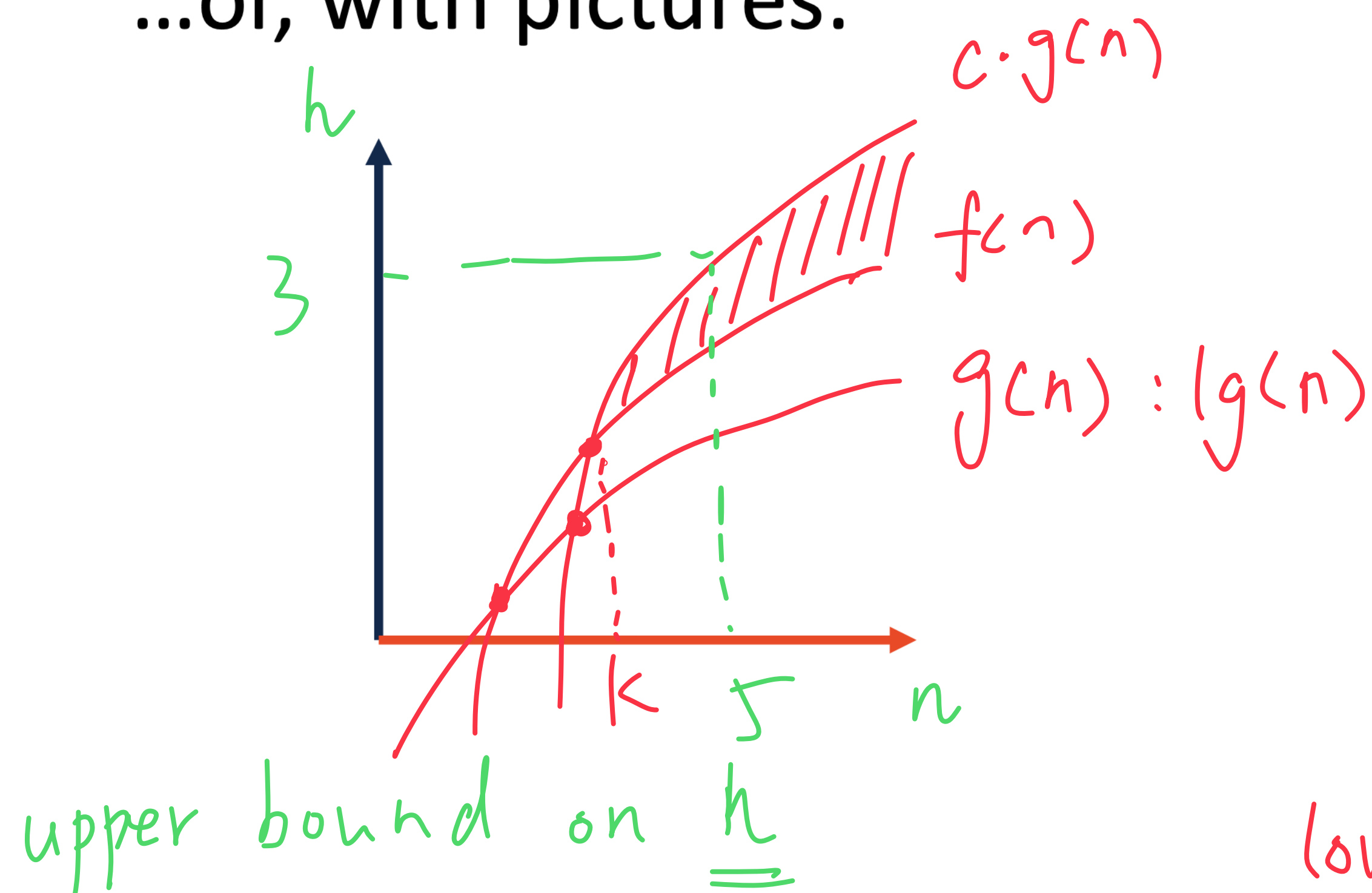
lower bound of n given h

AVL Tree Analysis

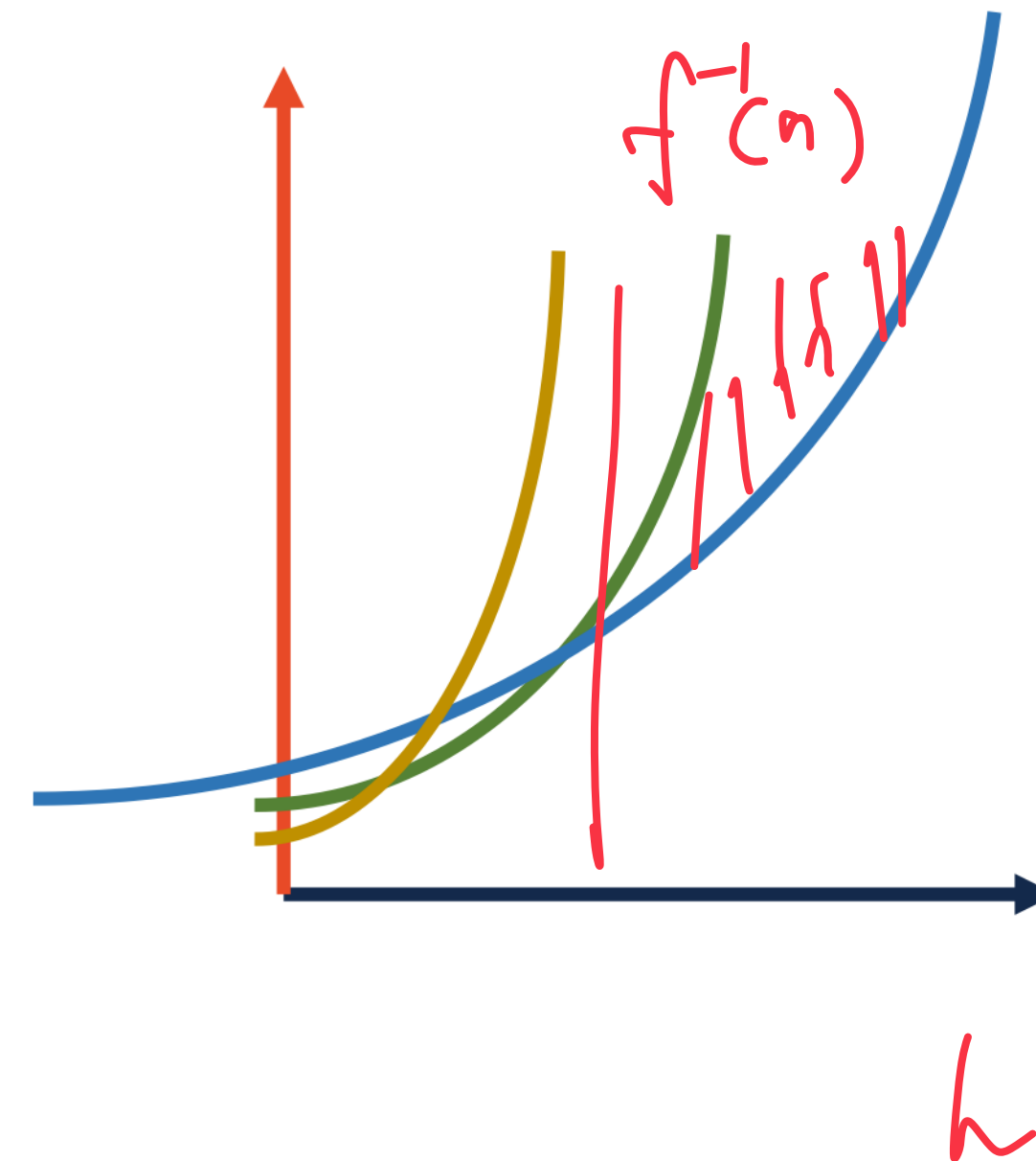
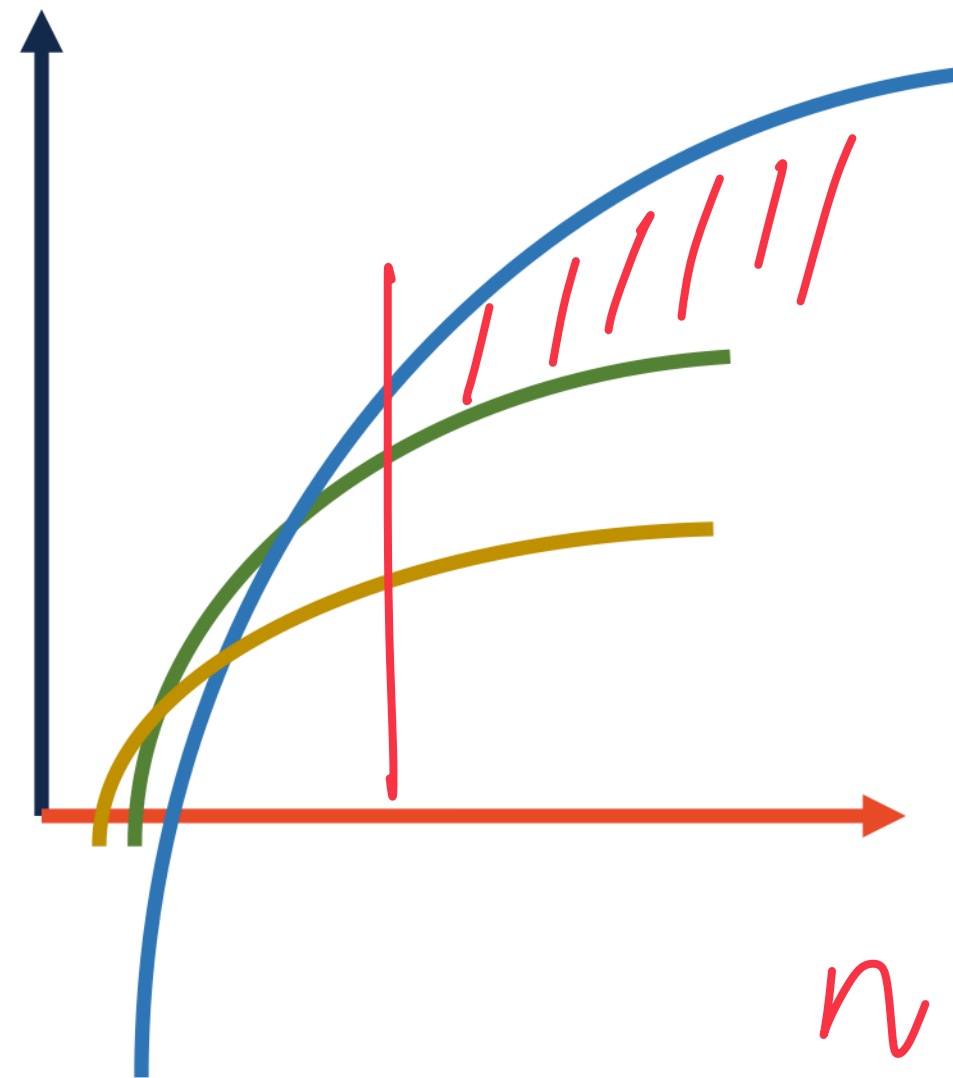
Definition of big-O:

$f(n) = O(g(n))$, iff $\exists c, k$, s.t.: $f(n) \leq c \cdot g(n)$, $\forall n > k$
 for $n=5$, h must ≤ 3

...or, with pictures:



AVL Tree Analysis



An upper bound on the height h for a tree of n nodes
...is the same as...

A lower bound on the number of nodes n in a tree of height h

Plan of Action

Begin by defining a function that defines the least number of nodes in an AVL tree of height **h**.

$f^{-1}(n) =$

N(h): The least number of nodes in an AVL tree of height **h**.

$$N(-1) = 0 \quad \{ \}$$

$N(h) ??$

$$N(0) = 1 \quad \circ$$

$$N(1) = 2 \quad \begin{array}{c} \circ \\ \swarrow \searrow \\ \circ \quad \circ \end{array}$$

$$N(2) = 4 \quad \begin{array}{c} \circ \\ \swarrow \searrow \\ \circ \quad \circ \\ \swarrow \searrow \quad \swarrow \searrow \\ \circ \quad \circ \quad \circ \quad \circ \end{array}$$

Plan of Action

Begin by defining a function that defines the least number of nodes in an AVL tree of height **h**.

$f^{-1}(n) =$

N(h): The least number of nodes in an AVL tree of height **h**.

$$N(-1) = 0 \quad \{ \}$$

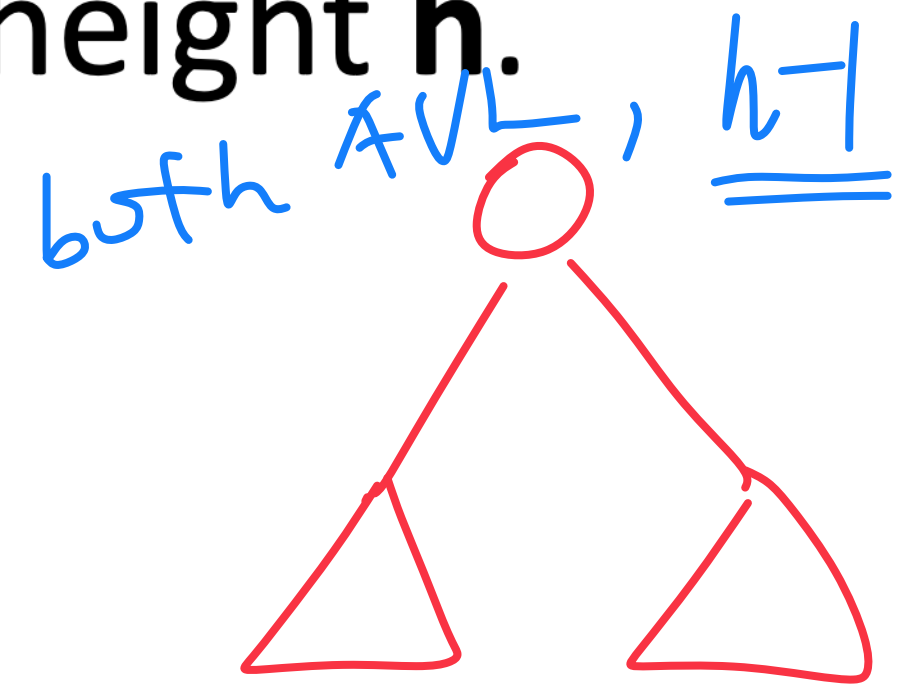
$$N(0) = 1 \quad \circ$$

$$N(1) = 2 \quad \begin{array}{c} \circ \\ \swarrow \searrow \\ \circ \quad \circ \end{array}$$

$$N(2) = 4 \quad \begin{array}{c} \circ \\ \swarrow \searrow \\ \circ \quad \circ \\ \swarrow \searrow \quad \swarrow \searrow \\ \circ \quad \circ \quad \circ \quad \circ \end{array}$$

$N(h) ??$

$$N(h) = 1 + \underline{N(h-1) + N(h-1)}$$



Plan of Action

Begin by defining a function that defines the least number of nodes in an AVL tree of height h .

$f^{-1}(n) =$

$N(h)$: The least number of nodes in an AVL tree of height h .

$$N(-1) = 0 \quad \{ \}$$

$$N(0) = 1 \quad \circ$$

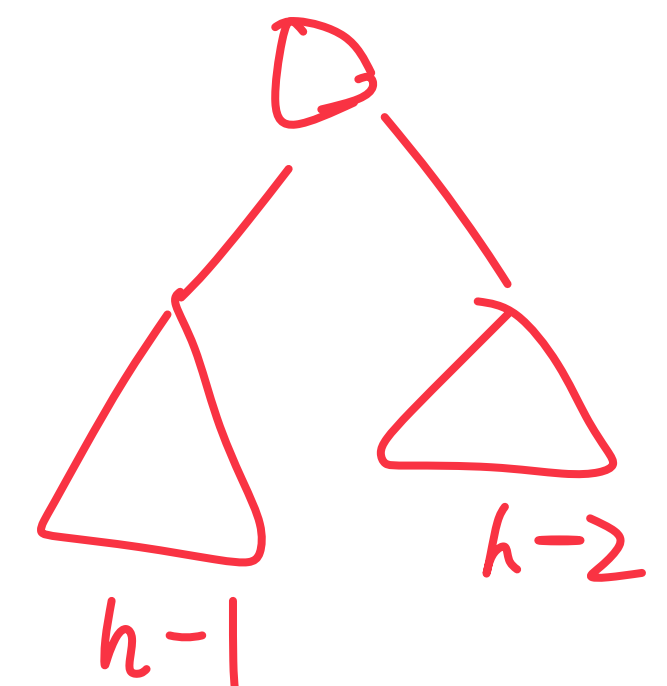
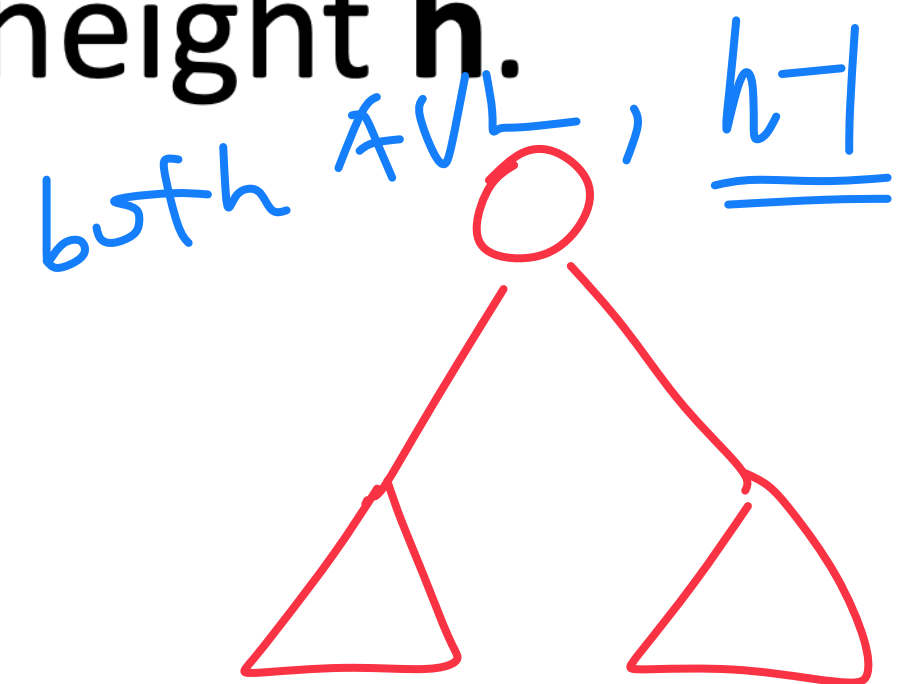
$$N(1) = 2 \quad \begin{array}{c} \circ \\ \swarrow \searrow \\ \circ \quad \circ \end{array}$$

$$N(2) = 4 \quad \begin{array}{c} \circ \\ \swarrow \searrow \\ \circ \quad \circ \\ \swarrow \searrow \swarrow \searrow \\ \circ \quad \circ \quad \circ \quad \circ \end{array}$$

$$N(h) ??$$

$$N(h) = 1 + \underline{N(h-1) + N(h-1)}$$

$$N(h) = 1 + N(h-1) + N(h-2)$$



Plan of Action

Begin by defining a function that defines the least number of nodes in an AVL tree of height h .

$f^{-1}(n) =$

$N(h)$: The least number of nodes in an AVL tree of height h .

$$N(-1) = 0 \quad \{\}$$

$$N(0) = 1 \quad \circ$$

$$N(1) = 2 \quad \begin{array}{c} \circ \\ \swarrow \searrow \\ \circ \quad \circ \end{array}$$

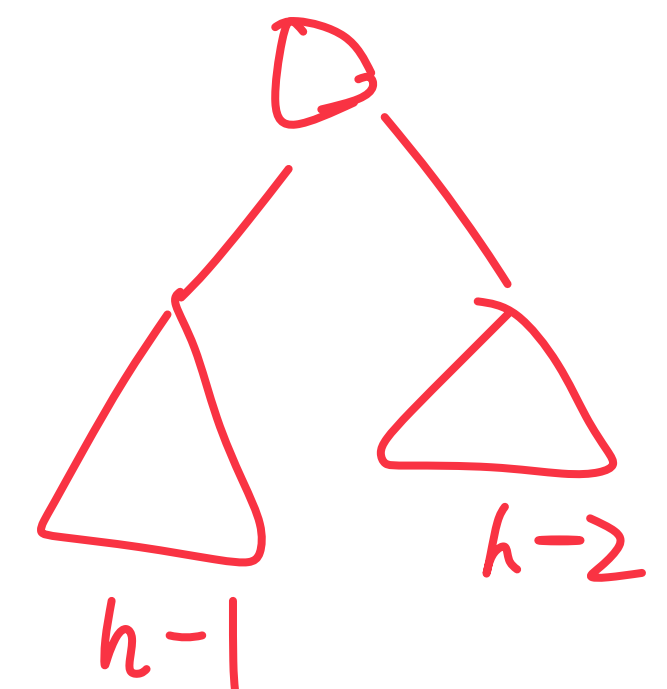
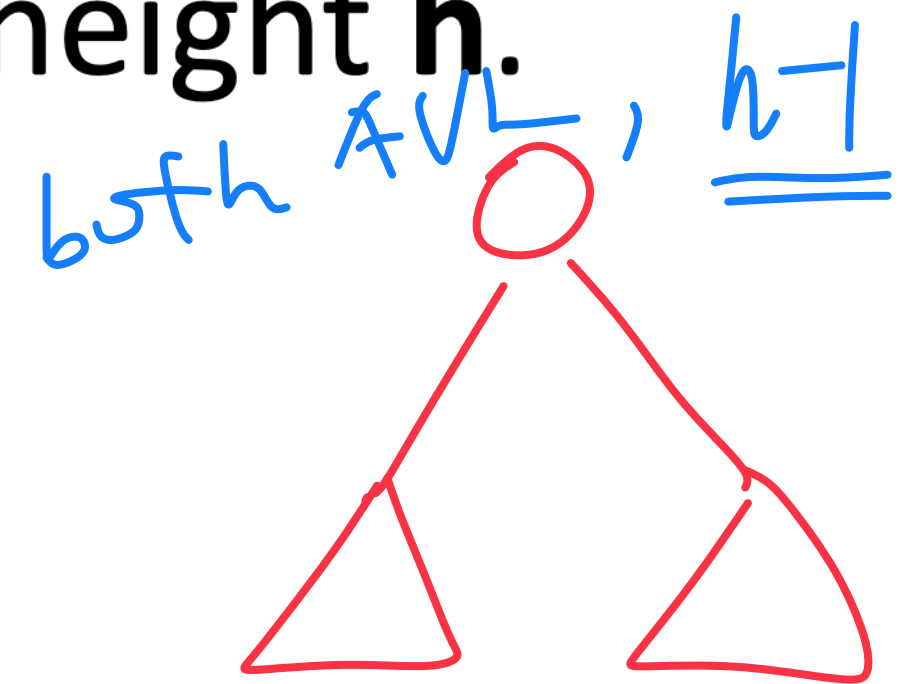
$$N(2) = 4 \quad \begin{array}{c} \circ \\ \swarrow \searrow \\ \circ \quad \circ \\ \swarrow \searrow \swarrow \searrow \\ \circ \quad \circ \quad \circ \quad \circ \end{array}$$

$$N(h) ??$$

$$N(h) = 1 + \underline{N(h-1) + N(h-1)}$$

$$N(h) = 1 + N(h-1) + N(h-2)$$

$$\begin{aligned} N(2) &= 1 + N(1) + N(0) = 1 + 2 + 1 \\ &= 4 \end{aligned}$$



Simplify the Recurrence

$$N(h) = 1 + N(h - 1) + N(h - 2)$$

$$> N(h-1) + N(h-2)$$

$$> 2 \cdot N(h-2), \quad h \geq 1, \quad N(h-1) > N(h-2)$$

Everytime we double the value :

2	2	4	4	8	8	16	16	32	32	...
h=0	h=1	h=2	h=3	h=4	h=5	h=6	h=7	h=8	h=9	...

Simplify the Recurrence

$$N(h) = 1 + N(h - 1) + N(h - 2)$$

$$> N(h-1) + N(h-2)$$

$$> 2 \cdot N(h-2), \quad h \geq 1, \quad N(h-1) > N(h-2)$$

Everytime we double the value :

2	2	4	4	8	8	16	16	32	32	...
h=0	h=1	h=2	h=3	h=4	h=5	h=6	h=7	h=8	h=9	...

$$\underline{\underline{> 2^{h/2}}}$$

Guess

State a Theorem

Theorem: An AVL tree of height h has at least $2^{h/2}$ nodes.

Proof:

I. Consider an AVL tree and let h denote its height.

II. Case: $h \geq 1$

$$N(1) = 2, \quad 2^{h/2} = \sqrt{2}$$

True?? ✓

at least $\sqrt{2}$, $z > \sqrt{2}$
actual: z ;

An AVL tree of height _____ has at least _____ nodes.

State a Theorem

Theorem: An AVL tree of height h has at least $2^{h/2}$ nodes.

Proof:

I. Consider an AVL tree and let h denote its height.

II. Case: $h=1$

$$N(1) = 2, \quad 2^{h/2} = \sqrt{2}$$

True?? ✓

at least $\sqrt{2}$, $z > \sqrt{2}$
actual: 2 ;

no fraction of
a node

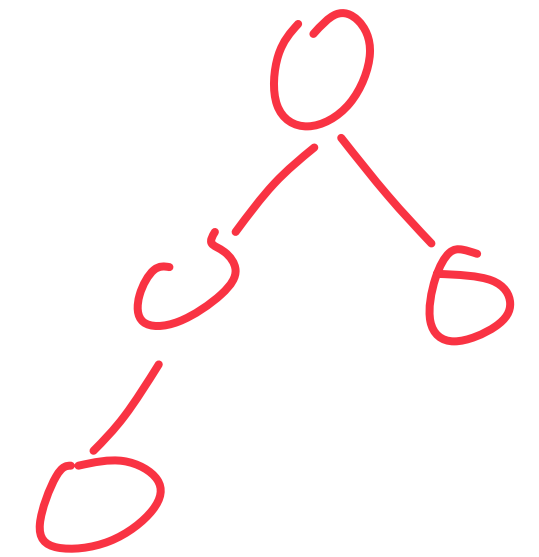
An AVL tree of height 1 has at least $\sqrt{2}$ nodes.
just lower bound

Prove a Theorem

III. Case: $h=2$

$$\text{Actual: } N(2) = 1 + N(h-2) + N(h-1) = 1 + 2 + 1 = 4$$

$$\text{Theorem: } 2^{h/2} = 2^{2/2} = 2^1 = 2$$



An AVL tree of height 2 has at least 2 nodes.

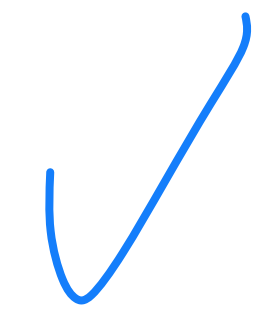
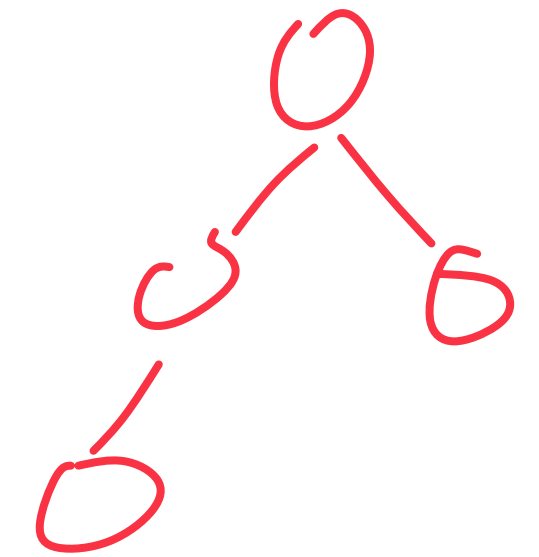
Prove a Theorem

III. Case: $h=2$

Actual: $N(2) = 1 + N(h-2) + N(h-1) = 1 + 2 + 1 = 4$

Theorem: $2^{h/2} = 2^{2/2} = 2^1 = 2$

$4 \geq 2$, at least 2



An AVL tree of height 2 has at least 2 nodes.

Prove a Theorem

IV. Case: h

By an Inductive Hypothesis (IH):

$$\forall j < h, N[j] > 2^{j/2}$$

We will show that:

An AVL tree of height _____ has at least _____ nodes.

Prove a Theorem

IV. Case: h

By an Inductive Hypothesis (IH):

$$\forall j < h, N(j) > 2^{j/2}$$

We will show that:

$$\begin{aligned} N(h) &> 2 \cdot N(h-2) > 2 \cdot 2^{(h-2)/2} \\ &= 2^{h/2 - 1 + 1} \\ &= 2^{h/2} \end{aligned}$$

An AVL tree of height h has at least $2^{h/2}$ nodes.



Prove a Theorem

V. Using a proof by induction, we have shown that:

$$N(h) > 2^{h/2}$$

...and inverting:

$$- n \geq N(h) > 2^{h/2}$$

$$- n > 2^{h/2}$$

$$- \log n > h/2 \Rightarrow 2 \cdot \log n > h$$

$$\Rightarrow h < 2 \cdot \log n,$$

Prove a Theorem

V. Using a proof by induction, we have shown that:

$$N(h) > 2^{h/2}$$

...and inverting:

$$- n \geq N(h) > 2^{h/2}$$

$$- n > 2^{h/2}$$

$$- \log n > h/2 \Rightarrow 2 \cdot \log n > h$$

$$\Rightarrow h < 2 \cdot \log n,$$

$$h = O(\log n)$$

$C=2$ here

Summary of Balanced BST

Red-Black Trees

- Max height: 2 * lg(n)
- Constant number of rotations on insert, remove, and find

AVL Trees

- Max height: 1.44 * lg(n)
- Rotations:

Insert : 1 rotation :

Find : 0 rotation

Remove : $O(\lg n)$

$\text{const} \cdot \lg(n)$

Summary of Balanced BST

Pros:

- Running Time: $O(\lg n)$
- Improvement Over: *array, list*
- Great for specific applications:
 - *range finding*
 - *Nearest neighbor (kd-trees)*

Summary of Balanced BST

Cons:

- Running Time:

faster ways to find a key (hashing)

- In-memory Requirement:

AVLs only work if store in memory.

if data in disk $\Rightarrow$ B Tree

Thanks!